

CP Teoria

Ricardo Rojas Carvajal

August 2026

Contents

1	Plantillas y utilidades	3
2	Matematicas	3
2.1	Teoria de numeros	3
2.1	Extended Euclidean Algorithm	3
2.2	Formulas utiles	5
2.3	Criba de Eratostenes	6
2.4	Criba lineal	7
2.5	Miller-Rabin primality	9
2.6	Pollard's Rho + factorizacion	10
2.7	Modular exponentiation	12
2.8	Modular inverse	13
2.9	Chinese Remainder Theorem	15
2.10	Euler totient precompute	16
2.11	Mobius precompute	17
2.12	Stern-Brocot (best fraction in interval)	19
2.2	Combinatoria	20
2.1	Catalan (formula)	20
2.2	Factoriales + $nCr \bmod p$	21
2.3	Lucas theorem	23
2.4	Stirling S2 (particiones)	24
2.5	Inclusion-exclusion over subsets	25
2.3	Polinomios y transformadas	27
2.1	FFT (iterativo, complex)	27
2.2	NTT (mod 998244353)	29
2.3	Polynomial utilities (mod p)	30
2.4	Algebra lineal	32
2.1	Gauss-Jordan (mod p)	32
2.2	Matrix exponentiation	35
2.5	Juegos y teoria de juegos	36
2.1	Sprague-Grundy template	36
2.2	Small game tables (Kayles SG values)	38
3	Estructuras de datos	39
3.1	Ordenado y contenedores	39
3.1	Ordered Set (pbds)	39
3.2	Basis (XOR basis)	40
3.3	Trie	41
3.2	Union-Find	43
3.1	DSU classico	43
3.2	DSU con rollback	45
3.3	Arboles de segmentos	46
3.1	Segment Tree Add + Sum	46
3.2	Segment Tree Lazy Assign + Sum	48

3.3	Segment Tree Add + lower bound index	50
3.4	Segment Tree Lazy Add + Set + Sum	52
3.5	Segment Tree iterativo	54
3.6	Segment Tree 2D	55
3.7	Segment Tree persistente	57
3.4	Otras	59
3.1	MO	59
3.5	Fenwick / BIT	60
3.1	BIT point update + prefix sum	60
3.2	BIT range add + point query	61
3.3	BIT range add + range sum	63
3.6	RMQ y sparse table	64
3.1	Sparse Table	64
3.2	Disjoint Sparse Table	66
3.7	Trees y DP en arbol	68
3.1	Binary Lifting / LCA	68
3.2	Heavy-Light Decomposition (HLD)	70
3.3	Centroid Decomposition	71
3.4	Euler Tour + range queries	73
3.8	BST balanceado	75
3.1	Treap implicito	75
3.9	Miscelaneas de estructuras	76
3.1	Mo con updates	76
3.2	sqrt-decomposition generico	78
4	Strings	80
4.1	Hashing	80
4.1	Rolling Hash (doble modulo)	80
4.2	Patrones	81
4.1	KMP + LPS	81
4.2	Z-algorithm	83
4.3	Rabin-Karp (single hash)	85
4.4	Aho-Corasick (basic)	86
4.5	Aho-Corasick DP for forbidden patterns	88
4.3	Estructuras avanzadas	90
4.1	Suffix Array	90
4.2	LCP Array (Kasai)	91
4.3	Suffix Automaton	93
4.4	Lyndon factorization (Duval)	95
4.4	Palindromos	97
4.1	Manacher	97
5	Grafos	98
5.1	Caminos minimos	98
5.1	Dijkstra	98
5.2	Bellman-Ford	99
5.3	Floyd-Warshall	100
5.4	0-1 BFS	101
5.2	Componentes y cortes	102
5.1	Kosaraju SCC	102
5.2	Tarjan (puentes)	103
5.3	Tarjan (puntos de articulacion)	104
5.4	Tarjan SCC	105
5.3	Matching	106
5.1	Kuhn (bipartite matching)	106
5.2	Hopcroft-Karp	107
5.4	MST	108

5.1	Kruskal	108
5.2	Prim (priority queue)	109
5.5	Basicos	110
5.1	DFS / BFS / componentes	110
5.2	Topological sort (Kahn + DFS)	111
5.3	Bipartite check + 2-coloring	112
5.6	Flujos	113
5.1	Edmonds-Karp (max flow)	113
5.2	Dinic (max flow)	114
5.3	Min-Cost Max-Flow	115
5.7	Especiales	115
5.1	2-SAT (implication graph + SCC)	115
5.2	Euler tour / Hierholzer (camino y circuito)	116
5.3	Hamiltonian path / cycle (bitmask DP + backtracking)	117
6	DP	118
6.1	Knapsack 0/1	118
6.2	Knapsack unbounded	119
6.3	Knapsack meet-in-the-middle	120
6.4	LIS $O(n \log n)$	121
6.5	LCS / Edit distance	122
6.6	Bitmask DP over subsets	123
6.7	SOS DP	124
6.8	Digit DP template	124
6.9	Tree DP (reroot)	125
7	Geometria	126
7.1	Puntos y segmentos	126
7.1	Point + cross + dot	126
7.2	Interseccion de segmentos	127
7.3	Contains colineal	128
7.4	Distance point-segment / point-line	129
7.5	Projection of point onto line	130
7.2	Poligonos	131
7.1	Convex Hull (monotono)	131
7.2	Polygon Area	132
7.3	Point in polygon (ray casting)	132
7.4	Minkowski sum (convex polygons)	133
7.3	Herramientas	134
7.1	Li Chao Tree (long long)	134
7.2	Sweep line template	135
7.3	Closest pair of points	136
7.4	Rotating calipers (polygon diameter)	136
8	Miscelanea	137
8.1	Utilidades del usuario	137
8.1	Hora	137

1 Plantillas y utilidades

2 Matematicas

2.1 Teoria de numeros

2.1 Extended Euclidean Algorithm

Idea intuitiva. Dados dos enteros a y b no ambos cero, existen enteros x, y tales que $ax + by = \gcd(a, b)$. El algoritmo extendido de Euclides los construye **mientras** calcula el gcd, mediante recursion: si (x_1, y_1) resuelve $b \cdot x_1 + (a \bmod b) \cdot y_1 = \gcd$, entonces $x = y_1$ e $y = x_1 - y_1 \cdot \lfloor a/b \rfloor$ resuelve $ax + by = \gcd$. La existencia se demuestra por la identidad de Bézout: en cada paso, $\gcd(a, b) = \gcd(b, a \bmod b)$, y la recursion garantiza que siempre podemos “desenrollar” los coeficientes.

Propiedades clave (invariantes).

- Siempre termina (Euclides baja el segundo argumento).
- x puede ser negativo.
- Si $\gcd(a, b) = 1$, entonces x es el **inverso modular** de a modulo b .
- $|x|, |y| \leq \max(a, b)/2$ en valor absoluto (propiedad util para cotas).

Codigo clave. La recursion que arma la base del algoritmo:

```
long long extgcd(long long a, long long b, long long& x, long long& y) {
    if (b == 0) { x = (a >= 0 ? 1 : -1); y = 0; return abs(a); }
    long long x1, y1;
    long long g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}
```

Complejidad.

- $O(\log \min(a, b))$ operaciones (misma que el gcd de Euclides).
- Cada llamada hace una division y dos asignaciones.

Donde tocar para modificar.

- **Inversa modular:** la linea $x = y1; y = x1 - y1 * (a/b);$ resuelve el caso recursivo. Para usar como inversa modular con mod primo, despues ajustar al rango $[0, \text{mod})$.
- **Cuidado con negativos:** si b es negativo, ajustar la division entera.
- Si el problema requiere **muchas** inversas del mismo mod, conviene precomputar con el algoritmo de Fermat ($a^{p-2} \bmod p$) en lugar de llamar a extgcd por cada una.
- **Para ecuaciones diofanticas:** anadir la busqueda de la solucion general $x = x_0 + (b/g) \cdot t, y = y_0 - (a/g) \cdot t$.

Trampas y errores frecuentes.

- Si los numeros exceden `int`, cambiar a `long long`. El snippet retorna `int`; insuficiente para $a > 2^{31}$.
- $a \% b$ con a o b negativos puede dar resultados inesperados en distintos compiladores; ajustar antes de la recursion.
- Confundir a/b (truncado hacia cero en C++17+) con $\lfloor a/b \rfloor$; afecta la correccion si a es negativo.

Explicacion linea por linea. Funcion extgcd:

- `int extgcd(int a, int b, int &x, int &y)` – calcula $\gcd(a, b)$ y los coeficientes de Bézout; los retorna por referencia.
- `if(b==0)` – caso base: cuando $b = 0$, $\gcd(a, 0) = a$.
- `x=1; y=0;` – en este caso $a \cdot 1 + 0 \cdot 0 = a$, así que $x = 1, y = 0$.
- `return a;` – devuelve el gcd.
- `int x1, y1;` – variables temporales para los coeficientes de la llamada recursiva.
- `int g=extgcd(b, a%b, x1, y1);` – llamada recursiva con el algoritmo de Euclides clasico.
- `x = y1;` – “desenrolla” los coeficientes: el nuevo x es el antiguo y_1 .
- `y = x1 - y1 * (a / b);` – reconstruye y a partir de la identidad de Bézout.
- `return g;` – devuelve el gcd (es el mismo que el de la llamada recursiva).

Programa principal:

- `int x, y;` – variables para los coeficientes de Bézout.
- `int g = extgcd(12, 8, x, y);` – calcula $\gcd(12, 8) = 4$ con coeficientes x, y .
- `cout << "gcd = " << g << "\textbackslash n";` – imprime $\gcd = 4$.
- `cout << "x = " << x << ", y = " << y << "\textbackslash n";` – imprime $x = 1, y = -1$ (verifica: $12 \cdot 1 + 8 \cdot (-1) = 4$).
- `int xi, yi;` – variables para el calculo del inverso modular.
- `int a = 3, m = 7;` – modulo primo pequeno para verificar.
- `int g2 = extgcd(a, m, xi, yi);` – aplica extgcd para $a = 3, m = 7$.
- `if(g2 == 1)` – si el gcd es 1, existe el inverso modular.
- `int inv = (xi % m + m) % m;` – ajusta x al rango positivo $[0, m)$.
- `cout << a << " * " << inv << " mod " << m << " = " << (a * inv) % m << "\textbackslash n";` – imprime la verificacion $3 \cdot 5 \equiv 1 \pmod{7}$.

Codigo. Ver `cpp.tex`, seccion *Extended Euclidean Algorithm*.

2.2 Formulas utiles

Idea intuitiva. Mini-referencia de identidades clasicas: numero de divisores, suma de divisores, formula de Euler ($a^b \equiv a^{b \bmod \varphi(n)} \pmod{n}$ cuando $\gcd(a, n) = 1$), e inclusion-exclusion. **No hay codigo ejecutable;** solo las formulas y, opcionalmente, comentarios sobre cuando aplicarlas. Estas identidades salen de la aritmética basica: si $n = \prod p_i^{e_i}$, entonces cada divisor es $\prod p_i^{f_i}$ con $0 \leq f_i \leq e_i$, así que el conteo se multiplica independientemente por cada primo.

Propiedades clave (invariantes).

- Numero de divisores: si $n = \prod p_i^{e_i}$, entonces $d(n) = \prod (e_i + 1)$.
- Suma de divisores: $\sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}$.
- Producto de divisores: $n^{d(n)/2}$ (cada divisor d se empareja con n/d).
- Inclusion-exclusion: $|\bigcup A_i| = \sum_{\emptyset \neq I} (-1)^{|I|+1} |\bigcap_{i \in I} A_i|$.
- $\sum_{d|n} d \cdot \varphi(n/d) = \sum_{d|n} \mu(d) \cdot \sigma(n/d)$ (identidades utiles).

Complejidad. No aplica para este modulo (es solo referencia de formulas).

Donde tocar para modificar. Este modulo **no tiene codigo**; es una seccion de formulas. Si quieres anadir una formula nueva (por ejemplo, suma de primos hasta n via criba), agregar el `\subsubsection` correspondiente en otra parte del notebook.

Ejemplo de uso. Para contar primos relativos a n en $[1, n]$: $\varphi(n)$. Para calcular $\sum_{i=1}^n \gcd(i, n)$: factorizar n , luego $\sum_{d|n} d \cdot \varphi(n/d)$. Para $\sum_{d|n} \mu(d) = [n = 1]$ (delta de Kronecker).

Trampas y errores frecuentes.

- La suma de primos no tiene formula cerrada simple; necesita criba.
- $\varphi(n)$ solo simplifica el exponente cuando $\gcd(a, n) = 1$; en caso contrario, hay que factorizar a mano.
- En inclusion-exclusion, el orden de los terminos no importa, pero si $|I| > 20$ el numero de terminos explota.
- $d(n) = 2$ si y solo si n es primo.

Explicacion linea por linea. Este modulo no contiene codigo ejecutable; las formulas estan escritas en LaTeX directamente:

- **Numero de divisores** $d(n)$: formula

$$d(n) = (e_1 + 1)(e_2 + 1) \cdots (e_k + 1)$$

para $n = p_1^{e_1} \cdots p_k^{e_k}$.

- **Suma de divisores** $\sigma(n)$: formula

$$\sigma(n) = \prod_{i=1}^k \frac{p_i^{e_i+1} - 1}{p_i - 1}$$

– suma geometrica para cada primo.

- **Producto de divisores** $P(n)$: formula

$$P(n) = n^{d(n)/2}$$

– cada divisor d se empareja con n/d .

- **Euler**: formula

$$a^b \equiv a^{b \bmod \varphi(n)} \pmod{n}$$

valida cuando $\gcd(a, n) = 1$.

- **Inclusion-Exclusion**: formula

$$\left| \bigcup_{\emptyset \neq I} A_i \right| = \sum_{\emptyset \neq I} (-1)^{|I|+1} \left| \bigcap_{i \in I} A_i \right|$$

– expansion de PIE sobre todos los subconjuntos no vacios.

Codigo. Ver `cpp.tex`, seccion *Formulas utiles*.

2.3 Criba de Eratostenes

Idea intuitiva. Para encontrar todos los primos hasta N , marcaremos como compuestos los multiplos de cada primo a partir de p^2 . Como el primo mas pequeno es 2, empezar en 4 evita marcar de mas. Es $O(N \log \log N)$ porque la cantidad de primos hasta N es $\sim N / \ln N$ y la mayoría tiene pocos multiplos pequenos. La cota asintotica precisa es $N \sum_{p \leq N} 1/p \approx N \ln \ln N$ por el teorema de Mertens; la constante es pequena (~ 1) en la practica.

Propiedades clave (invariantes).

- `is_composite[i]` es `true` sii i no es primo (o $i \leq 1$).
- `primes` contiene todos los primos en orden ascendente.
- Se itera solo hasta $\sqrt{N}$ en el bucle exterior porque cualquier compuesto $c \leq N$ tiene un factor primo $\leq \sqrt{N}$.
- Cada primo p marca $\lfloor N/p \rfloor - p + 1$ multiplos a partir de p^2 .

Codigo clave. El nucleo de la criba:

```
vector<bool> comp(N + 1, false);
for (int i = 2; (long long)i * i <= N; ++i)
    if (!comp[i])
        for (int j = i * i; j <= N; j += i)
            comp[j] = true;
vector<int> primes;
for (int i = 2; i <= N; ++i)
    if (!comp[i]) primes.push_back(i);
```

Complejidad.

- Tiempo $O(N \log \log N)$, memoria $O(N)$.
- Con $N = 10^7$: ~ 0.3 segundos en hardware moderno.

Donde tocar para modificar.

- **Obtener factorizacion de cada numero:** anadir un array `lp[i]` (lowest prime factor) que se llena junto al sieve. Es $O(N)$ si se hace en orden creciente de i .
- **Criba hasta $N = 10^7$:** este snippet con `vector<bool>` (que es bitset-packed) alcanza. Para $N = 10^8$ o mas, conviene criba segmentada (no incluida).
- **Sumar primos:** anadir `long long sumPrimes;` que acumule en el bucle final.
- **Primos en $[L, R]$:** criba segmentada, donde $R - L$ es chico pero L es grande.

Trampas y errores frecuentes.

- `vector<bool>` es proxy; si necesitas pasarlo a C-style API, copiar a `vector<char>`.
- `is_composite[0]` e `is_composite[1]` deben inicializarse a `true`.
- El bucle interno empieza en $i*i$, no en $2*i$; si arrancas en $2*i$ marcas el doble y pierdes tiempo pero el resultado es el mismo.
- Overflow si $i*i > \text{INT_MAX}$: usar `(long long)i * i <= N`.

Explicacion linea por linea. Funcion sieve:

- `vector<int> primes;` – lista global de primos encontrados.
- `vector<bool> is_composite;` – marca de compuesto (bitset-packed).
- `void sieve(int n)` – llena `primes` e `is_composite` hasta n .
- `is_composite.assign(n + 1, false);` – inicializa todos en `false`.
- `is_composite[0] = is_composite[1] = true;` – 0 y 1 no son primos.
- `for(int i=2 ; i*i<=n ; i++)` – itera solo hasta $\sqrt{n}$ (los primos $> \sqrt{n}$ no marcan nada nuevo).
- `if(!is_composite[i])` – si i no fue marcado, es primo.
- `for(int j=i*i;j<=n;j+=i) is_composite[j] = true;` – marca multiplos desde i^2 (los menores ya fueron marcados).
- `primes.clear();` – vacia la lista antes de repoblar.
- `for(int i=2;i<=n;i++) if(!is_composite[i]) primes.push_back(i);` – recolecta los primos.

Programa principal:

- `sieve(100);` – ejecuta la criba hasta 100.
- `for(int p : primes) cout << p << " ";` – imprime los 25 primos hasta 100.
- `cout << "#primes <= 100: " << primes.size() << "\textbackslash n";` – muestra el conteo.

Codigo. Ver `cpp.tex`, seccion *Criba de Eratostenes*.

2.4 Criba lineal

Idea intuitiva. La criba de Eratostenes marca cada compuesto varias veces (por ejemplo 12 se marca por 2, por 3 y por 4). La criba lineal asegura que **cada compuesto se marca exactamente una vez**, lo cual da $O(N)$ total. La idea: para cada i , iterar sobre los primos p en orden creciente y poner $lp[i \cdot p] = p$, **cortando** el bucle cuando $p = lp[i]$ (asi $i \cdot p$ queda con su factor primo mas chico como p). La prueba de $O(N)$: el bucle interno suma trabajo proporcional al numero de primos que **no** cortan, y solo se corta cuando $p \mid i$; la suma total sobre todos los i es exactamente N .

Propiedades clave (invariantes).

- $lp[i]$ = menor factor primo de i (con $lp[0] = lp[1] = 0$).
- Cada i tiene asignado exactamente un primo que “lo cierra”.
- Recorriendo i en orden creciente, cuando entramos a i , todos sus primos menores ya estan en **primes**.
- La condicion de corte $p == lp[i]$ garantiza que $i \cdot p$ tiene lp minimo.

Codigo clave. La criba lineal en su forma esencial:

```
vector<int> lp(n + 1), primes;
for (int i = 2; i <= n; ++i) {
    if (!lp[i]) { lp[i] = i; primes.push_back(i); }
    for (int p : primes) {
        if (p > lp[i] || (long long)i * p > n) break;
        lp[i * p] = p;
    }
}
```

Complejidad.

- Tiempo $O(N)$ exacto, memoria $O(N)$.
- En la practica, $\sim 2x$ mas rapido que la criba clasica para $N \geq 10^6$.

Donde tocar para modificar.

- **Factorizacion de un numero n :** usando lp , iterar `while (n > 1) { p = lp[n]; ...; n /= p; }` en $O(\log n)$.
- **Cantidad de divisores para todos los $i \leq N$:** anadir `cntDiv[i] = cntDiv[i/lp[i]] + 1` durante la criba.
- **Suma de divisores o totient por i :** igual, precomputar en el mismo pase.
- **Anadir mobius:** `mu[1] = 1; mu[i*p] = (p == lp[i]) ? 0 : -mu[i];`.

Trampas y errores frecuentes.

- La condicion `if (p == lp[i]) break;` es **crucial**; sin ella la criba vuelve a ser $O(N \log \log N)$.
- Overflow: chequear `1LL * i * p > n` antes de asignar.
- Si $lp[i] == 0$ (i es primo), la condicion `p > lp[i]` es `p > i`, lo que tambien detiene el bucle correctamente.

Explicacion linea por linea. Funcion linearSieve:

- `vector<int> lp;` – arreglo global del menor factor primo (*lowest prime*) de cada i .
- `vector<int> primes;` – lista de primos encontrados en orden creciente.
- `void linearSieve(int n)` – llena `lp[i]` y `primes` para todo $i \leq n$.
- `lp.assign(n + 1, 0);` – inicializa todos los menores primos en 0.
- `primes.clear();` – vacia la lista de primos.
- `for(int i=2 ; i<=n ; i++)` – itera sobre cada entero desde 2.
- `if(lp[i] == 0)` – si i no fue marcado, es primo.
- `lp[i] = i; primes.push_back(i);` – registra el primo en ambas estructuras.
- `for(int p : primes)` – bucle interno sobre primos conocidos.
- `long long v = 1LL * i * p;` – calcula el producto $i \cdot p$ en `long long` para evitar overflow.
- `if(v > n) break;` – si excede n , no procesamos mas.
- `lp[v] = p;` – marca el menor primo de $i \cdot p$ como p .
- `if(p == lp[i]) break;` – condicion crucial: detiene cuando p divide a i , garantizando $O(N)$ total.

Programa principal:

- `linearSieve(100);` – precomputa hasta $n = 100$.
- `cout << "lp[12] = " << lp[12] << "\textbackslash n";` – imprime el menor primo de $12 = 2$.
- `cout << "#primes <= 100: " << primes.size() << "\textbackslash n";` – imprime el conteo de primos (≤ 100 son 25).

Codigo. Ver `cpp.tex`, seccion *Criba lineal*.

2.5 Miller-Rabin primality

Idea intuitiva. Para n impar, escribir $n - 1 = 2^s \cdot d$ con d impar. Si n es primo, entonces para **cualquier** a con $1 < a < n - 1$ se cumple que $a^d \equiv 1 \pmod{n}$ o $a^{2^r \cdot d} \equiv -1 \pmod{n}$ para algun $r < s$. El test verifica esto con varios a (testigos). Si ningun testigo delata a n , es primo. Con un conjunto fijo de testigos, es **determinista** para $n < 2^{64}$. La demostracion se basa en el hecho de que en $\mathbb{Z}/p\mathbb{Z}$ con p primo, el grupo $(\mathbb{Z}/p\mathbb{Z})^*$ es ciclico de orden $p - 1$, asi que $a^{p-1} \equiv 1$ por Fermat, y la extraccion de raices cuadradas solo tiene dos posibilidades (± 1).

Propiedades clave (invariantes).

- Probabilistico si los testigos son aleatorios; **determinista** con el set del snippet para $u64$.
- Falla para $n < 2$ (no es primo).
- Bases: $\{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$ cubren todos los $u64$.
- Falsos positivos: si un testigo no delata, n puede ser primo o un fuerte pseudoprimo respecto a ese testigo.

Codigo clave. La ronda de Miller-Rabin para un testigo a :

```
long long x = modpow(a, d, n);
if (x == 1 || x == n - 1) return true; // pasa este testigo
for (int r = 0; r < s - 1; ++r) {
    x = mul_mod(x, x, n);
    if (x == n - 1) return true; // aparece -1 en alguna iteracion
}
return false; // n es compuesto
```

Complejidad.

- $O(\log^3 n)$ por la exponenciación modular, pero con constante pequeña. Para *u64*, unos pocos microsegundos.
- Con 7 testigos para *u64*: < 10 microsegundos en hardware moderno.

Donde tocar para modificar.

- **Cambiar el set de testigos**: si necesitas testear números $> 2^{64}$, añadir más testigos o usar BPSW.
- **Devolver un certificado**: añadir la secuencia de cuadrados que llevo al fallo para debugging.
- *u128*: el snippet usa `__uint128_t` para multiplicar mod n sin overflow. En GCC/clang funciona; en MSVC añadir `#include <boost/multiprecision/cpp_int.hpp>`.
- **Solo testigos pequeños**: si sabes que $n < 2^{32}$, basta con $\{2, 7, 61\}$ (solo 3 testigos).

Trampas y errores frecuentes.

- Multiplicar *u64* * *u64* puede overflow; **siempre** usar *u128*.
- Si n es par, el test responde rápido (lo chequea con `n % p == 0`).
- Cuidado con bases que sean múltiplos de n ; el snippet las saltea con `if (a % n == 0) continue;`.
- Si el loop `for (int r = ...) itera  $r < s - 1$  (no  $r < s$ ), el último caso se chequea afuera.`

Explicación línea por línea. Exponenciación modular `modpow_mr`:

- `using u64 = unsigned long long;` – tipo de 64 bits sin signo (para evitar overflow en restos).
- `using u128 = __uint128_t;` – tipo de 128 bits (solo en GCC/Clang) para multiplicación sin overflow.
- `static u64 modpow_mr(u64 a, u64 e, u64 mod)` – calcula $a^e \bmod \text{mod}$.
- `u64 r = 1; a %= mod;` – inicializa resultado en 1 y reduce la base.
- `if(e & 1) r = (u128)r * a % mod;` – si el bit de exponente está encendido, multiplica.
- `a = (u128)a * a % mod;` – eleva al cuadrado la base.
- `e >>= 1;` – divide exponente entre 2.

Test de Miller-Rabin:

- `if(n < 2) return false;` – n menor que 2 no es primo.
- `for(u64 p : {2ULL, ..., 37ULL})` – chequea los primos pequeños primero.
- `if(n % p == 0) return n == p;` – si divisible por p , es primo solo si $n = p$.
- `u64 d = n - 1, s = 0;` – descompone $n - 1 = 2^s \cdot d$.
- `while((d & 1) == 0){ d >>= 1; s++; }` – cuenta los factores de 2.
- `for(u64 a : {2ULL, 325ULL, ..., 1795265022ULL})` – conjunto fijo de bases para determinismo hasta 2^{64} .
- `if(a % n == 0) continue;` – salta bases que no son coprimas con n .
- `u64 x = modpow_mr(a, d, n);` – calcula $a^d \bmod n$.
- `if(x == 1 || x == n - 1) continue;` – si cumple trivialmente, pasa al siguiente testigo.
- `bool comp = true;` – asume n es compuesto inicialmente.
- `for(u64 r=1; r<s; r++)` – itera los cuadrados sucesivos.
- `x = (u128)x * x % n;` – eleva al cuadrado.
- `if(x == n - 1){ comp = false; break; }` – si aparece -1 , el testigo pasa.
- `if(comp) return false;` – si ningún cuadrado dio -1 , n es compuesto.

Código. Ver `cpp.tex`, sección *Miller-Rabin primality*.

2.6 Pollard's Rho + factorizacion

Idea intuitiva. Miller-Rabin detecta primalidad pero no factoriza. Pollard's Rho busca un **factor no trivial** de n usando una secuencia pseudoaleatoria $x_{k+1} = x_k^2 + c \pmod n$ y aplicando la **paradoja del cumpleaños**: si la secuencia cae en un ciclo modulo un factor primo p de n , entonces $\gcd(|x_i - x_j|, n)$ sera p para algun i, j . Esto es esperado $O(n^{1/4})$ por factor. La intuicion: la secuencia modulo p (donde $p \leq \sqrt{n}$) tiene un ciclo esperado de longitud $O(\sqrt{p}) = O(n^{1/4})$, mucho mas corto que el ciclo modulo n ($\sim n$).

Propiedades clave (invariantes).

- Cada llamada a `pollard(n)` devuelve **un** factor (no necesariamente primo); se llama recursivamente.
- Si `milllerRabin(n)` retorna `true`, n es primo y se ania de directo.
- El factor final sale **ordenado** porque el snippet ordena al final.
- El algoritmo de Floyd ("tortuga y liebre") detecta el ciclo comparando dos punteros a distinta velocidad.

Codigo clave. El bucle de Floyd (tortuga y liebre):

```
long long x = rand2(rng) % n, y = x, c = rand2(rng) % (n - 1) + 1, d = 1;
while (d == 1) {
    x = (mul_mod(x, x, n) + c) % n;          // tortuga: 1 paso
    y = (mul_mod(y, y, n) + c) % n;
    y = (mul_mod(y, y, n) + c) % n;          // liebre: 2 pasos
    d = gcd(abs(x - y), n);
}
if (d != n) return d; // encontro factor
```

Complejidad.

- Esperado $O(n^{1/4})$ por factor. Practico para n hasta 10^{18} .
- Para $n = 10^{18}$, tarda $\sim 10^3$ iteraciones por factor.

Donde tocar para modificar.

- **Mejorar la constante:** ania de el algoritmo de Brent (variante del Floyd para el ciclo, $\sim 24\%$ mas rapido).
- **Determinismo:** si la corrida con c falla, reintentar con otro c aleatorio (ya lo hace el snippet con `while(true)`).
- **Salida con multiplicidad:** el snippet devuelve la lista con repeticiones (porque `factorRec` empuja n cuando es primo y tambien n/d). Si solo queres factores distintos, ania de un `set` o deduplicar al final.
- **Acelerar con product acumulado:** Brent acumula productos parciales y reduce gcd; ahorra muchos gcd.

Trampas y errores frecuentes.

- Si n es potencia de un primo pequeno (ej. 2^k), Pollard puede tardar. Considerar chequeo previo de potencia.
- Si n es primo, llamar a `pollard(n)` causaria loop infinito porque `d == n`; el snippet tiene la guarda `if (milllerRabin(n)) return`.
- Si $c = 0$ y $x = 0$ o x es punto fijo, la secuencia es constante y $\gcd = 0$; por eso $c \neq 0$.
- `abs(x - y)` puede overflow si ambos son cercanos a $n/2$; usar `u64` con cuidado o comparar con `std::abs(int64_t(x - y))`.

Explicacion linea por linea. Multiplicacion modular modmul:

- `static u64 modmul(u64 a, u64 b, u64 mod)` – calcula $a \cdot b \pmod{\text{mod}}$ sin overflow.
- `return (u128)a * b % mod;` – cast a 128 bits para evitar overflow de la multiplicacion.

Pollard Rho pollard:

- `if(n % 2 == 0) return 2;` – caso trivial: si n es par, 2 es factor.

- `static std::mt19937_64 rng(...)`; – generador aleatorio (seed con tiempo).
- `while(true)` – reintentos con distintos parametros si falla.
- `u64 c = uniform_int_distribution<u64>(1, n - 1)(rng)`; – constante c aleatoria en $[1, n - 1]$.
- `u64 x = uniform_int_distribution<u64>(0, n - 1)(rng)`; – valor inicial x_0 aleatorio.
- `u64 y = x`; – “liebre” empieza igual que la “tortuga”.
- `auto f = [&](u64 v){ return (modmul(v, v, n) + c) % n; }`; – funcion pseudoaleatoria $f(v) = v^2 + c \bmod n$.
- `while(d == 1)` – ciclo de Floyd: tortuga 1 paso, liebre 2 pasos.
- `x = f(x)`; – avanza la tortuga.
- `y = f(f(y))`; – avanza la liebre el doble.
- `d = std::gcd<u64>(x > y ? x - y : y - x, n)`; – calcula $\gcd(|x - y|, n)$.
- `if(d != n) return d`; – si encontro un factor no trivial, lo retorna.

Recursion factorRec:

- `if(n == 1) return`; – caso base.
- `if(millerRabin(n)){ out.push_back(n); return; }` – si es primo, lo guarda.
- `u64 d = pollard(n)`; – busca un factor.
- `factorRec(d, out)`; – factoriza la primera mitad.
- `factorRec(n / d, out)`; – factoriza la segunda mitad.

factorize:

- `factorRec(n, out)`; – llena out recursivamente.
- `sort(out.begin(), out.end())`; – ordena los factores (para consistencia).

Codigo. Ver `cpp.tex`, seccion *Pollard’s Rho + factorizacion*.

2.7 Modular exponentiation

Idea intuitiva. Calcular $a^e \bmod m$ en $O(\log e)$ mediante **exponenciacion binaria** (squaring). La idea: si e es par, $a^e = (a^{e/2})^2$; si es impar, $a^e = a \cdot (a^{(e-1)/2})^2$. Asi, en cada paso, el exponente se reduce a la mitad y se hace una multiplicacion modular. El nombre “squaring” viene de que en cada paso elevamos al cuadrado una variable (“base”). Se puede demostrar por induccion que tras k iteraciones, la variable “resultado” contiene $a^{(\text{bits consumidos})}$, manteniendo la invariante $\text{result} \cdot \text{base}^e = a^{e_{\text{original}}}$.

Propiedades clave (invariantes).

- $a \% m$ se hace al inicio (en caso de que a sea negativo, se ajusta $+= \text{mod}$).
- Multiplicacion: para evitar overflow, usar `__int128` o `long long` solo si $m < 2^{32}$ (porque $a \cdot b < m^2 < 2^{64}$).
- Resultado siempre en $[0, m)$.
- El algoritmo usa exactamente $\lfloor \log_2 e \rfloor + 1$ multiplicaciones (medido en bits del exponente).

Codigo clave. El bucle de squaring en su forma mas pura:

```
long long modpow(long long a, long long e, long long m) {
    long long r = 1; a %= m;
    while (e > 0) {
        if (e & 1) r = (__int128)r * a % m;
        a = (__int128)a * a % m;
        e >>= 1;
    }
    return r;
}
```

Complejidad.

- $O(\log e)$ multiplicaciones modulares.
- ~ 30 iteraciones para e hasta 10^9 ; ~ 60 para e hasta 10^{18} .

Donde tocar para modificar.

- **Modulo de 10^{18} :** añadir `__int128` a las multiplicaciones (ya hecho si el snippet usa `u128`; si no, añadir manualmente).
- **Exponentes muy grandes ($e > 10^{18}$):** el bucle sigue funcionando pero no entra; añadir modularizacion del exponente via Euler si $\gcd(a, m) = 1$ y quieres reducir e primero.
- **Pow matrix/Poly:** la misma estructura sirve; solo cambia el tipo de la base y la operacion (mul de matrices, mul de polinomios).
- **Exponenciacion rapida sin division:** Montgomery multiplication para criptografia.

Trampas y errores frecuentes.

- Si m es 1, el resultado siempre es 0; añadir check si te molesta.
- Overflow en `a * a % mod` si `a` y `mod` son `long long` grandes.
- En MSVC, `__int128` no existe; añadir `#include <boost/multiprecision/cpp_int.hpp>`.
- Si e es int negativo (raro, pero por un bug), el bucle no termina.

Explicacion linea por linea. Funcion modpow:

- `static const long long MOD = 998244353LL;` – modulo por defecto (primo usado en contests).
- `long long modpow(long long a, long long e, long long mod = MOD)` – calcula $a^e \bmod \text{mod}$.
- `long long r = 1;` – resultado inicializado en 1 (neutro multiplicativo).
- `a %= mod;` – reduce la base al rango $[0, \text{mod})$.
- `if(a < 0) a += mod;` – corrige negativos al positivo equivalente.
- `while(e)` – itera mientras queden bits por procesar.
- `if(e & 1) r = r * a % mod;` – si el bit menos significativo es 1, multiplica el resultado.
- `a = a * a % mod;` – eleva al cuadrado la base para el proximo bit.
- `e >>= 1;` – desplaza el exponente (divide entre 2).
- `return r;` – retorna el resultado final.

Programa principal:

- `cout << modpow(2, 10) << "\textbackslash n";` – calcula $2^{10} \bmod 998244353 = 1024$.
- `cout << modpow(2, 10, 1000000007LL) << "\textbackslash n";` – calcula $2^{10} \bmod 10^9+7 = 1024$ (mismo resultado por ser pequeno).

Codigo. Ver `cpp.tex`, seccion *Modular exponentiation*.

2.8 Modular inverse

Idea intuitiva. La **inversa** de a modulo m es un b tal que $a \cdot b \equiv 1 \pmod{m}$. Si m es primo y $\gcd(a, m) = 1$, entonces $b = a^{m-2} \pmod{m}$ (por el **teorema pequeno de Fermat**). Si m no es primo, se usa el algoritmo extendido de Euclides, que resuelve $ax + my = \gcd(a, m)$. El truco de Fermat: si m es primo, $(\mathbb{Z}/m\mathbb{Z})^*$ es un grupo de orden $m - 1$, asi que $a^{m-1} \equiv 1$ y $a \cdot a^{m-2} \equiv 1$.

Propiedades clave (invariantes).

- La inversa existe sii $\gcd(a, m) = 1$.
- Para mod primo, $\text{modInv}(a, \text{mod}) = \text{modpow}(a, \text{mod} - 2, \text{mod})$.
- `modInvGeneral` retorna -1 si no hay inversa.
- Si existe, la inversa es **única** modulo m .

Código clave. Recurrencia lineal para precomputar todas las inversas (solo funciona con mod primo):

```
// inv[1] = 1
// inv[i] = -(mod/i) * inv[mod%i] mod mod
for (int i = 2; i <= N; ++i) {
    inv[i] = MOD - (long long)(MOD / i) * inv[MOD % i] % MOD;
}
```

Complejidad.

- $O(\log \text{mod})$ en ambos casos (Fermat y extgcd).
- Precomputar N inversas: $O(N)$.

Donde tocar para modificar.

- **Precomputar muchas inversas** (ej. para todos los $1..N$): precomputar en $O(N)$ con la recurrencia `inv[i] = -(mod/i) * inv[mod % i] % mod` y ajustar a positivo.
- **Cambiar mod**: si modificas el MOD global, asegurate de que sea primo (sino, usar extgcd).
- **Ajustar a positivo**: tras `-- MOD` o `+= MOD` para tener resultado en $[0, MOD)$.

Trampas y errores frecuentes.

- Si $\text{MOD} = 1$, $a^{\text{MOD}-2} = a^{-1}$ no esta definido; añadir check.
- Si $a = 0$, no hay inversa (excepto en mod trivial).
- El snippet usa `modInv(a, mod)` que asume mod primo por construccion.
- La recurrencia lineal `inv[i] = -(mod/i)*inv[mod%i]` solo sirve si mod es primo y $i < \text{mod}$.
- Si llamas `modInv` con a divisible por mod, devuelve cualquier cosa (generalmente 0 porque $a^{m-2} \equiv 0$).

Explicacion linea por linea. Inversa con Fermat modInv:

- `long long modInv(long long a, long long mod)` – calcula $a^{-1} \bmod p$ asumiendo p primo.
- `return modpow(a, mod - 2, mod);` – aplica el teorema de Fermat: $a^{p-2} \equiv a^{-1} \pmod{p}$.

Algoritmo extendido de Euclides extGcd:

- `long long extGcd(long long a, long long b, long long& x, long long& y)` – calcula gcd y coeficientes de Bézout.
- `if(b == 0){ x = 1; y = 0; return a; }` – caso base: $a \cdot 1 + 0 \cdot 0 = a$.
- `long long x1, y1;` – variables temporales.
- `long long g = extGcd(b, a % b, x1, y1);` – llamada recursiva.
- `x = y1;` – el nuevo x es el antiguo y_1 .
- `y = x1 - y1 * (a / b);` – reconstruye y .

Inversa general modInvGeneral:

- `long long g = extGcd(a, mod, x, y);` – obtiene el gcd y los coeficientes.
- `if(g != 1) return -1;` – si el gcd no es 1, no existe inversa.

- `return (x % mod + mod) % mod;` – ajusta al rango $[0, \text{mod})$.

Programa principal:

- `cout << modInv(3, 1000000007LL) << "\textbackslash n";` – inversa de 3 modulo $10^9 + 7$ (via Fermat).
- `cout << modInvGeneral(7, 26LL) << "\textbackslash n";` – inversa de 7 modulo 26 (via extgcd), resultado 15.

Codigo. Ver `cpp.tex`, seccion *Modular inverse*.

2.9 Chinese Remainder Theorem

Idea intuitiva. Dado un sistema $x \equiv r_i \pmod{m_i}$ con los m_i **coprimos dos a dos**, hay **una** solucion modulo $M = \prod m_i$. Se construye incrementalmente: combinando $x \equiv r_1 \pmod{m_1}$ con $x \equiv r_2 \pmod{m_2}$, la solucion es $x = r_1 + m_1 \cdot t$ donde $t \equiv (r_2 - r_1) \cdot m_1^{-1} \pmod{m_2}$. Iterar con cada m_i . La prueba de existencia y unicidad es por induccion sobre N : con una sola ecuacion la solucion es trivial; al anadir la ecuacion $i + 1$ con $\gcd(m_i, m_{i+1}) = 1$, existe $m_i^{-1} \pmod{m_{i+1}}$ y se resuelve el nuevo offset.

Propiedades clave (invariantes).

- El snippet asume m_i coprimos dos a dos; **no** maneja el caso general.
- Solucion unica modulo M .
- Si los modulos **no** son coprimos y $\gcd(m_i, m_j) \nmid (r_i - r_j)$, no hay solucion; si divide, se reduce el sistema.
- Cada paso incremental no destruye las ecuaciones previas.

Codigo clave. El paso inductivo (combinar una solucion parcial con una ecuacion nueva):

```
long long k = ((r2 - r1) % m2 + m2) % m2;
k = k * modInv(m1 % m2, m2) % m2; // inversa de m1 mod m2
r1 = r1 + m1 * k; // nueva solucion
m1 = m1 * m2; // nuevo modulo
r1 = (r1 % m1 + m1) % m1;
```

Complejidad.

- $O(N \log M)$ con N el numero de ecuaciones.
- Cada paso usa una inversa modular ($O(\log m_i)$) y unas multiplicaciones.

Donde tocar para modificar.

- **Modulos no coprimos:** anadir el test $\gcd(m_i, m_j)$ en cada paso y devolver error o reducir.
- **Devolver todas las soluciones:** si el problema quiere todas las x en $[0, L]$, ajustar al final con un `if (x > L) return -1;`.
- **Garner:** para evitar overflow con M muy grande, usar el algoritmo de Garner (mas estable numericamente).
- **Caso $k = 0$:** anadir check para retornar temprano (ya resuelto).

Trampas y errores frecuentes.

- Overflow al multiplicar $M * t$ si los m_i son grandes; usar `__int128` o reducir modulo M en cada paso (ya lo hace el snippet).
- Si una ecuacion es $x \equiv r_i \pmod{m_i}$ con $r_i \geq m_i$, normalizar antes.
- Confundir “modulos coprimos” con “modulos primos individuales”; los m_i solo necesitan ser coprimos entre si.
- En la condicion “hay solucion”, devolver un valor sentinela (e.g., -1) consistente con el resto del codigo.

Explicacion linea por linea.

- `#include <bits/stdc++.h>` – incluye todas las bibliotecas estandar (necesario para `vector`, `cout`, `modpow`).
- `using namespace std;` – evita prefijar `std::`.
- `long long crt(const vector<long long>& r, const vector<long long>& m)` – recibe los residuos `r[i]` y los modulos `m[i]` (estos deben ser coprimos dos a dos).
- `long long x = 0;` – solucion parcial actual: cumple las primeras i congruencias.
- `long long M = 1;` – modulo parcial actual: producto de los primeros i modulos.
- `for(int i=0 ; i<r.size() ; i++)` – procesa cada congruencia secuencialmente.
- `long long diff = (r[i] - x) % m[i];` – calcula la diferencia que falta entre la solucion parcial y la nueva congruencia.
- `if(diff < 0) diff += m[i];` – ajusta al rango positivo (residuo canonico).
- `long long inv = modpow(M % m[i], m[i] - 2, m[i]);` – inversa de M modulo m_i via Fermat (porque m_i es primo o coprimo).
- `long long t = diff * inv % m[i];` – calcula el desplazamiento necesario para corregir la solucion parcial.
- `x = x + M * t;` – actualiza la solucion parcial sumando el offset correcto.
- `M *= m[i];` – extiende el modulo parcial al producto acumulado.
- `x %= M;` – reduce la solucion modulo el nuevo M para evitar overflow.
- `if(x < 0) x += M;` – corrige a rango positivo.
- `return x;` – retorna la solucion unica modulo $\prod m_i$.
- `cout << crt({2, 3, 2}, {3, 5, 7}) << "\textbackslash n";` – resuelve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$ (respuesta: 23).

Codigo. Ver `cpp.tex`, seccion *Chinese Remainder Theorem*.

2.10 Euler totient precompute

Idea intuitiva. La funcion $\varphi(n)$ cuenta cuantos enteros en $[1, n]$ son coprimos con n . Hay una formula: $\varphi(n) = n \cdot \prod_{p|n} (1 - 1/p)$. Para precomputar $\varphi(i)$ para todo $i \leq N$ en $O(N \log \log N)$, se inicializa $\varphi(i) = i$ y para cada primo p se hace $\varphi(j) -= \varphi(j)/p$ para todos los multiples j de p . La razon geometrica: para cada primo p , la fraccion $1/p$ de los enteros hasta n son multiples de p , asi que φ “pierde” ese factor en cada primo divisor.

Propiedades clave (invariantes).

- $\varphi(1) = 1$ por convencion (aunque coprimos con 1 es solo $\{1\}$).
- φ es **multiplicativa**: si $\gcd(a, b) = 1$, $\varphi(ab) = \varphi(a)\varphi(b)$.
- **Identidad de Euler**: si $\gcd(a, n) = 1$, entonces $a^{\varphi(n)} \equiv 1 \pmod{n}$.
- $\varphi(n) \geq \sqrt{n}$ para $n > 6$ (acotacion util en busqueda).
- $\sum_{d|n} \varphi(d) = n$ (propiedad de inversion, util en Dirichlet).

Codigo clave. Criba Euler-style para totients (un solo pase):

```
vector<int> phi(n + 1);
for (int i = 0; i <= n; ++i) phi[i] = i;
for (int i = 2; i <= n; ++i)
    if (phi[i] == i) // i es primo
        for (int j = i; j <= n; j += i)
            phi[j] -= phi[j] / i;
```

Complejidad.

- Precompute $O(N \log \log N)$, consulta $O(1)$. Single $\varphi(n)$ con factorizacion es $O(\sqrt{n})$.

Donde tocar para modificar.

- **Single phi via factorizacion:** ya hay una funcion `phiSingle(n)`. Para n hasta 10^{12} factoriza en $\leq 10^6$ pasos.
- **Aplicar a inclusion-exclusion:** si el problema requiere $\sum_{i=1}^N \gcd(i, n)$, se puede usar $\sum_{d|n} d \cdot \varphi(n/d)$.
- **Criba lineal:** durante la criba, anadir `phi[i*p] = (p == lp[i]) ? phi[i] * p : phi[i] * (p-1);`.

Ejemplo. Para $n = 12$: $\varphi(12) = 12 \cdot (1 - 1/2) \cdot (1 - 1/3) = 12 \cdot 1/2 \cdot 2/3 = 4$. Los coprimos son $\{1, 5, 7, 11\}$.

Trampas y errores frecuentes.

- Olvidar inicializar $\varphi(0) = 0$.
- Si llamas a `phiSingle` sobre 0, devuelve 0 (no hay coprimos con 0 en el sentido usual).
- La criba Euler-style modifica $\varphi[j]$ por division **antes** de que termine; para j compuestos, se hace varias veces (correcto, multiplicativo).
- Confundir $\varphi(0) = 0$ con $\varphi(1) = 1$; la criba comun los trata igual.

Explicacion linea por linea. Criba `phiSieve`:

- `vector<int> phi;` – arreglo global con valores $\varphi(i)$.
- `phi.assign(n + 1, 0);` – inicializa a 0 (luego se sobrescribe).
- `for(int i=0; i<=n; i++) phi[i] = i;` – inicializa $\varphi(i) = i$ (caso base: todo coprimo).
- `for(int i=2; i<=n; i++)` – itera sobre posibles primos.
- `if(phi[i] == i)` – si $\varphi(i) = i$, entonces i es primo (aun no modificado).
- `for(int j=i; j<=n; j+=i)` – itera sobre multiples de i .
- `phi[j] -= phi[j] / i;` – resta el factor $(1 - 1/p)$ a $\varphi(j)$.

Version single `phiSingle`:

- `long long phiSingle(long long n)` – calcula $\varphi(n)$ factorizando (sin precompute).
- `long long result = n;` – inicia en n .
- `for(long long p=2 ; p*p<=n ; p++)` – itera sobre posibles primos hasta $\sqrt{n}$.
- `if(n % p == 0)` – encontro un factor primo.
- `while(n % p == 0) n /= p;` – consume todas las ocurrencias de p .
- `result -= result / p;` – aplica la formula $\varphi(n) = n \cdot (1 - 1/p)$.
- `if(n > 1) result -= result / n;` – si queda un factor primo grande al final.

Programa principal:

- `phiSieve(100);` – precomputa φ hasta 100.
- `cout << "phi(" << i << ") = " << phi[i] << "\textbackslash n";` – imprime $\varphi(1..10)$.
- `cout << "phi(36) = " << phiSingle(36) << "\textbackslash n";` – calcula $\varphi(36) = 12$ via factorizacion.

Codigo. Ver `cpp.tex`, seccion *Euler totient precompute*.

2.11 Mobius precompute

Idea intuitiva. La funcion $\mu(n)$ (“Mobius”) vale 1 si n es producto de un numero par de primos distintos, -1 si es impar, 0 si tiene un primo al cuadrado. Aparece en inclusion-exclusion: si $f(n) = \sum_{d|n} g(d)$, entonces $g(n) = \sum_{d|n} \mu(n/d) f(d)$ (**inversion de Mobius**). Se precomputa junto a la criba lineal. La relacion con inclusion-exclusion: μ es la “inversa” del operador “suma sobre divisores”, asi que si conocemos los acumulados podemos recuperar los originales.

Propiedades clave (invariantes).

- $\sum_{d|n} \mu(d) = [n = 1]$ (funcion indicador de 1).
- Si $\gcd(a, b) = 1$, $\mu(ab) = \mu(a)\mu(b)$ (multiplicativa).
- $\sum_{d|n} \mu(d) \cdot \frac{n}{d} = \varphi(n)$ (relacion con totient).
- Para n cuadrado libre: $|\mu(n)| = 1$ y $\mu(n) = (-1)^{\omega(n)}$ donde ω es el numero de factores primos.

Codigo clave. Mobius durante la criba lineal:

```
mu[1] = 1;
for (int i = 2; i <= n; ++i) {
    if (!lp[i]) { lp[i] = i; primes.push_back(i); mu[i] = -1; }
    for (int p : primes) {
        if (p > lp[i] || (long long)i * p > n) break;
        lp[i * p] = p;
        if (p == lp[i]) mu[i * p] = 0;           // cuadrado
        else mu[i * p] = -mu[i];                // nuevo primo
    }
}
```

Complejidad.

- Precompute $O(N)$ con la criba lineal. Consulta $O(1)$.

Donde tocar para modificar.

- **Suma de gcd:** $\sum_{i=1}^N \gcd(i, n) = \sum_{d|n} d \cdot \varphi(n/d)$, alternativa con Mobius: $\sum_{d|n} \mu(d) \cdot S(\lfloor n/d \rfloor)$ donde $S(x) = \sum_{k \leq x} k$.
- **Mobius en inclusion-exclusion:** el snippet es solo para precompute; usarlo combinado con divisor enumeration en queries.
- **Conteo de coprimos:** $\sum_{d|n} \mu(d) \cdot \lfloor n/d \rfloor = \varphi(n)$.

Ejemplo. $\mu(1) = 1$, $\mu(2) = -1$, $\mu(6) = 1$ (dos primos distintos), $\mu(4) = 0$ (cuadrado). $\sum_{d|6} \mu(d) = \mu(1) + \mu(2) + \mu(3) + \mu(6) = 1 - 1 - 1 + 1 = 0$ (correcto: $6 \neq 1$).

Trampas y errores frecuentes.

- $\mu(0) = 0$ (por definicion, 0 no es producto de primos distintos).
- El snippet inicializa $\text{mu}[0] = 0$, $\text{mu}[1] = 1$ por separado.
- Si precomputas junto a la criba lineal, mantener consistencia: $\text{mu}[p]$ para primo es -1 (un solo primo).
- En queries de Mobius, sumar con el signo correcto; $(-1)^{\omega(n)}$ se puede calcular con la criba.

Explicacion linea por linea. Funcion mobiusSieve:

- `vector<int> mu;` – arreglo global con los valores de $\mu(n)$.
- `mu.assign(n + 1, 1);` – inicializa todos en 1 (luego se corrige).
- `mu[0] = 0; mu[1] = 1;` – casos base: $\mu(0) = 0$, $\mu(1) = 1$.
- `vector<bool> is_composite(n + 1, false);` – marca de compuesto (estilo Eratostenes).
- `for(int i=2; i<=n; i++)` – itera sobre cada entero.
- `if(!is_composite[i])` – si no fue marcado, es primo.
- `primes.push_back(i); mu[i] = -1;` – registra el primo y $\mu(i) = -1$ (un solo factor).
- `for(int p : primes)` – bucle interno estilo criba lineal.

- `long long v = 1LL * i * p;` – producto $i \cdot p$.
- `if(v > n) break;` – si excede n , termina.
- `is_composite[v] = true;` – marca el compuesto.
- `if(i % p == 0)` – si p divide a i , v tiene un cuadrado de p .
- `mu[v] = 0; break;` – $\mu(v) = 0$ y termina (preserva linealidad).
- `else mu[v] = -mu[i];` – si p es nuevo factor, $\mu(v) = -\mu(i)$ (alterna signo).

Programa principal:

- `mobiusSieve(20);` – precomputa μ hasta 20.
- `cout << "mu(" << i << ") = " << mu[i] << "\textbackslash n";` – imprime $\mu(1) = 1, \mu(2) = -1, \mu(6) = 1, \mu(4) = 0$, etc.

Codigo. Ver `cpp.tex`, seccion *Mobius precompute*.

2.12 Stern-Brocot (best fraction in interval)

Idea intuitiva. Stern-Brocot es un arbol binario de fracciones irreducibles. Cada nodo es el **mediantes** de sus “padres” en el arbol: si tienes a/b y c/d , el mediantes es $(a+c)/(b+d)$. Para encontrar la fraccion irreducible con denominador acotado mas cercana a un valor x , se navega el arbol decidiendo ir a izquierda o derecha segun si x es menor o mayor. La fraccion mas simple en un intervalo $(a/b, c/d)$ se obtiene iterando mediantes y quedandose con el **maximo** k tal que el mediantes sigue dentro del intervalo. La propiedad clave: las fracciones irreducibles con denominador acotado estan **todas** en el arbol.

Propiedades clave (invariantes).

- Cada par de fracciones irreducibles consecutivas en Stern-Brocot tiene diferencia $1/(bd)$ en valor.
- Permite generar todas las fracciones irreducibles con un denominador acotado.
- El mediantes m/n de a/b y c/d satisface $b \cdot m - a \cdot n = 1$ (cruz = 1).

Codigo clave. La busqueda en Stern-Brocot (izquierda/derecha segun comparacion):

```
long long findBest(long long x, long long a, long long b, long long c, long long d) {
    long long p = a + c, q = b + d; // mediantes
    if (q > Q) return ...; // denominador excede
    if (p/q > x) return find(x, a, b, p, q); // izquierda
    else return find(x, p, q, c, d); // derecha
}
```

Complejidad.

- $O(\log(\max(a, b)))$ por query.
- Generar todas las fracciones con denominador $\leq Q$: $O(Q^2)$ (hay $\sim 3Q^2/\pi^2$ irreducibles).

Donde tocar para modificar.

- **Acotar denominador:** aniadir un parametro Q (denominador maximo) y ajustar $k = \min(k, Q/\dots)$.
- **Devolver la fraccion irreducible (no reducida):** aniadir division por $\gcd(p, q)$ al final.
- **Buscar fraccion mas cercana:** aniadir comparacion entre actual y candidato al estilo del BS.

Trampas y errores frecuentes.

- El snippet tiene una aproximacion cruda; un mejor implementation debe iterar **exactamente** el maximo k permitido.
- Cuidado con overflow en $(p - a) / b$ si los numeros son grandes.
- El mediantes es irreducible sii $\gcd(a + c, b + d) = 1$; en el arbol de Stern-Brocot se preserva.
- La raiz del arbol es $1/1$; los hijos son $1/2$ y $2/1$.

Explicacion linea por linea. Estructura Frac:

- `struct Frac { long long p, q; };` – encapsula numerador y denominador.

Funcion bestInInterval:

- `Frac bestInInterval(long long a, long long b, long long c, long long d)` – encuentra la fraccion irreducible mas simple en $(a/b, c/d)$.
- `long long p = a + c, q = b + d;` – mediantes inicial: $(a + c)/(b + d)$.
- `while(true)` – itera hasta encontrar el limite.
- `long long k = max((p - a) / b, (p - c) / d);` – maximo k tal que el mediantes sigue en el intervalo.
- `if(k <= 1) break;` – si $k \leq 1$, ya estamos en el limite.
- `p = a + k * c; q = b + k * d;` – salta al mediantes mas lejano permitido.
- `return {p, q};` – retorna la fraccion encontrada.

Programa principal:

- `auto f = bestInInterval(0, 1, 1, 1);` – busca la fraccion mas simple en $(0/1, 1/1)$.
- `cout << f.p << "/" << f.q << "\textbackslash n";` – imprime $1/2$ (la mas simple en ese intervalo).

Codigo. Ver `cpp.tex`, seccion

2.2 Combinatoria

2.1 Catalan (formula)

Idea intuitiva. Los numeros de Catalan cuentan una gran variedad de estructuras (arboles binarios, parentesis balanceados, triangulaciones de poligono). La formula es $C_n = \frac{1}{n+1} \binom{2n}{n}$. La version recursiva es $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$, que da una DP en $O(n^2)$. La conexion con $\binom{2n}{n}$ viene del **lattice path counting**: C_n es el numero de caminos de $(0, 0)$ a (n, n) que no cruzan la diagonal, y los caminos sin esa restriccion son $\binom{2n}{n}$ (elegir cuando subir).

Propiedades clave (invariantes).

- $C_0 = C_1 = 1$.
- C_n crece rapidamente; C_{19} ya pasa el millon.
- La formula cerrada evita la DP pero requiere multiplicacion que **siempre** es exacta (la division por $i + 2$ no deja residuo).
- Funcion generadora: $C(x) = \sum_{n \geq 0} C_n x^n$ satisface $C(x) = 1 + xC(x)^2$.

Codigo clave. Formula iterativa (sin division intermedia):

```
long long catalanLL(int n) {
    long long c = 1;
    for (int i = 0; i < n; ++i) {
        c = c * 2 * (2*i + 1) / (i + 2); // division siempre exacta
    }
    return c;
}
```

Complejidad.

- DP: $O(N^2)$ tiempo, $O(N)$ memoria.
- Formula cerrada (iterativa): $O(N)$ tiempo, $O(1)$ memoria.
- Para un solo C_n : $O(n)$ (no $O(n^2)$ con DP).

Donde tocar para modificar.

- **Catalan mod p**: anadir MOD y hacer % MOD en cada multiplicacion. La division $/(i+2)$ no se puede hacer mod p directamente; conviene usar el inverso modular: `ans = ans * 2 * (2*i+1) % MOD * inv[i+2] % MOD;`
- **Catalan con doble precision**: `catalanLL` retorna `long long`; para $n \geq 35$ desborda.
- **Catalan via binomial**: usar el snippet de factoriales: $C_n = \binom{2n}{n} - \binom{2n}{n+1}$ (numericamente estable mod primo).

Trampas y errores frecuentes.

- La division entera $(i+2)$ en `catalanLL` es exacta solo porque matematicamente el cociente es entero. Si cambia el orden de operaciones (por ejemplo, modular primero), la division falla.
- Para Catalan mod p NO primo, no se puede usar la recurrencia con division; usar DP pura.
- C_{35} ya excede 2^{63} ; usar `BigInt` si lo necesitas.

Explicacion linea por linea. Version DP $O(n^2)$:

- `vector<int> dp;` – vector que almacena $C_0, C_1, \dots, C_n$; cada posicion guarda el numero de Catalan correspondiente.
- `const int MOD=998244353;` – primo usado para todas las operaciones modulo (tipico en contests).
- `void catalan(int n)` – funcion que llena el vector `dp` con $C_0, \dots, C_n$.
- `dp.resize(n+1);` – ajusta el tamaño del vector a $n+1$ para indexar hasta n .
- `dp[0]=dp[1]=1;` – casos base: $C_0 = C_1 = 1$.
- `for(int i=2 ; i<=n ; i++)` – itera n desde 2 hasta n aplicando la recurrencia.
- `for(int j=0 ; j<i ; j++)` – elige el tamaño del subarbol izquierdo (j) entre 0 y $i-1$.
- `dp[i]=(dp[i]+(dp[j]*dp[i-j-1]))%MOD)%MOD;` – acumula $C_j \cdot C_{i-1-j}$ en C_i (recurrencia clasica de Catalan).

Version formula cerrada $O(n)$:

- `int catalan(int n)` – calcula C_n en $O(n)$ sin division intermedia riesgosa.
- `int ans=1;` – inicializa el acumulador en $C_0 = 1$.
- `for(int i=0 ; i<n ; i++)` – itera n veces para obtener C_n desde C_0 .
- `ans=ans*2*(2*i+1)/(i+2);` – aplica la recurrencia $C_{i+1} = C_i \cdot 2(2i+1)/(i+2)$; la division es exacta.
- `return ans;` – retorna C_n .
- `cout << "catalan(10) = " << catalan(10) << "\textbackslash n";` – imprime $C_{10} = 16796$.

Codigo. Ver `cpp.tex`, seccion *Catalan (formula)*.

2.2 Factoriales + nCr mod p

Idea intuitiva. Si p es primo, $\binom{n}{k} \bmod p$ se puede calcular en $O(1)$ con factoriales precomputados: $\binom{n}{k} = \frac{n!}{k!(n-k)!} = n! \cdot (k!)^{-1} \cdot ((n-k)!)^{-1} \pmod{p}$. Las inversas se precomputan una vez con $fact^{-1}[n] = fact[n]^{p-2}$ y se propaga hacia abajo. La optimizacion clave es propagar $invFact[i-1] = invFact[i] \cdot i$ hacia abajo en vez de calcular i^{p-2} para cada i .

Propiedades clave (invariantes).

- Precompute en $O(N)$, query en $O(1)$.
- Asume p primo; si p no es primo, la inversa modular no siempre existe.
- $\binom{n}{k} = 0$ si $k > n$.
- Simetria: $\binom{n}{k} = \binom{n}{n-k}$ (util para reducir k).
- $\sum_k \binom{n}{k} = 2^n$.

Codigo clave. Construir factoriales + inversas en $O(N)$:

```
fact[0] = 1;
for (int i = 1; i <= N; ++i) fact[i] = (long long)fact[i-1] * i % MOD;
invFact[N] = modpow(fact[N], MOD - 2, MOD);
for (int i = N - 1; i >= 0; --i) invFact[i] = (long long)invFact[i+1] * (i+1) % MOD;

long long nCr(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * invFact[k] % MOD * invFact[n-k] % MOD;
}
```

Complejidad.

- Precompute $O(N)$, query $O(1)$.
- Memoria $O(N)$.

Donde tocar para modificar.

- **n grande con p primo pequeno** ($\leq 10^6$): usar Lucas theorem.
- **Cambiar mod:** anadir nuevo MOD y regenerar fact/invFact.
- **Sumar $\binom{n}{k}$ sobre k :** usar DP o suma geometrica.
- **Permutaciones:** $P(n, k) = \binom{n}{k} \cdot k! = fact[n] \cdot invFact[n - k]$.
- **Recurrence:** $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ (util en DP).

Trampas y errores frecuentes.

- MOD = 1 no es primo, falla en modpow(fact[n], MOD - 2).
- buildFact(0) deja fact = {1}; nCr(0, 0) = 1 OK.
- Si $N > p$, el factorial crece y eventualmente $\binom{n}{k} \bmod p$ puede no ser el esperado sin Lucas.
- Para $k > N$ pero $k \leq n$, el resultado es 0; chequealo en la funcion.

Explicacion linea por linea. Preprocesamiento buildFact:

- const long long MOD = 998244353LL; – define el primo para todas las operaciones.
- vector<long long> fact, invFact; – arreglos globales para factoriales y factoriales inversos.
- void buildFact(int n) – precomputa factoriales e inversos hasta n .
- fact.assign(n + 1, 1); – inicializa fact con $n + 1$ unos (luego se sobrescriben).
- invFact.assign(n + 1, 1); – inicializa invFact de forma similar.
- for(int i=1 ; i<=n ; i++) fact[i] = fact[i-1] * i % MOD; – calcula $i! \bmod p$ incrementalmente.
- invFact[n] = modpow(fact[n], MOD - 2, MOD); – calcula $(n!)^{-1}$ via Fermat.
- for(int i=n ; i>=1 ; i--) invFact[i-1] = invFact[i] * i % MOD; – propaga hacia abajo: $((i-1)!)^{-1} = (i!)^{-1} \cdot i$.

Consulta nCr:

- `long long nCr(int n, int k)` – retorna $\binom{n}{k} \bmod p$.
- `if(k < 0 || k > n) return 0;` – valida el rango de k .
- `return fact[n] * invFact[k] % MOD * invFact[n-k] % MOD;` – aplica $\frac{n!}{k!(n-k)!}$ con aritmética modular.

Programa principal:

- `buildFact(100);` – precomputa factoriales hasta 100.
- `cout << "C(10,3) = " << nCr(10, 3) << "\textbackslash n";` – imprime $\binom{10}{3} = 120$.
- `cout << "C(100,50) mod = " << nCr(100, 50) << "\textbackslash n";` – imprime $\binom{100}{50} \bmod 998244353$.

Codigo. Ver `cpp.tex`, seccion *Factoriales + nCr mod p*.

2.3 Lucas theorem

Idea intuitiva. Para p primo y n, k grandes, $\binom{n}{k} \bmod p$ se calcula descomponiendo n y k en base p : $n = n_0 + n_1p + n_2p^2 + \dots$, $k = k_0 + k_1p + \dots$. Entonces $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}$ (**Lucas**). Cada $\binom{n_i}{k_i}$ se calcula con el snippet anterior. La demostracion usa que $(1+x)^n \equiv \prod_i (1+x^{p^i})^{n_i}$ en $\mathbb{F}_p[x]$ (por el “freshman’s dream” $a^p \equiv a \pmod{p}$).

Propiedades clave (invariantes).

- Solo funciona con p primo.
- Si algun $k_i > n_i$, el coeficiente es 0.
- Reduce el problema a $\log_p(n)$ coeficientes chiquitos.
- Cada $\binom{n_i}{k_i}$ se computa con factoriales precomputados hasta $p-1$.

Codigo clave. Lucas iterativo (descomponiendo en base p):

```
long long lucas(long long n, long long k, long long p) {
    long long res = 1;
    while (n > 0 || k > 0) {
        int ni = n % p, ki = k % p;
        if (ki > ni) return 0;
        res = res * nCr_small(ni, ki) % p; // usa fact hasta p-1
        n /= p; k /= p;
    }
    return res;
}
```

Complejidad.

- $O(\log_p n)$ multiplicaciones, cada una requiere un $\binom{n_i}{k_i}$ que es $O(1)$ con factoriales precomputados.
- Para $p = 10^6$ y $n = 10^{18}$: ~ 12 iteraciones.

Donde tocar para modificar.

- **p no primo:** requiere extender Lucas a modulos compuestos (no trivial, basado en factorizacion).
- **Devolver la suma:** si necesitas $\sum_k \binom{n}{k} \bmod p$, es $2^n \bmod p$ via Fermat (cuando p es primo).
- **Multiples queries:** precomputar todos los $\binom{n_i}{k_i}$ en una tabla y consultar.

Trampas y errores frecuentes.

- `buildFact(MOD)` solo precomputa hasta `MOD`, no hasta `n`. Pero como cada $\binom{n_i}{k_i}$ tiene $n_i < p$, eso es suficiente.
- Si p es muy chico (≤ 100), los factoriales repiten valores (`fact[p] = 0`); eso no rompe Lucas pero aniadir logica.
- Lucas **no** funciona para modulos no primos sin extension (algoritmo basado en factorizacion).

Explicacion linea por linea. Funcion `nCrLucas`:

- `long long nCrLucas(long long n, long long k, long long MOD)` – calcula $\binom{n}{k} \bmod p$ para n, k grandes via Lucas.
- `if(k == 0) return 1;` – caso base: $\binom{n}{0} = 1$.
- `long long ni = n % MOD, ki = k % MOD;` – extrae el digito menos significativo de n y k en base p .
- `if(ki > ni) return 0;` – si el digito de k excede el de n , $\binom{n_i}{k_i} = 0$.
- `return nCrLucas(n / MOD, k / MOD, MOD) * nCr((int)ni, (int)ki) % MOD;` – aplica la recurrencia: producto sobre los digitos.

Programa principal:

- `buildFact(MOD);` – precalcula los factoriales e inversos hasta $p - 1$.
- `cout << nCrLucas(1000, 100, MOD) << "\textbackslash n";` – evalua $\binom{1000}{100} \bmod 998244353$ usando Lucas.

Notar que `nCr` aqui es la version factorial con precomputo (ver `factorial_ncr.tex`).

Codigo. Ver `cpp.tex`, seccion *Lucas theorem*.

2.4 Stirling S2 (particiones)

Idea intuitiva. $S(n, k)$ = numero de formas de particionar n elementos en k subconjuntos no vacios. La recurrencia es $S(n, k) = S(n - 1, k - 1) + k \cdot S(n - 1, k)$ (o pones el elemento n -esimo solo en un nuevo subconjunto, o lo metes en uno de los k existentes). La recurrencia sale de un caso especial de la recurrencia general de inclusion-exclusion sobre funciones sobreyectivas.

Propiedades clave (invariantes).

- $S(n, 0) = [n = 0]$.
- $S(n, n) = 1, S(n, 1) = 1$.
- $\sum_k S(n, k) = B_n$ (numero de Bell).
- $S(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n$ (formula con inclusion-exclusion).
- $\sum_{k=0}^n S(n, k) \cdot k! =$ numero de funciones sobreyectivas de $[n]$ a $[k]$.

Codigo clave. DP bidimensional clasica:

```
S2[0][0] = 1;
for (int n = 1; n <= N; ++n)
    for (int k = 1; k <= n; ++k)
        S2[n][k] = (S2[n-1][k-1] + k * S2[n-1][k]) % MOD;
```

Complejidad.

- $O(N^2)$ tiempo y memoria.
- Para $N = 5000$: $\sim 25\text{MB}$ si se usa `int`; con `long long` el doble.

Donde tocar para modificar.

- **Stirling de primera especie:** recurrencia distinta; añadir nueva tabla.
- **Memory optimization:** solo guardar la fila anterior (DP en espacio $O(N)$).
- **Output sin mod:** si el conteo es chico, retornar `long long` y omitir `% MOD`.
- **Bell numbers:** $\sum_k S(n, k)$ por fila.

Trampas y errores frecuentes.

- Olvidar inicializar `S2[0][0] = 1` y el resto en 0.
- Overflow sin modulo.
- $S(n, k)$ crece muy rapido; sin modulo desborda para $n \geq 20$.
- El termino $k \cdot S(n - 1, k)$ es donde se acumula la mayoria del costo; en modulo p , esto es $O(1)$ por iteracion.

Explicacion linea por linea. **Stirling S2 (particiones en subconjuntos)** `buildStirlingS2`:

- `vector<vector<long long>> S2;` – tabla bidimensional $S2[n][k]$.
- `void buildStirlingS2(int n)` – llena la tabla para $i, k \leq n$.
- `S2.assign(n + 1, vector<long long>(n + 1, 0));` – inicializa a 0.
- `S2[0][0] = 1;` – caso base: un solo modo de particionar 0 elementos en 0 conjuntos.
- `for(int i=1; i<=n; i++)` – itera sobre el numero de elementos.
- `for(int j=1; j<=i; j++)` – itera sobre el numero de subconjuntos (no puede exceder i).
- `S2[i][j] = (S2[i-1][j-1] + j * S2[i-1][j]) % MOD;` – recurrencia: elemento i forma un bloque nuevo ($S2[i-1][j-1]$) o se une a uno de los j existentes ($j \cdot S2[i-1][j]$).

Stirling S1 (permutaciones por ciclos) `buildStirlingS1`:

- `vector<vector<long long>> S1;` – tabla bidimensional $S1[n][k]$.
- `void buildStirlingS1(int n)` – llena la tabla para permutaciones con k ciclos.
- `S1.assign(n + 1, vector<long long>(n + 1, 0));` – inicializa a 0.
- `S1[0][0] = 1;` – caso base: permutacion vacia con 0 ciclos.
- `for(int i=1; i<=n; i++)` – itera sobre el numero de elementos.
- `for(int j=1; j<=i; j++)` – itera sobre el numero de ciclos.
- `S1[i][j] = (S1[i-1][j-1] + (long long)(i - 1) * S1[i-1][j]) % MOD;` – recurrencia: i forma un ciclo propio ($S1[i-1][j-1]$) o se inserta en uno de los $(i-1)$ huecos disponibles en los ciclos previos ($(i-1) \cdot S1[i-1][j]$).

Codigo. Ver `cpp.tex`, seccion *Stirling S2 (particiones)*.

2.5 Inclusion-exclusion over subsets

Idea intuitiva. Para contar elementos que cumplen **alguna** de n propiedades, la formula clasica de inclusion-exclusion es: $|\bigcup A_i| = \sum_{\emptyset \neq I} (-1)^{|I|+1} |\bigcap_{i \in I} A_i|$. Esto se computa iterando sobre todos los $2^n - 1$ subconjuntos no vacios y aplicando el signo correspondiente. La demostracion usa que la formula del PIE cancela exactamente las sobreposiciones multiples, dejando solo los elementos que cumplen al menos una propiedad.

Propiedades clave (invariantes).

- El usuario pasa una funcion `f(mask)` que cuenta cuantos elementos cumplen **exactamente** las propiedades en `mask`.
- El signo es $(-1)^{|I|+1}$, equivalente a $+1$ si $|I|$ impar, -1 si par.
- Para $n \leq 20$ es factible ($2^{20} \approx 10^6$); para mas, usar otra tecnica (Mobius, formula cerrada).
- Variantes: “todos cumplen” = $(-1)^{|I|}$ (signo invertido).

Codigo clave. Inclusion-exclusion clasica sobre todos los subconjuntos:

```
long long ans = 0;
for (int mask = 1; mask < (1 << n); ++mask) {
    long long cnt = f(mask); // |interseccion|
    if (__builtin_popcount(mask) % 2 == 1) ans += cnt;
    else ans -= cnt;
}
return ans;
```

Complejidad.

- $O(2^n \cdot \text{costo de } f)$.
- Memoria $O(1)$ (no se almacena subconjunto).

Donde tocar para modificar.

- **Subset de subconjuntos** (elementos con un subconjunto de propiedades): usar bitmask para mapear cada elemento a un mask, luego contar ocurrencias por mask y sumar con signo.
- **Aniadir la interseccion:** en lugar de $|\bigcap A_i|$ exacta, aproximar o usar otra estructura.
- **Cambiar el signo:** si tu problema tiene inclusion-exclusion “inversa”, invertir el patron $\pm$.
- **Version con Mobius:** si las propiedades son divisibilidad, $\mu(d)$ reemplaza el patron $\pm$.

Ejemplo. Contar elementos de $[1, N]$ no divisibles por ninguno de $\{2, 3, 5\}$:

$$N - \lfloor N/2 \rfloor - \lfloor N/3 \rfloor - \lfloor N/5 \rfloor + \lfloor N/6 \rfloor + \lfloor N/10 \rfloor + \lfloor N/15 \rfloor - \lfloor N/30 \rfloor.$$

Trampas y errores frecuentes.

- El snippet dado es solo el **esqueleto** (`f` no esta implementado); hay que adaptarlo al problema.
- Para $n > 25$ la iteracion sobre 2^n se vuelve intratable; buscar otra estructura.
- El signo se calcula con el bit menos significativo (LSB) si la mascara lo tiene, o con `popcount` modulo 2.
- Si $|I|$ es par pero muy grande, considerar $2^{|I|}$ “pesado”; aniadir memoizacion en `f`.

Explicacion linea por linea. Plantilla inclusionExclusion:

- `template<class F>` – permite pasar cualquier funcion/lambda como argumento.
- `long long inclusionExclusion(int n, F&& f)` – recibe el numero de propiedades y la funcion que cuenta intersecciones.
- `long long result = 0;` – acumulador del resultado.
- `int total = 1 << n;` – numero total de subconjuntos (2^n).
- `for(int mask=1; mask<total; mask++)` – itera sobre todos los subconjuntos no vacios.
- `long long cnt = f(mask);` – evalua la funcion para este subconjunto: $|\bigcap_{i \in \text{mask}} A_i|$.
- `int bits = __builtin_popcount(mask);` – numero de bits encendidos (tamano del subconjunto).
- `if(bits % 2 == 1) result += cnt;` – si $|I|$ impar, suma (signo +1).
- `else result -= cnt;` – si $|I|$ par, resta (signo -1).

Programa principal (ejemplo):

- `int primes[] = {2,3,5};` – las propiedades son “ser divisible por 2, 3 o 5”.
- `for(int p : primes) if(mask & (1<<(p==2?0:p==3?1:2))) { /* set bit */ }` – mapea cada primo a su bit correspondiente.
- `return cnt;` – retorna el conteo para este subconjunto.

Codigo. Ver `cpp.tex`, seccion

2.3 Polinomios y transformadas

2.1 FFT (iterativo, complex)

Idea intuitiva. La **Transformada Rapida de Fourier** calcula el DFT de un vector en $O(N \log N)$. La inversa tambien. La convolucion de dos polinomios $A(x) \cdot B(x)$ se obtiene como $\text{IFFT}(\text{FFT}(A) \cdot \text{FFT}(B))$ componente a componente. La version **iterativa** reordena los elementos por bit-reversal antes y aplica “mariposas” (combina pares en $O(1)$). La intuicion matematica: cualquier secuencia $a_0, a_1, \dots, a_{n-1}$ se puede escribir como $\sum_k c_k \omega^{jk}$ donde $\omega = e^{2\pi i/n}$ y c_k son los coeficientes de Fourier; el FFT calcula c_k rapidamente usando el “divide y venceras” sobre la paridad de los indices.

Propiedades clave (invariantes).

- Trabaja con `complex<double>`; hay errores de redondeo que se acumulan.
- Tamano N debe ser potencia de 2 (se hace `n <= 1` hasta alcanzar `a.size() + b.size()`).
- Resultado se redondea con `llround`.
- Parseval: $\sum |a_i|^2 = \frac{1}{N} \sum |A_k|^2$ (util para verificar).

Codigo clave. El corazon del FFT iterativo (mariposa):

```
for (int len = 2; len <= n; len <= 1) {
    double ang = 2 * PI / len * invert; // invert = -1 para FFT, +1 para IFFT
    complex<double> wlen(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
        complex<double> w(1);
        for (int j = 0; j < len/2; ++j) {
            complex<double> u = a[i+j], v = a[i+j+len/2] * w;
            a[i+j] = u + v;
            a[i+j+len/2] = u - v;
            w *= wlen;
        }
    }
}
```

Complejidad.

- $O(N \log N)$ con N la primera potencia de 2 que cubre el resultado.
- Constante ~ 5 para el iterative, ~ 1 para recursive (pero recursive tiene overhead).

Donde tocar para modificar.

- **Modulo arbitrario:** aniadir tres FFTs y “FFT con tres modulos” para reconstruir el resultado con CRT (mejor precision).
- **Tamano mas pequeno:** si sabes que el resultado cabe en $K < a.size() + b.size() - 1$, podes usar $N = 2^{\lceil \log_2 K \rceil}$ para ahorrar.
- **Cambiar precision:** aniadir `long double` y `acosl(-1)` para mayor precision (marginal).
- **Bluestein:** para tamano no potencia de 2, bluestein permite NTT/FFT exacto.

Trampas y errores frecuentes.

- Coeficientes **negativos**: `llround(-0.5)` da 0, no -1; añadir `floor(x + 0.5)` si hay negativos.
- Overflow al convertir a `long long` si los coeficientes son grandes ($\geq 2^{52}$).
- La precision de `double` llega a $\sim 10^{-15}$; con $N = 10^5$ el error acumulado puede ser $\sim 10^{-3}$, problematico para enteros $> 10^{15}$.
- Si los coeficientes son `int` ($\leq 10^9$), FFT funciona; para mas, usar NTT mod primes.

Explicacion linea por linea. Funcion `fft`:

- `using cd = complex<double>;` – alias para números complejos de doble precision.
- `const double PI = acos(-1);` – define π de forma precisa.
- `void fft(vector<cd>& a, bool invert)` – aplica FFT (o IFFT si `invert=true`) sobre `a` en el lugar.
- `int n = a.size();` – tamaño del vector (debe ser potencia de 2).
- `for(int i=1,j=0;i<n;i++)` – permutacion bit-reversal: reordena para que la recursion iterativa funcione.
- `int bit = n >> 1;` – arranca con el bit mas alto.
- `for(; j<bit; bit>>=1) j = (j ^ (j >> bit)) << bit;` – busca el bit a flippear.
- `j = (j ^ (j >> bit)) << bit;` – realiza el flip.
- `if(i < j) swap(a[i], a[j]);` – intercambia los pares correspondientes.
- `for(int len=2; len<=n; len<<=1)` – itera sobre los niveles de la mariposa (longitud de bloque).
- `double ang = 2 * PI / len * (invert ? -1 : 1);` – angulo de la raiz n -esima correspondiente (invertido para IFFT).
- `cd wlen(cos(ang), sin(ang));` – raiz n -esima de la unidad w_{len} .
- `for(int i=0; i<n; i+=len)` – itera sobre cada bloque de longitud `len`.
- `cd w(1);` – factor w que avanza geometricamente.
- `for(int j=0; j<len/2; j++)` – mitad del bloque (la otra mitad usa el conjugado).
- `cd u = a[i+j], v = a[i+j+len/2] * w;` – mariposa que combina el par $(u, v \cdot w)$.
- `a[i+j] = u + v;` – almacena la suma.
- `a[i+j+len/2] = u - v;` – almacena la diferencia.
- `w *= wlen;` – avanza w por la raiz de la unidad.
- `for(cd& x : a) x /= n;` – si es IFFT, divide entre n para invertir la transformacion.

Funcion `multiplyFFT`:

- `vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());` – copia a complejos.
- `while(n < (int)a.size() + (int)b.size()) n <<= 1;` – encuentra la potencia de 2 que cubra el resultado.
- `fa.resize(n); fb.resize(n);` – rellena con ceros (padding).
- `fft(fa, false); fft(fb, false);` – aplica FFT directo.
- `for(int i=0;i<n;i++) fa[i] *= fb[i];` – multiplica punto a punto (convolucion en el dominio de Fourier).
- `fft(fa, true);` – aplica IFFT para volver al dominio del tiempo.
- `res[i] = (long long)llround(fa[i].real());` – redondea al entero mas cercano (los imaginarios deben ser 0).
- `cout << "\textbackslash n";` – imprime los coeficientes del polinomio producto.

Codigo. Ver `cpp.tex`, seccion *FFT (iterativo, complex)*.

2.2 NTT (mod 998244353)

Idea intuitiva. La **Number Theoretic Transform** es la version entera de FFT. En vez de raices de unidad complejas, usa raices modulo un primo p con $p - 1$ divisible por una potencia grande de 2. El primo $998244353 = 119 \cdot 2^{23} + 1$ sirve para NTT hasta $2^{23} = 8 \cdot 10^6$. La existencia de tales primos se basa en buscar p con $p - 1$ altamente divisible por 2, lo cual requiere que p sea grande.

Propiedades clave (invariantes).

- **Exacto** modulo p (sin redondeo).
- Raiz primitiva $g = 3$.
- Misma estructura que FFT (bit-reversal + mariposas).
- $\omega_n = g^{(p-1)/n}$ es una raiz n -esima de la unidad en $\mathbb{F}_p$.

Codigo clave. NTT iterativo (la mariposa es analoga a FFT, con multiplicacion modular):

```
for (int len = 2; len <= n; len <= 1) {
    long long wlen = modpow(ROOT, (MOD - 1) / len, MOD);
    if (invert) wlen = modpow(wlen, MOD - 2, MOD);
    for (int i = 0; i < n; i += len) {
        long long w = 1;
        for (int j = 0; j < len / 2; ++j) {
            long long u = a[i+j], v = a[i+j+len/2] * w % MOD;
            a[i+j] = (u + v) % MOD;
            a[i+j+len/2] = (u - v + MOD) % MOD;
            w = w * wlen % MOD;
        }
    }
}
if (invert) {
    long long n_inv = modpow(n, MOD - 2, MOD);
    for (auto& x : a) x = x * n_inv % MOD;
}
```

Complejidad.

- $O(N \log N)$.
- Para 998244353 y $N \leq 2^{23}$: un par de milisegundos.

Donde tocar para modificar.

- **Otro primo:** $985661441 = 235 \cdot 2^{22} + 1$, raiz 3; $754974721 = 45 \cdot 2^{24} + 1$, raiz 11. Ajustar MOD y PRIMITIVE_ROOT.
- **Multiplicacion de varias cosas:** NTT es lineal; anadir cache si llamas con los mismos vectores.
- **Poly inv (Newton):** ver el modulo de polynomial utilities.
- **Tres NTTs + CRT:** si necesitas modulo $10^9 + 7$ con coeficientes grandes, hacer NTT con tres primos y reconstruir.

Trampas y errores frecuentes.

- El “exponente magico” $(MOD - 1) / len$ requiere que $MOD - 1$ sea divisible por len ; para 998244353, len debe ser potencia de 2 hasta 2^{23} .
- Si len no divide $MOD - 1$, el algoritmo da basura.
- Confundir la raiz primitiva g con la raiz n -esima $g^{(p-1)/n}$; esta ultima depende del nivel.
- En el inverse, olvidar el factor $1/n$ da basura escalada.

Explicacion linea por linea. Funcion ntt:

- `const long long PRIMITIVE_ROOT = 3LL;` – raiz primitiva modulo 998244353 (genera todo el grupo).
- `void ntt(vector<long long>& a, bool invert)` – aplica NTT o INTT (inversa) sobre `a`.
- `int n = a.size();` – tamaño (potencia de 2, requisito del algoritmo).
- `for(int i=1,j=0;i<n;i++)` – permutacion bit-reversal para colocar elementos en orden.
- `int bit = n >> 1;` – empieza con el bit mas significativo.
- `for(; j<bit; bit>>=1) j` – busca el bit a flippear.
- `j` – flip y swap condicional.
- `for(int len=2; len<=n; len<<=1)` – niveles de mariposa (longitud doblada).
- `long long wlen = modpow(PRIMITIVE_ROOT, (MOD - 1) / len, MOD);` – raiz n -esima: $g^{(p-1)/len}$.
- `if(invert) wlen = modpow(wlen, MOD - 2, MOD);` – invierte la raiz para INTT.
- `for(int i=0; i<n; i+=len)` – itera sobre bloques.
- `long long w = 1;` – acumulador de potencias w^j .
- `long long u = a[i+j], v = a[i+j+len/2] * w % MOD;` – elementos de la mariposa.
- `a[i+j] = (u + v) % MOD;` – suma de mariposa.
- `a[i+j+len/2] = (u - v + MOD) % MOD;` – diferencia de mariposa (ajustada a positivo).
- `w = w * wlen % MOD;` – avanza w multiplicando por la raiz.
- `if(invert) { for(auto& x : a) x = x * n_inv % MOD; }` – divide entre n para invertir NTT.

Funcion multiplyNTT:

- `vector<long long> fa, fb;` – copias en formato NTT.
- `while(n < (int)a.size() + (int)b.size()) n <<= 1;` – potencia de 2 que cubra el resultado.
- `fa.resize(n); fb.resize(n);` – rellena con ceros (padding).
- `ntt(fa, false); ntt(fb, false);` – aplica NTT directo.
- `fa[i] = fa[i] * fb[i] % MOD;` – multiplicacion punto a punto.
- `ntt(fa, true);` – INTT para volver al dominio del polinomio.

Codigo. Ver `cpp.tex`, seccion *NTT (mod 998244353)*.

2.3 Polynomial utilities (mod p)

Idea intuitiva. Capa de utilidades sobre NTT/FFT: suma, resta, **inversa** (Newton iterativo), derivada, integral. La inversa usa que si $B \cdot B^{-1} \equiv 1 \pmod{x^n}$, entonces aproximar a $2n$ y doblar la precision en cada paso. La recurrencia de Newton: si r_k es inversa mod x^{2^k} , entonces $r_{k+1} \equiv r_k \cdot (2 - B \cdot r_k) \pmod{x^{2^{k+1}}}$. La intuicion: r_{k+1} satisface $B \cdot r_{k+1} \equiv 1 + (B \cdot r_k - 1)^2 \pmod{x^{2^{k+1}}}$ y el error cuadratico decae mas rapido.

Propiedades clave (invariantes).

- `polyInv(b, n)`: asume `b[0]` invertible (no cero).
- Usa NTT internamente; misma complejidad $O(N \log N)$.
- `polyInteg` necesita el inverso de $i+1 \pmod p$; se precomputa en una `static vector` que crece a demanda.
- Cada iteracion de Newton dobla la precision.

Codigo clave. Newton iterativo para inversa polinomial:

```
vector<int> polyInv(const vector<int>& b, int n) {
    vector<int> r(1, modpow(b[0], MOD - 2, MOD));
    int cur = 1;
    while (cur < n) {
        cur <<= 1;
        vector<int> b_cut(b.begin(), b.begin() + min((int)b.size(), cur));
        vector<int> nr = r * r % MOD * b_cut % MOD; // 2*r - r^2*b
        // nr = 2*r - r^2 * b mod x^cur
        nr.resize(cur);
        r = (2LL * r[0] * ...); // simplificacion algebraica
        r = nr;
    }
    r.resize(n);
    return r;
}
```

Complejidad.

- Operaciones polinomiales $O(N \log N)$.
- Newton converge en $O(\log n)$ pasos; cada paso $O(N \log N)$.

Donde tocar para modificar.

- **Division:** anadir `polyDiv(a, b)` usando la inversa de b truncada.
- **Multi-point evaluation:** $O(N \log^2 N)$ con divide y venceras.
- **Composition:** muy dificil en general; no recomendado en contests.
- **Logaritmo polinomial:** $\log(f) = \int f'/f$, requiere inversa de f .

Trampas y errores frecuentes.

- `polyInv` asume `b[0]` no es 0 modulo p ; sino, no existe la inversa.
- Olvidar `r.resize(cur)` deja residuos de la iteracion anterior.
- En Newton, r se modifica en cada paso; no reusar la misma variable.
- Los vectores de NTT deben tener tamano potencia de 2; anadir padding si hace falta.

Explicacion linea por linea. Suma y resta:

- `using Poly = vector<long long>;` – alias para polinomio (vector de coeficientes).
- `polyAdd(...)` – suma componente a componente.
- `Poly c(max(a.size(), b.size()));` – crea resultado del tamano del mayor.
- `c[i] = (c[i] + a[i]) % MOD;` – suma coeficientes de a .
- `c[i] = (c[i] + b[i]) % MOD;` – suma coeficientes de b .
- `polySub(...)` – resta componente a componente (mod p).
- `c[i] = (c[i] - b[i] + MOD) % MOD;` – resta y ajusta a positivo.

Inversa polinomial `polyInv`:

- `Poly r(1, modpow(b[0], MOD - 2, MOD));` – $r \equiv 1/b_0 \pmod{x}$ (termino constante).
- `int cur = 1;` – precision actual.
- `while(cur < n)` – duplica precision hasta alcanzar n .
- `cur <<= 1;` – dobla la precision objetivo.

- Poly `a(min((int)b.size(), cur));` – trunca b a la precision actual.
- Poly `nr = multiplyNTT(a, multiplyNTT(r, r));` – calcula $b \cdot r^2$ truncado (NTT).
- `r.resize(cur);` – prepara r para la nueva precision.
- `r[i] = (2LL * r[i] - nr[i] + MOD) % MOD;` – Newton: $r' = 2r - b \cdot r^2 \pmod{x^{\text{cur}}}$.
- `r.resize(n);` – trunca al tamaño pedido.

Derivada `polyDeriv`:

- `if(a.empty()) return {};` – polinomio vacio.
- Poly `b((int)a.size() - 1);` – derivada tiene un termino menos.
- `b[i-1] = a[i] * i % MOD;` – $b'_i = (i+1) \cdot a_{i+1}$.

Integral `polyInteg`:

- Poly `b(a.size() + 1);` – integral tiene un termino mas.
- `static vector<long long> inv(1, 1);` – cache de inversas $(i+1)^{-1} \pmod p$.
- `inv.push_back(modpow(inv.size(), MOD - 2, MOD));` – precomputa inversas a demanda.
- `b[i+1] = a[i] * inv[i+1] % MOD;` – $b_{i+1} = a_i / (i+1)$.

Codigo. Ver `cpp.tex`, seccion

2.4 Algebra lineal

2.1 Gauss-Jordan (mod p)

Idea intuitiva. Reduccion gaussiana para resolver sistemas $Ax = b$ y calcular determinante y rango. En la version **Gauss-Jordan**, se elimina hacia arriba y hacia abajo (obteniendo la forma escalonada reducida por filas, RREF). El algoritmo opera sobre la matriz aumentada $[A \mid b]$ modulo p . La justificacion: las operaciones elementales de fila (intercambio de filas, multiplicacion por escalar no nulo y suma de un multiplo de una fila a otra) preservan el conjunto de soluciones del sistema y las propiedades del espacio vectorial.

Propiedades clave (invariantes).

- Rango r : numero de filas no nulas (o pivotes) tras la reduccion.
- **3 casos** para $Ax = b$ con matriz $n \times n$:
 - **Sin solucion** (inconsistente): existe alguna fila nula en los coeficientes de A cuyo termino independiente es no nulo ($\text{aug}[i][n] \neq 0$ para algun $i \geq r$).
 - **Solucion unica**: sistema consistente y $r = n$ (rango igual al numero de incognitas).
 - **Infinitas soluciones**: sistema consistente y $r < n$ (quedan $n - r$ variables libres).
- Determinante: producto de los elementos diagonales en la forma triangular, multiplicado por $(-1)^s$ donde s es el numero de swaps de filas realizados (en mod p : `det = (MOD - det) % MOD`).
- En el caso de infinitas soluciones, fijar las variables libres en 0 permite obtener una solucion particular de forma directa.

Codigo clave. Eliminacion Gauss-Jordan completa (RREF in-place modulo p):

```
int gaussJordan(vector<vector<long long>>& a) {
    int n = a.size(), m = a[0].size(), rank = 0, row = 0;
    for (int col = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i) if (a[i][col]) { sel = i; break; }
        if (!a[sel][col]) continue;
        swap(a[sel], a[row]);
        long long inv = modpow(a[row][col], MOD - 2, MOD);
```

```

    for (int j = col; j < m; ++j) a[row][j] = a[row][j] * inv % MOD;
    for (int i = 0; i < n; ++i) {
        if (i != row && a[i][col]) {
            long long factor = a[i][col];
            for (int j = col; j < m; ++j) {
                a[i][j] = (a[i][j] - factor * a[row][j] % MOD + MOD) % MOD;
            }
        }
    }
    ++row; ++rank;
}
return rank;
}

```

Complejidad.

- $O(N^3)$ para una matriz cuadrada $N \times N$ (en general $O(N \cdot M \cdot \min(N, M))$ para N filas y M columnas).
- La eliminacion o triangularizacion es $O(N^3)$; la sustitucion regresiva en la eliminacion gaussiana clasica es $O(N^2)$.
- Para $N \leq 500$: $\sim 6 \cdot 10^7$ operaciones modulares, perfectamente factible en ≤ 1 segundo.

Donde tocar para modificar.

- **Sin modulo (numeros reales)**: usar `double`, pivoteo parcial eligiendo el elemento con mayor `fabs(a[i][col])` y comparar con un umbral `eps` ($\approx 10^{-9}$) en vez de 0.
- **Modulo 2 (GF(2) / XOR)**: usar `std::bitset` para representar las filas. Las sumas se vuelven XORs y la complejidad se reduce a $O(N^3/64)$, permitiendo $N \leq 2000 \sim 4000$.
- **Solucion particular con libres en 0**: el snippet lo hace de manera natural; para asignar otros valores a las variables libres, inicializar las columnas no pivote.
- **Rango sobre matriz sin aumentar**: pasar directamente la matriz de coeficientes A ($n \times m$), la funcion retornara su rango.
- **Matriz de Vandermonde**: si la matriz tiene estructura de Vandermonde, se puede resolver o invertir en $O(N^2)$ mediante polinomios de Lagrange/Newton.

Trampas y errores frecuentes.

- **Sobrescritura prematura del factor**: calcular y guardar `long long factor = a[i][col]`; antes de entrar al bucle de columnas j . Si se usa directamente `a[i][col]` al actualizar `a[i][j]`, cuando $j = col$ se vuelve cero y anula la eliminacion en el resto de la fila.
- **Restas negativas en C++**: la operacion `(a - b) % MOD` puede ser negativa; siempre usar `(a - b % MOD + MOD) % MOD`.
- **Division por cero y modulo no primo**: se requiere el inverso modular de cada pivote. Esto asume que el modulo p es primo (usando $a^{p-2} \bmod p$). Si el modulo es compuesto, se debe usar eliminacion gaussiana basada en el algoritmo de Euclides.
- **Signo del determinante**: cada swap de filas invierte el signo del determinante; en modulo p , multiplicar por -1 equivale a `det = (MOD - det) % MOD`.
- **Inestabilidad numerica**: con `double`, siempre usar pivoteo parcial (buscar el maximo valor absoluto en la columna) para minimizar errores de precision por punto flotante.

Explicacion linea por linea. Funcion gaussJordan:

- `int gaussJordan(vector<vector<long long>>& a)` – reduccion RREF in-place; retorna el rango.
- `int n = a.size(), m = a[0].size(), rank = 0, row = 0;` – dimensiones de la matriz, rango y fila pivote actual.

- `for(int col=0; col<m && row<n; col++)` – avanza columna por columna mientras haya filas disponibles.
- `int sel = row;` – índice candidato para la fila pivote.
- `for(int i=row; i<n; i++) if(a[i][col]){ sel = i; break; }` – busca la primera fila con pivote no cero en esta columna.
- `if(!a[sel][col]) continue;` – si toda la columna debajo de `row` es cero, es variable libre; salta a la siguiente columna.
- `swap(a[sel], a[row]);` – mueve la fila elegida a la posición `row`.
- `long long inv = modpow(a[row][col], MOD - 2, MOD);` – calcula el inverso modular del pivote.
- `for(int j=col; j<m; j++) a[row][j] = a[row][j] * inv % MOD;` – normaliza la fila pivote para que el pivote sea 1.
- `for(int i=0; i<n; i++)` – elimina la columna en todas las demás filas (hacia arriba y hacia abajo).
- `if(i != row && a[i][col])` – evita la propia fila pivote y filas que ya tienen cero en esta columna.
- `long long factor = a[i][col];` – guarda el factor antes de modificar la fila para no sobrescribirlo.
- `a[i][j] = (a[i][j] - factor * a[row][j] % MOD + MOD) % MOD;` – resta el múltiplo de la fila pivote.
- `row++; rank++;` – avanza a la siguiente fila y cuenta un nuevo pivote en el rango.

Funcion determinant:

- `long long det = 1;` – determinante inicializado en 1.
- `for(int i=0; i<n; i++)` – eliminación gaussiana para triangularizar la matriz.
- `int sel = i;` – busca la fila pivote a partir de la fila i .
- `for(int j=i; j<n; j++) if(a[j][i]){ sel = j; break; }` – encuentra una fila con elemento no cero en la diagonal/columna.
- `if(!a[sel][i]) return 0;` – si todos son cero, el rango es menor a n y el determinante es 0.
- `if(sel != i){ swap(a[sel], a[i]); det = (MOD - det) % MOD; }` – swap de filas invierte el signo del determinante.
- `det = det * a[i][i] % MOD;` – multiplica el determinante por el elemento diagonal (pivote).
- `long long inv = modpow(a[i][i], MOD - 2, MOD);` – inversa del pivote para eliminar por debajo.
- `for(int j=i+1; j<n; j++)` – itera sobre las filas por debajo de la diagonal.
- `long long factor = a[j][i] * inv % MOD;` – factor de eliminación para la fila j .
- `a[j][k] = (a[j][k] - factor * a[i][k] % MOD + MOD) % MOD;` – elimina la entrada bajo el pivote en toda la fila.

Funcion solveSystem:

- `vector<vector<long long>> aug(n, vector<long long>(n + 1));` – construye la matriz aumentada $[A \mid b]$ de $n \times (n + 1)$.
- `aug[i][j] = A[i][j]; aug[i][n] = b[i];` – copia los coeficientes de A y el vector b .
- `int r = gaussJordan(aug);` – reduce a RREF y obtiene el rango.
- `for(int i = r; i < n; i++) if(aug[i][n] != 0) return 0;` – verifica inconsistencia: fila nula con $b_i \neq 0$ (retorna 0 = sin solución).
- `if(r < n) return 2;` – rango menor que n : variables libres (retorna 2 = infinitas soluciones).
- `sol.assign(n, 0); for(int i=0; i<n; i++) sol[i] = aug[i][n];` – lee la solución particular de la última columna.
- `return 1;` – retorna 1 indicando solución única almacenada en `sol`.

Codigo. Ver `cpp.tex`, sección *Gauss-Jordan (mod p)*.

2.2 Matrix exponentiation

Idea intuitiva. Para calcular M^k en $O(N^3 \log k)$, se usa la misma idea que `modpow` pero la multiplicación es de matrices. Así, M^k se descompone en bits y se multiplican las matrices correspondientes. La idea subyacente: M^k actúa sobre vectores; si tenemos una recurrencia lineal $\mathbf{v}_{n+1} = A\mathbf{v}_n$, entonces $\mathbf{v}_k = A^k \mathbf{v}_0$, así que calcular A^k nos da la respuesta en $O(\log k)$.

Propiedades clave (invariantes).

- Multiplicación asociativa: $(M^a)(M^b) = M^{a+b}$.
- Identidad es la matriz diagonal con 1s.
- Si M es la matriz de transición de un sistema lineal (recurrencia, automata), M^k da el estado tras k pasos.
- Si M es diagonalizable, $M^k = PD^kP^{-1}$; útil para closed-form.

Código clave. El squaring matricial:

```
Mat operator*(const Mat& o) const {
    Mat r(n, vector<long long>(n, 0));
    for (int i = 0; i < n; ++i)
        for (int k = 0; k < n; ++k) if (a[i][k])
            for (int j = 0; j < n; ++j)
                r[i][j] = (r[i][j] + a[i][k] * o.a[k][j]) % MOD;
    return r;
}

Mat mpow(Mat M, long long e) {
    Mat R = identity(M.n);
    while (e > 0) {
        if (e & 1) R = R * M;
        M = M * M;
        e >>= 1;
    }
    return R;
}
```

Complejidad.

- $O(N^3 \log k)$.
- Para $N = 50$, $k = 10^{18}$: $\sim 6 \cdot 10^6$ operaciones.

Donde tocar para modificar.

- **Aplicar a recurrencia:** para $F_n = aF_{n-1} + bF_{n-2}$, construir $M = \begin{pmatrix} a & b \\ 1 & 0 \end{pmatrix}$, M^n da F_n en $M[0][0] * F_1 + M[0][1] * F_0$.
- **Multiplicación sin la optimización** `if (a[i][k])`: para matrices densas, la guarda no ayuda; quitarla.
- **Matrices pequeñas** ($N \leq 3$): hardcodear la multiplicación puede ser más rápido.
- **Camino count en grafo:** A^k da el número de caminos de largo k entre pares de nodos.

Trampas y errores frecuentes.

- Multiplicación `a[i][k] * b[k][j]` puede overflow; el snippet usa `% MOD` al final, pero el producto intermedio puede pasar 2^{63} si los valores son grandes y `MOD` es chico. Mejor hacer el módulo en cada paso.
- Para grafos con $N = 10^3$ y aristas, A^k no entra en memoria si no se usa sparse matrix.
- Construir mal la matriz de transición es la fuente más común de bugs; verificar con casos pequeños.

Explicacion linea por linea. Multiplicacion de matrices matMul:

- `using Matrix = vector<vector<long long>>;` – alias para una matriz $n \times n$.
- `void matMul(const Matrix& a, const Matrix& b, long long MOD)` – calcula $c = a \cdot b \bmod p$.
- `Matrix c(n, vector<long long>(n, 0));` – inicializa el resultado en ceros.
- `for(int i=0;i<n;i++)` – itera sobre filas de a .
- `for(int k=0;k<n;k++) if(a[i][k])` – itera sobre columnas de a (= filas de b), saltando ceros.
- `for(int j=0;j<n;j++)` – itera sobre columnas de b .
- `c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % MOD;` – acumula $a_{ik} \cdot b_{kj}$ en c_{ij} .

Exponenciacion matPow:

- `void matPow(Matrix base, long long exp, long long MOD)` – calcula $\text{base}^{\text{exp}} \bmod p$.
- `Matrix res(n, vector<long long>(n, 0));` – matriz resultado inicializada en ceros.
- `for(int i=0;i<n;i++) res[i][i] = 1;` – coloca los 1 en la diagonal (matriz identidad).
- `while(exp)` – bucle principal del squaring.
- `if(exp & 1) res = matMul(res, base, MOD);` – si el bit actual esta encendido, multiplica el resultado.
- `base = matMul(base, base, MOD);` – eleva al cuadrado la base.
- `exp >>= 1;` – divide el exponente entre 2.

Programa principal:

- `Matrix T = {{1,1},{1,0}};` – matriz de transicion clasica de Fibonacci.
- `Matrix Fn = matPow(T, 10, MOD);` – calcula T^{10} en $O(\log 10)$ multiplicaciones.
- `cout << "F(10) = " << Fn[0][1] << "\textbackslash n";` – imprime $F_{10} = 55$.

Codigo. Ver `cpp.tex`, seccion

2.5 Juegos y teoria de juegos

2.1 Sprague-Grundy template

Idea intuitiva. Para juegos **imparciales** (ambos jugadores tienen los mismos movimientos), el teorema de Sprague-Grundy dice que cada posicion tiene un numero (el **Grundy** o **mex**) y la posicion es **ganadora** para el jugador actual sii el XOR de los Grundy de los componentes es no cero. El **mex** (“minimum excludant”) es el menor entero no negativo que no esta en el conjunto de Grundy de las posiciones alcanzables. La demostracion: para cada posicion, hay un movimiento a cada Grundy menor; la posicion perdedora es exactamente Grundy = 0.

Propiedades clave (invariantes).

- Posiciones terminales (sin movimientos) tienen Grundy = 0.
- La composicion XOR de Grundys da el Grundy del juego disjunto.
- **Caso base:** si `state` no tiene transiciones, Grundy = 0.
- Si Grundy $\neq 0$, hay un movimiento a una posicion con Grundy 0 (estrategia ganadora).
- Si Grundy = 0, todos los movimientos van a Grundy $\neq 0$.

Codigo clave. Memoizacion clasica:

```
unordered_map<long long, long long> memo;
long long grundy(long long state) {
    if (memo.count(state)) return memo[state];
    set<long long> reachable;
    // populate 'reachable' with Grundy of all moves
    long long g = 0;
    while (reachable.count(g)) ++g;
    return memo[state] = g;
}
// Ganador: grundy(start) != 0
```

Complejidad.

- Depende del juego: $O(\text{estados} \cdot \text{transiciones})$ con memoization.
- Para Nim con N heaps: $O(N)$.

Donde tocar para modificar.

- **Adaptar al juego concreto:** reemplazar el comentario `// populate 'reachable'` ... con la logica del juego (ej. para Nim: anadir todos los heap sizes reducidos).
- **Memoization con map:** usar `unordered_map<long long, long long>` si el estado es grande.
- **Computar Grundy de varios estados independientes:** para Nim, es el XOR de los heap sizes; para Kayles, hay tabla precomputada.
- **Encontrar movimiento ganador:** anadir busqueda del Grundy 0 accesible.

Trampas y errores frecuentes.

- Olvidar memoizar: el algoritmo sin cache es exponencial.
- Considerar `unordered_map` vs `map` segun el tamano del espacio de estados.
- Si dos juegos tienen el mismo Grundy, no necesariamente son "iguales" (mex lo captura).
- mex sobre un set vacio es 0 (caso base).

Explicacion linea por linea. Funcion grundy:

- `unordered_map<long long, long long> sgMemo;` – cache memoizado: estado $\rightarrow$ Grundy.
- `long long grundy(long long state)` – calcula el Grundy (mex) del estado.
- `if(sgMemo.count(state)) return sgMemo[state];` – si ya calculado, retorna directo.
- `set<long long> reachable;` – conjunto de Grundys alcanzables (los "moves").
- `long long mex = 0;` – inicia la busqueda del mex.
- `while(reachable.count(mex)) mex++;` – incrementa hasta encontrar un entero no presente.
- `return sgMemo[state] = mex;` – memoiza y retorna.

Programa principal:

- `int g = grundy(1) ^ grundy(2) ^ grundy(3);` – XOR de los Grundys de las 3 pilas.
- `cout << (g ? "first player wins" : "second player wins") << "\textbackslash n";` – decide el ganador por el XOR total.

Codigo. Ver `cpp.tex`, seccion *Sprague-Grundy template*.

2.2 Small game tables (Kayles SG values)

Idea intuitiva. Cuando el juego tiene **pocas** posiciones conocidas, conviene **hardcodear** la tabla de Grundys para responder en $O(1)$ por query. Kayles es un juego clasico donde tiras un bolo y tumbas uno o dos adyacentes; sus Grundys para $n \leq 71$ son bien conocidos y estan precomputados. La razon de precomputar: el Grundy de Kayles satisface una recurrencia compleja y no se puede calcular con formula cerrada simple.

Propiedades clave (invariantes).

- Solo util para Kayles; otros juegos necesitan su propia tabla.
- La tabla se obtuvo experimentalmente (o por recurrencia) y se almacena literal en el codigo.
- Los Grundys tienen **periodo** tras cierto n (e.g., Kayles tiene periodo 12 tras $n = 72$).

Codigo clave. Si el problema tiene n en $[1, N]$ con N conocido, precomputar la tabla:

```
vector<int> grundy(N + 1, 0);
for (int n = 1; n <= N; ++n) {
    set<int> s;
    for (int k = 0; k < n; ++k) {
        // un bolo en k: kayles se parte en [0, k-1] y [k+1, n-1]
        s.insert(grundy[k] ^ grundy[n-1-k]);
        if (k+1 < n) s.insert(grundy[k] ^ grundy[n-2-k]);
    }
    int g = 0; while (s.count(g)) ++g;
    grundy[n] = g;
}
```

Complejidad.

- $O(1)$ por query (con tabla hardcodeada).
- Precompute $O(N^2)$.

Donde tocar para modificar.

- **Otro juego:** aniadir otra tabla literal (Nim, Dawson's Kayles, Grundy's Game, etc.).
- **Tabla generada por codigo:** aniadir un precompute al inicio que llene la tabla con el template Sprague-Grundy.
- **Usar periodicidad:** si el Grundy tiene periodo P a partir de N_0 , aniadir `int g = (n >= N0) ? tabla[(n - N0) % P] : tabla[n];`.

Trampas y errores frecuentes.

- Si el problema da n fuera de rango, aniadir bounds check antes de indexar.
- La tabla es especifica de la variante de Kayles (original, Dawson's, etc.); verificar cual es.
- En algunos juegos (Wythoff), la estrategia no es Grundy sino un par de moves explicitos.

Explicacion linea por linea.

- `#include <bits/stdc++.h>` – incluye todas las bibliotecas estandar.
- `using namespace std;` – evita prefijar `std::`.
- `vector<int> kaylesSG = {...}` – tabla hardcodeada con los Grundys de Kayles para $n = 0, \dots, 71$ (precalculada fuera de linea).
- Los valores en la tabla siguen la recurrencia SG de Kayles: cada n tiene un Grundy unico que codifica las estrategias ganadoras/perdedoras.

- `cout << "SG Kayles(7) = " << kaylesSG[7] << "\textbackslash n";` – consulta en $O(1)$: el Grundy de Kayles con 7 bolos es 3.

Para entender la teoria de la tabla: el Grundy $g(n)$ de Kayles se calcula como $g(n) = \text{mex}\{g(k) \oplus g(n-1-k), g(k) \oplus g(n-2-k)\}$ para todos los k validos; los valores en la tabla son la solucion de esta recurrencia (eventualmente periodica con periodo 12).

Codigo. Ver `cpp.tex`, seccion

3 Estructuras de datos

3.1 Ordenado y contenedores

3.1 Ordered Set (pbds)

Idea intuitiva. La **policy-based data structure** de GCC es un arbol rojo-negro (`rb_tree`) con dos operaciones extra: `find_by_order(k)` devuelve un iterador al k -esimo elemento (0-indexed) en $O(\log n)$, y `order_of_key(x)` cuenta cuantos elementos son **menores** que x en $O(\log n)$. Equivale a tener un **set** con “rango” y “rango por posicion” gratis. La estructura subyacente: cada nodo lleva un campo **prioridad** (aleatoria o fija) y el arbol mantiene la propiedad de heap por prioridad; eso da las estadisticas de orden en $O(\log n)$.

Propiedades clave (invariantes).

- Solo en GCC/clang (header `ext/pb_ds/...`); **no** en MSVC.
- `less_equal<int>` permite multiset; ojo con `erase` que ahi usa `upper_bound`.
- Tamano maximo $\sim 10^7$ elementos (limite de la libreria).
- `find_by_order(k)` con $k \geq$ tamano devuelve `s.end()`.

Codigo clave. Declaracion y uso tipico:

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update> s;
s.insert(5);
cout << s.order_of_key(3) << "\n";      // 0
cout << *s.find_by_order(0) << "\n";    // 5
```

Complejidad.

- $O(\log n)$ por operacion.
- Memoria: $\sim 5x$ un set normal (cada nodo guarda prioridad).

Donde tocar para modificar.

- **Multiset:** cambiar `less<int>` por `less_equal<int>` y borrar con `s.erase(s.upper_bound(x))`.
- **Estadisticas de orden:** `find_by_order(k)` da el k -esimo, util para mantener el k -esimo menor dinamicamente.
- **Acceso por rango:** anadir un wrapper que combine `lower_bound` + `order_of_key`.
- **Tipo custom:** anadir hash si quieres una `unordered_map` con order statistics (no funciona directo).

Trampas y errores frecuentes.

- `find_by_order(n)` (con n fuera de rango) es UB.
- No olvidar `#include <ext/pb_ds/assoc_container.hpp>` y `<ext/pb_ds/tree_policy.hpp>`.
- El namespace `__gnu_pbds` es feo; “using” acelera pero contamina el scope global.
- Algunos jueces online (Codeforces, AtCoder) lo soportan; otros no.

Explicacion linea por linea. Includes y typedef:

- `#include <ext/pb_ds/assoc_container.hpp>` – header de containers asociativos de pbds.
- `#include <ext/pb_ds/trie_policy.hpp>` – header con la politica de order statistics.
- `using namespace __gnu_pbds;` – namespace de la extension GNU.
- `typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update> pbds;`
– define el tipo con orden por estadística.
- Los parametros del template: tipo de clave, tipo mapeado (none), comparador, tipo de arbol (rb_tree), y la politica de orden.
- `// find_by_order, order_of_key` – las dos operaciones extra del pbds.
- `// greater<int>, less_equal<int>` – alternativas al comparador (multiset).
- Operaciones disponibles: `insert`, `erase`, `find_by_order(k)`, `order_of_key(x)`, todas $O(\log n)$.
- `find_by_order(k)` devuelve el k-esimo elemento (0-indexed).
- `order_of_key(x)` cuenta cuantos elementos son menores que x.
- `upper_bound` y `lower_bound` funcionan al revés que en `std::set`.
- Para borrar en multiset usar `s.erase(s.upper_bound(x))`.

Codigo. Ver `cpp.tex`, seccion *Ordered Set (pbds)*.

3.2 Basis (XOR basis)

Idea intuitiva. Una **base de XOR** mantiene un conjunto de mascaras (numeros) tal que cualquier XOR de un subconjunto del input se puede expresar como XOR de un subconjunto de la base. La idea es similar a la base de un espacio vectorial sobre $\mathbb{F}_2$. Construir: insertar cada mascara m recorriendo los bits de mayor a menor; si el bit i de m esta “libre” en la base, se aniaade; sino, se hace $m \oplus basis[i]$ y se continua. La estructura formal: el conjunto de XORs posibles es un subespacio vectorial sobre $\mathbb{F}_2$, y la base lo describe con $\leq \log_2(\text{max value})$ elementos.

Propiedades clave (invariantes).

- Tamano maximo de la base: ≤ 64 (bits) o $\leq \log_2(\text{max value})$.
- Base **minimal**: la construccion garantiza que cada vector tiene un unico bit alto distinto.
- El subespacio generado es invariante.
- Numero de elementos en la base = dimension del subespacio generado.

Codigo clave. Algoritmo de insercion (Gaussian elimination sobre $\mathbb{F}_2$):

```
void insert(long long x) {
    for (int i = MAXB; i >= 0; --i) {
        if (!(x & (1LL << i))) continue;
        if (!basis[i]) { basis[i] = x; return; }
        x ^= basis[i];
    }
    // x es combinacion lineal de los basis actuales
}
```

Complejidad.

- Insertar una mascara: $O(\text{bits})$. Construir la base completa: $O(N \cdot \text{bits})$.
- Query (maximo XOR): $O(\text{bits})$ si la base esta ordenada.

Donde tocar para modificar.

- **Obtener el maximo XOR de un subconjunto:** ordenar la base por bit alto descendente y aplicar XOR si aumenta.
- **Obtener el minimo XOR positivo:** el minimo distinto de 0 es el minimo `basis[i]` no cero.
- **Cuantos subconjuntos dan XOR k :** con la base, reducir k y contar grados de libertad.
- **XOR basis en 32 bits:** cambiar `int d = 64` a `int d = 32`.
- **K-esimo XOR:** ordenar la base y contar las combinaciones $\leq k$ (aplicacion clasica).

Trampas y errores frecuentes.

- Si los numeros son `long long`, usar `1LL << i`.
- Si se quiere la base **reducible** (que no necesariamente tenga vectores con unico bit alto), usar otra variante.
- Insertar en orden descendente de bits asegura la propiedad “unico bit alto”.
- Si dos mascarar tienen el mismo bit alto, una se descarta; la informacion se preserva en el XOR.

Explicacion linea por linea. Estructura Basis:

- `struct Basis{\ int d=64;\}` – XOR basis con $d = 64$ bits (long long).
- `vector<int> basis;` – `basis[i]` = elemento con bit alto en la posicion i .
- `int sz=0;` – contador de elementos no cero de la base.
- `Basis() basis.resize(d);` – inicializa todos los slots en 0.
- `void insertVector(int mask)...` – inserta una mascara en la base.
- `for(int i=d-1 ; i>=0 ; i--) if( (mask & (1LL<<i)) == 0 ) continue;` – itera de mayor a menor bit.
- `if( !basis[i] ) basis[i]=mask; ++sz; return; |||` – si el slot esta libre, aniade.
- `mask ^= basis[i];` – si no, cancela el bit i y continua.

Programa principal:

- `vector<int> nums = 1, 2, 3, 5;` – numeros a procesar.
- `Basis b; for(int x : nums) b.insertVector(x);` – construye la base.
- `int ans = 0;` – respuesta.
- `for(int i=b.d-1 ; i>=0 ; i--) ans = max(ans, ans ^ b.basis[i]);` – greedy: aplica XOR si aumenta.
- `// ans = maximo xor de un subconjunto de nums` – comentario: la respuesta es el maximo XOR alcanzable.

Codigo. Ver `cpp.tex`, seccion *Basis (XOR basis)*.

3.3 Trie

Idea intuitiva. Un **trie** (o prefix tree) almacena strings donde cada nodo tiene hijos por cada caracter posible (tipicamente 26 letras). Permite insercion, busqueda exacta y busqueda por prefijo en $O(|word|)$. La estructura formal: cada nodo representa un prefijo; las aristas son los caracteres; el flag `isEnd` indica si ese prefijo completo es una palabra valida. La ventaja sobre un set de strings: encontrar todos los strings con un prefijo dado en tiempo proporcional al prefijo + output.

Propiedades clave (invariantes).

- Cada nodo tiene `links[26]` (punteros a hijos) y un flag `isEnd`.
- Memoria: $O(\text{total chars})$; cada nodo cuesta ~ 26 punteros (puede comprimirse con `map<char, Node*>` si el alfabeto es disperso).
- Las búsquedas descienden siguiendo caracteres; si algún enlace es nulo, falla.
- El camino de la raíz a un nodo es exactamente el prefijo que el nodo representa.

Código clave. Inserción y búsqueda:

```
void insert(const string& s) {
    Node* v = root;
    for (char c : s) {
        int k = c - 'a';
        if (!v->links[k]) v->links[k] = new Node();
        v = v->links[k];
    }
    v->isEnd = true;
}

bool search(const string& s) {
    Node* v = root;
    for (char c : s) {
        int k = c - 'a';
        if (!v->links[k]) return false;
        v = v->links[k];
    }
    return v->isEnd;
}
```

Complejidad.

- Inserción y búsqueda: $O(|s|)$ donde $|s|$ es el largo del string.
- Memoria total: $O(\sum |s_i|)$ nodos para N strings.

Donde tocar para modificar.

- **Alfabeto mas grande:** cambiar `links[26]` por `links[256]` y eliminar `ch - 'a'`.
- **Trie de bits:** para problemas con mascarar, usar `links[2]` y trabajar con bits en vez de caracteres.
- **Trie persistente:** añadir `vector<Node*> versions;` y clonar nodos en cada inserción (como el persistent segtree).
- **Delete:** implementar recursión que borre nodos sin hijos; cuidado con nodos compartidos por varias palabras.
- **Prefijos con peso:** añadir `sumWeight` en cada nodo para queries de suma por prefijo.

Trampas y errores frecuentes.

- En este snippet los nodos se crean con `new` y nunca se liberan; en contests esta bien, pero en producción añadir destructor.
- El alfabeto tiene que estar acotado; sino, el trie explota en memoria.
- Si los strings son de tamaño $\leq 10^6$, el trie tendrá nodos por cada carácter, no solo por palabra.
- `search(s)` vs `startsWith(s)`: el primero requiere `isEnd`, el segundo no.

Explicacion linea por linea. Estructura Node y clase Trie:

- `struct Node{\ Node *links[26]; bool flag=false;\}` – nodo con 26 punteros a hijos y flag de fin de palabra.
- `bool containsKey(char ch) return links[ch-'a'] != NULL;` – verifica si existe el enlace.
- `void put(char ch, Node* node) links[ch-'a'] = node;` – crea el enlace al hijo.
- `Node* get(char ch) return links[ch-'a'];` – obtiene el hijo.
- `void setEnd() flag=true;` – marca el nodo como fin de palabra.
- `bool isEnd() return flag;` – verifica si es fin de palabra.
- `class Trie{ private: Node *root; public: Trie(){ root = new Node(); } –` clase Trie con raiz.
- `void insert(string word)...` – inserta una palabra.
- `Node* node = root;` – empieza en la raiz.
- `if(!node->containsKey(word[i])) node->put(word[i], new Node());` – crea hijo si no existe.
- `node = node->get(word[i]);` – avanza al siguiente nodo.
- `node->setEnd();` – al final, marca como fin de palabra.
- `bool search(string word)...` – busqueda exacta.
- `if(!node->containsKey(word[i])) return false;` – si no existe el enlace, falla.
- `return node->isEnd();` – al final, debe estar marcado como fin.
- `bool startsWith(string prefix)...` – busqueda por prefijo.
- `return true;` – al final del prefijo, retorna true (no requiere `isEnd`).

Programa principal:

- `Trie trie;` – instancia.
- `trie.insert("hello"); trie.insert("world");` – inserta dos palabras.
- `cout << trie.search("hello") << endl;` – imprime 1 (existe exacto).
- `cout << trie.search("hell") << endl;` – imprime 0 (no es palabra completa).
- `cout << trie.startsWith("hell") << endl;` – imprime 1 (prefijo existe).
- `cout << trie.startsWith("wor") << endl;` – imprime 1 (prefijo existe).

Codigo. Ver `cpp.tex`, seccion *Trie*.

3.2 Union-Find

3.1 DSU classico

Idea intuitiva. El **Disjoint Set Union** (Union-Find) mantiene una particion de elementos en conjuntos disjuntos. Las dos operaciones basicas son `find(x)` (devuelve el representante del conjunto de x) y `union(x, y)` (fusiona los conjuntos). La optimizacion **path compression** aplanar el arbol durante `find`, y **union by size/rank** mantiene el arbol balanceado. Combinadas, dan costo **amortizado** $O(\alpha(n))$ por operacion, casi $O(1)$. La cota de Ackermann aparece porque las alturas de los arboles son **extremadamente pequenas** (del orden de $\alpha(n)$ que para $n < 10^{600}$ es ≤ 5).

Propiedades clave (invariantes).

- `parent[i]` apunta al padre en el arbol; la raiz es `parent[i] == i`.
- `size[i]` solo es valido en la raiz.
- **Representante unico:** cada conjunto tiene exactamente una raiz.
- Union by size garantiza que la altura del arbol es $O(\log n)$ peor caso (antes de path compression).

Codigo clave. DSU con union by size y path compression:

```
int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); }
void unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return;
    if (size[a] < size[b]) swap(a, b);
    parent[b] = a;
    size[a] += size[b];
}
bool connected(int a, int b) { return find(a) == find(b); }
```

Complejidad.

- $O(\alpha(n))$ amortizado, donde α es la inversa de Ackermann.
- En la practica: ~ 4 operaciones por llamada para $n \leq 10^6$.

Donde tocar para modificar.

- **Aniadir informacion al conjunto** (ej. tamano del conjunto ya esta; suma de los elementos; etc.): aniadir campos al struct y mantenerlos en **union**.
- **DSU para borrar**: borrar el ultimo elemento es trivial, borrar en medio requiere reasignar o rehacer.
- **Weighted DSU**: para resolver ecuaciones tipo $x \equiv a \pmod n$ con restricciones, mantener el “peso” relativo al padre.
- **DSU on next**: aniadir `next[i]` para saltar elementos borrados (proximo no en el mismo set).

Trampas y errores frecuentes.

- `find` recursivo puede overflow del stack para n muy grande; cambiar a iterativo si pasa.
- Olvidar inicializar `size[i] = 1`.
- Si `size[a] == size[b]`, aniadir tie-break (e.g., por id) para determinismo.
- Confundir union por size con union por rank; en size, se compara el tamano del set, no la altura.

Explicacion linea por linea. Clase DisjointSet:

- `class DisjointSet\{\}` – encapsula el DSU.
- `vector<int> parent, size;\}` – arreglos de padre y tamano del set.
- `DisjointSet(int n)...` – constructor: tamano $n+1$ (1-based o para incluir indice 0).
- `for(int i=0 ; i<=n ; i++) parent[i]=i; size[i]=1;` – cada nodo inicia como su propio set de tamano 1.
- `int findUPar(int node)...` – encuentra el representante (raiz).
- `if(node == parent[node]) return node;` – caso base: si es su propio padre, es la raiz.
- `return parent[node]=findUPar(parent[node]);` – recursion con path compression.
- `void UnionBySize(int u, int v)...` – une los sets de u y v .
- `int ulp_u=findUPar(u); int ulp_v=findUPar(v);` – encuentra las raices.
- `if(ulp_u == ulp_v) return;` – ya estan en el mismo set.
- `if(size[ulp_u] < size[ulp_v])...` – union by size: el pequeno se cuelga del grande.
- `parent[ulp_u]=ulp_v; size[ulp_v]+=size[ulp_u];` – actualiza puntero y tamano.
- `else parent[ulp_v]=ulp_u; size[ulp_u]+=size[ulp_v];` – caso simetrico.

Codigo. Ver `cpp.tex`, seccion *DSU classico*.

3.2 DSU con rollback

Idea intuitiva. Variante del DSU donde cada **unite se puede deshacer** en orden LIFO. La idea: en vez de path compression (que es irreversible), solo se usa union by size. Cada **unite** guarda un par (**parent_changed**, **old_size**) en una pila. **rollback()** desapila hasta el ultimo “snapshot” (marcador) restaurando los valores. La razon: path compression modifica la estructura de multiples nodos, no solo uno, asi que es dificil de deshacer; union by size solo modifica dos nodos (el padre del menor y su tamano), asi que es trivial deshacer.

Propiedades clave (invariantes).

- **Sin path compression:** cada **find** es $O(\log n)$ (peor caso del union by size) en el peor caso, pero amortizado $O(\log n)$.
- **Snapshots:** **snapshot()** marca un punto; **rollback()** deshace hasta ahi.
- Cada **unite** exitoso apila **dos** pares (uno por cada cambio).
- La pila de historial crece monotonamente; **rollback** la reduce.

Codigo clave. El unite con guardado de historial:

```
void unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) { history.push({-1, -1}); return; }
    if (size[a] < size[b]) swap(a, b);
    history.push({b, size[a]});
    parent[b] = a;
    size[a] += size[b];
    components--;
}

void rollback() {
    auto [b, sz] = history.top(); history.pop();
    if (b == -1) return;
    size[parent[b]] = sz;
    parent[b] = b;
    components++;
}
```

Complejidad.

- $O(\log n)$ por **unite/find**, $O(1)$ por **rollback** de una operacion.
- Sin path compression: peor caso $O(\log n)$ por **find**.

Donde tocar para modificar.

- **Rollback parcial** (no todo): implementar snapshots parciales guardando el tamano del historial.
- **Mas informacion por conjunto:** anadir mas campos al struct y guardarlos en **history**.
- **Persistente:** anadir versionado completo (clonar el struct en cada cambio).
- **DSU de Arpa:** variante con $O(\log n)$ amortizado y **rollback** en $O(1)$.

Trampas y errores frecuentes.

- El snippet guarda dos pares por **unite**; al **rollback** **desapilar en orden correcto** (no invertir).
- Olvidar el **components--** en **unite** o el **components++** en **rollback** deja el contador mal.
- El sentinel `{-1, -1}` indica un unite trivial; importante para mantener consistencia.
- **find** puede ser $O(\log n)$ peor caso sin path compression; verificar que es OK para el problema.

Explicacion linea por linea. Estructura DSUrb:

- `struct DSUrb{}` – DSU con rollback.
- `vector<int> parent, sz;` – padre y tamaño.
- `vector<pair<int,int>> history;` – pila de operaciones: (nodo modificado, valor anterior).
- `int components;` – contador de componentes conexas.
- `DSUrb(int n)...` – constructor: n elementos, cada uno su propio set.
- `iota(parent.begin(), parent.end(), 0);` – llena `parent` con $0, 1, \dots, n-1$.
- `int find(int v) while(v != parent[v]) v = parent[v]; return v;` – find sin path compression (para rollback).
- `bool unite(int a, int b)...` – une a y b .
- `a = find(a); b = find(b);` – encuentra raíces.
- `if(a == b) return false;` – ya están unidos.
- `if(sz[a] > sz[b]) swap(a, b);` – asegura $sz[a] \leq sz[b]$.
- `history.push_back(a, sz[a]); history.push_back(b, sz[b]);` – guarda estado anterior.
- `parent[a] = b; sz[b] += sz[a]; components--;` – une y decrementa contador.
- `return true;` – indica que se unieron.
- `void snapshot() history.push_back(-1, -1);` – marca un punto de rollback.
- `void rollback()...` – deshace hasta el último snapshot.
- `while(history.back().first != -1)...` – procesa entradas hasta el marcador.
- `parent[v] = v; sz[w] = s0; components++;` – restaura los valores.

Programa principal:

- `DSUrb d(5);` – DSU con 5 elementos.
- `d.unite(0,1); d.unite(1,2);` – une 0,1,2.
- `cout << d.components << "\n";` – imprime 3 (quedan 0,1,2, 3, 4).
- `d.rollback();` – deshace las uniones.
- `cout << d.components << "\n";` – imprime 5 (estado original).

Codigo. Ver `cpp.tex`, seccion *DSU con rollback*.

3.3 Árboles de segmentos

3.1 Segment Tree Add + Sum

Idea intuitiva. Un **Segment Tree** es un árbol binario donde cada nodo cubre un rango del arreglo. La raíz cubre $[0, n-1]$; cada nodo interno $[l, r]$ se divide en dos hijos: $[l, m]$ y $[m+1, r]$ con $m = \lfloor (l+r)/2 \rfloor$. Para mantener la **suma** de un rango y soportar **suma en rango** eficientemente en $O(\log n)$, se utiliza **lazy propagation**: cuando un rango de actualización cubre por completo el segmento $[l, r]$ de un nodo, se acumula el valor en su campo **lazy** y se posterga la propagación a los hijos hasta que sea estrictamente necesario consultar o modificar dicho subárbol.

Propiedades clave (invariantes).

- `st[node].val`: suma acumulada del rango $[l, r]$ gestionado por el nodo.
- `st[node].lazy`: valor pendiente de propagar hacia los hijos del nodo.
- La función `push(node, l, r)`: aplica el **lazy** acumulado a `val` sumando $(r-l+1) \times \text{lazy}$, transfiere el valor pendiente a los hijos si no es hoja ($l \neq r$), y restablece **lazy** a 0.
- Si n se redondea a la siguiente potencia de 2, un arreglo de tamaño $2n$ es suficiente; para un n arbitrario sin redondear, se requieren hasta $4n$ nodos.

Codigo clave. Propagacion perezosa (lazy propagation) y actualizacion en rango:

```
void push(int node, int l, int r) {
    if (st[node].lazy == 0) return;
    st[node].val += (r - l + 1) * st[node].lazy;
    if (l != r) {
        st[node * 2].lazy += st[node].lazy;
        st[node * 2 + 1].lazy += st[node].lazy;
    }
    st[node].lazy = 0;
}

void add_range(int node, int l, int r, int ql, int qr, int val) {
    push(node, l, r);
    if (ql <= l && r <= qr) {
        st[node].lazy += val;
        push(node, l, r);
        return;
    }
    if (r < ql || qr < l) return;
    int mid = l + (r - l) / 2;
    add_range(node * 2, l, mid, ql, qr, val);
    add_range(node * 2 + 1, mid + 1, r, ql, qr, val);
    st[node].val = st[node * 2].val + st[node * 2 + 1].val;
}
```

Complejidad.

- Construcción: $O(n)$ en tiempo.
- Actualización y consulta: $O(\log n)$ en el peor caso (no amortizado).
- Memoria: $O(n)$ (vector de tamaño $2^{\lceil \log_2 n \rceil + 1} \leq 4n$).

Donde tocar para modificar.

- **Cambiar suma por min/max con add:** al combinar hijos usar min o max. **Atencion:** en push, el nodo se actualiza como `st[node].val += st[node].lazy`; (sin multiplicar por $(r - l + 1)$).
- **Asignacion en rango (set range):** el lazy debe ser el nuevo valor fijado y se requiere un valor centinela (o bandera booleana) para distinguir cuando no hay operacion pendiente (ya que 0 puede ser un valor a asignar).
- **Tamano a potencia de 2:** para redondear correctamente, iniciar en $n = \text{size}$ (`while(__builtin_popcount(n) != 1) n++;`) o usar `while(n < size) n <<= 1;`.
- **Soporte para 64 bits:** si los valores o la suma total superan $2 \cdot 10^9$, cambiar int por long long tanto en val como en lazy.

Trampas y errores frecuentes.

- **Condicion de hoja:** en push, verificar siempre `if (l != r)` antes de propagar a `node*2` y `node*2+1`; de lo contrario, las hojas accederan fuera de los limites del vector.
- **Llamar a push al inicio:** siempre llamar a `push(node, l, r)` al inicio tanto de `add_range` como de `query`, incluso antes de evaluar las condiciones de corte.
- **Multiplicar por la longitud:** en suma por rango, olvidar el factor $(r - l + 1)$ al aplicar el lazy al nodo produce sumas erroneas.
- **Overflow:** la suma de un rango entero de 10^5 elementos con valores hasta 10^9 excede con creces el rango de `int32`; usar `long long`.

Explicacion linea por linea. Estructura SegmentTree:

- `struct Node { int val = 0, lazy = 0; };` – nodo con la suma del rango y valor pendiente de suma.
- `SegmentTree(int size)` – ajusta n a una potencia de 2 mayor o igual a `size` y reserva $2n$ nodos.
- `void push(int node, int l, int r)` – si hay lazy acumulado, actualiza `val`, transfiere a los hijos si $l \neq r$, y limpia lazy.
- `st[node].val += (r-l+1) * st[node].lazy;` – aplica el valor pendiente sumandolo por cada elemento del rango.
- `if(l != r)` – asegura no propagar fuera de rango si el nodo actual es una hoja.
- `st[node*2].lazy += st[node].lazy;` – suma el delta al hijo izquierdo.
- `st[node*2+1].lazy += st[node].lazy;` – suma el delta al hijo derecho.
- `st[node].lazy = 0;` – marca como propagado el nodo actual.
- `void add_range(int node, int l, int r, int ql, int qr, int val)` – actualizacion recursiva en rango $[ql, qr]$.
- `if(ql <= l && r <= qr)` – si el rango del nodo esta totalmente contenido en la consulta, actualiza su lazy y aplica push.
- `if(r < ql || qr < l) return;` – si el nodo es totalmente disjunto del rango de consulta, retorna sin cambios.
- `st[node].val = st[node*2].val + st[node*2+1].val;` – combina los valores actualizados de ambos hijos.
- `int query(int node, int l, int r, int ql, int qr)` – consulta de suma en rango; hace push y suma recursivamente los hijos relevantes.
- `void add(int l, int r, int val)` – interfaz publica para sumar en rango sobre la raiz $([0, n-1])$.
- `int sum(int l, int r)` – interfaz publica para consultar suma en rango.

Programa principal:

- `SegmentTree st(5);` – crea el Segment Tree para 5 elementos (redondeado a $n = 8$).
- `st.add(0, 4, 1);` – suma 1 al rango $[0, 4]$ (arreglo: $[1, 1, 1, 1, 1]$).
- `st.add(1, 3, 2);` – suma 2 al rango $[1, 3]$ (arreglo: $[1, 3, 3, 3, 1]$).
- `cout << st.sum(0, 4) << endl;` – consulta la suma total ($1 + 3 + 3 + 3 + 1 = 11$).
- `st.add(0, 4, -1);` – resta 1 al rango $[0, 4]$.

Codigo. Ver `cpp.tex`, seccion *Segment Tree Add + Sum*.

3.2 Segment Tree Lazy Assign + Sum

Idea intuitiva. Estructura analoga al Segment Tree con suma, pero la operacion en rango es **asignacion** (fijar todos los elementos del intervalo $[ql, qr]$ a un mismo valor v). La asignacion sobrescribe por completo el estado previo del rango: si existian actualizaciones pendientes en ese nodo, quedan invalidadas por el nuevo valor base. Para propagar de forma perezosa en $O(\log n)$, se mantiene un valor centinela en **lazy** (o una bandera de control) que indica cuando un nodo no tiene asignaciones pendientes.

Propiedades clave (invariantes).

- `st[node].val`: suma total de los elementos en el rango $[l, r]$.
- `st[node].lazy`: valor a asignar a todos los elementos del subarbol. Un valor centinela (`LLONG_MAX` o `INF`) denota que no hay asignacion pendiente.
- La funcion `push(node, l, r)`: si hay asignacion pendiente, actualiza la suma del nodo con $(r-l+1) \times \text{lazy}$, transfiere la asignacion a los hijos si no es hoja ($l \neq r$), y restablece el **lazy** al centinela.
- Un arreglo de tamano $2n$ es suficiente si n se redondea a potencia de 2; se requieren hasta $4n$ nodos para un n arbitrario sin redondear.

Codigo clave. Propagacion perezosa y asignacion en rango:

```
void push(int node, int l, int r) {
    if (st[node].lazy == LLONG_MAX) return;
    st[node].val = (r - l + 1) * st[node].lazy;
    if (l != r) {
        st[node * 2].lazy = st[node].lazy;
        st[node * 2 + 1].lazy = st[node].lazy;
    }
    st[node].lazy = LLONG_MAX;
}

void assign_range(int node, int l, int r, int ql, int qr, long long val) {
    push(node, l, r);
    if (ql <= l && r <= qr) {
        st[node].lazy = val;
        push(node, l, r);
        return;
    }
    if (r < ql || qr < l) return;
    int mid = l + (r - l) / 2;
    assign_range(node * 2, l, mid, ql, qr, val);
    assign_range(node * 2 + 1, mid + 1, r, ql, qr, val);
    st[node].val = st[node * 2].val + st[node * 2 + 1].val;
}
```

Complejidad.

- Construcción: $O(n)$ en tiempo.
- Actualización y consulta: $O(\log n)$ en el peor caso (no amortizado).
- Memoria: $O(n)$ (vector de tamaño $2^{\lceil \log_2 n \rceil + 1} \leq 4n$).

Donde tocar para modificar.

- **Cambiar suma por min/max con assign:** al combinar hijos tomar min o max. En push, el nuevo valor es simplemente `st[node].val = st[node].lazy;` (sin multiplicar por la longitud).
- **Mezclar assign y add:** requiere dos campos `lazy_set` y `lazy_add`. Al recibir un `set`, se borra el `lazy_add`; al recibir un `add`, se suma sobre el `lazy_set` pendiente si existe, o se acumula en `lazy_add`.
- **Tipo de datos y centinela:** si los valores a asignar son enteros de 64 bits o pueden tomar valores arbitrarios, definir los campos como `long long` y verificar que el centinela no colisione con entradas legítimas del problema (o usar un `bool has_lazy`).
- **Tamaño a potencia de 2:** iniciar en $n = \text{size}$ con `n = size; while(__builtin_popcount(n) != 1) n++;` o `while(n < size) n <<= 1;`.

Trampas y errores frecuentes.

- **Colision o desbordamiento del centinela:** usar `LLONG_MAX` cuando los campos son de tipo `int` causa desbordamiento y conversión implícita. Usar `INT_MAX` para enteros o migrar los tipos a `long long`.
- **Condición de hoja en push:** verificar siempre `if (l != r)` antes de propagar a `node*2` y `node*2+1` para evitar accesos fuera de rango.
- **Multiplicar por la longitud:** en suma de rango, olvidar el factor $(r - l + 1)$ al aplicar la asignación en push resulta en cálculos de suma inválidos.
- **Limpieza de lazy:** olvidar restablecer `st[node].lazy = LLONG_MAX;` tras propagar aplicara repetidamente la asignación en consultas posteriores.

Explicacion linea por linea. Estructura SegmentTree:

- `struct Node { long long val = 0, lazy = LLONG_MAX; };` – nodo con suma y lazy de asignacion inicializado en el centinela neutro.
- `SegmentTree(int size)` – ajusta n a una potencia de 2 y reserva $2n$ nodos.
- `void push(int node, int l, int r)` – si hay asignacion pendiente, actualiza el nodo multiplicando por $(r - l + 1)$, propaga a hijos si $l \neq r$, y limpia `lazy`.
- `if(l != r)` – evita acceder a hijos inexistentes en nodos hoja.
- `st[node*2].lazy = st[node].lazy;` – sobrescribe la asignacion pendiente en el hijo izquierdo.
- `st[node*2+1].lazy = st[node].lazy;` – sobrescribe la asignacion pendiente en el hijo derecho.
- `st[node].lazy = LLONG_MAX;` – restablece el estado sin asignacion pendiente en el nodo actual.
- `void assign_range(int node, int l, int r, int ql, int qr, long long val)` – asignacion en rango recursiva.
- `if(ql <= l && r <= qr)` – rango totalmente cubierto: fija el lazy y aplica push de inmediato.
- `if(r < ql — qr < l) return;` – rango disjunto: retorna sin modificar.
- `st[node].val = st[node*2].val + st[node*2+1].val;` – actualiza la suma combinando ambos hijos.
- `long long query(int node, int l, int r, int ql, int qr)` – consulta de suma en rango con push en cada nivel.
- `void assign(int l, int r, long long val)` – funcion publica para invocar asignacion en rango.
- `long long sum(int l, int r)` – funcion publica para invocar consulta de suma en rango.

Programa principal:

- `SegmentTree st(5);` – inicializa el arbol para 5 elementos.
- `st.assign(0, 4, 1);` – asigna 1 a todo el rango (arreglo: $[1, 1, 1, 1, 1]$).
- `st.assign(1, 3, 2);` – asigna 2 al rango $[1, 3]$ (arreglo: $[1, 2, 2, 2, 1]$).
- `cout << st.sum(0, 4) << endl;` – imprime suma total ($1 + 2 + 2 + 2 + 1 = 8$).
- `st.assign(0, 4, 5);` – asigna 5 a todos los elementos (suma: $5 \times 5 = 25$).

Codigo. Ver `cpp.tex`, seccion *Segment Tree Lazy Assign + Sum*.

3.3 Segment Tree Add + lower bound index

Idea intuitiva. Variante de Segment Tree (a menudo llamada *walk on segment tree*) que responde eficientemente: “¿cual es el primer indice $i \geq \text{bound}$ tal que $A[i] \geq v$?”. En lugar de almacenar la suma, cada nodo mantiene el **maximo** de su rango. Al realizar la consulta, se desciende recursivamente priorizando el hijo izquierdo; si el maximo del subarbol izquierdo no alcanza el valor v (o queda totalmente a la izquierda de **bound**), se descarta en $O(1)$ y se explora el hijo derecho. Esto permite encontrar el primer indice en tiempo $O(\log n)$ sin necesidad de hacer busqueda binaria externa (la cual tomaria $O(\log^2 n)$).

Propiedades clave (invariantes).

- `st[node].val`: valor maximo en el rango $[l, r]$ administrado por el nodo.
- `st[node].lazy`: incremento aditivo pendiente de propagar.
- Monotonicidad de decision: si el maximo de un subarbol es estrictamente menor que v , se garantiza que ningun elemento dentro de el cumple $A[i] \geq v$, permitiendo podar la busqueda en $O(1)$.
- A diferencia de la busqueda sobre sumas acumuladas (que exige elementos no negativos), el walk sobre maximos funciona con cualquier valor (positivo, negativo o cero) y admite tanto actualizaciones aditivas (**add**) como de asignacion (**set**).

Codigo clave. Búsqueda del primer índice $i \geq \text{bound}$ con valor $\geq v$ ($O(\log n)$):

```
int query(int node, int l, int r, int val, int bound) {
    push(node, l, r);
    if (r < bound || st[node].val < val) return -1;
    if (l == r) return l;
    int mid = l + (r - l) / 2;
    int left = query(node * 2, l, mid, val, bound);
    if (left != -1) return left;
    return query(node * 2 + 1, mid + 1, r, val, bound);
}
```

Complejidad.

- Actualización (`add_range`): $O(\log n)$ en el peor caso.
- Consulta (`lower_bound_index`): $O(\log n)$ en el peor caso.
- Memoria: $O(n)$ (vector de tamaño $2n$ con padding a potencia de 2, o $4n$ directo).

Donde tocar para modificar.

- **Ultimo indice (búsqueda hacia atras):** para encontrar el *ultimo* índice $i \leq \text{bound}$ con $A[i] \geq v$, invertir la prioridad: descartar si $l > \text{bound}$ o $\text{val} < v$, explorar primero el hijo derecho y si retorna -1 , explorar el izquierdo.
- **Lower bound sobre suma acumulada:** si se busca el primer índice donde la suma prefija alcance v , los elementos deben ser no negativos ($A[i] \geq 0$). Si $\text{st}[node * 2].\text{val} \geq v$, se desciende a la izquierda; si no, se resta el valor izquierdo ($v \leftarrow v - \text{st}[node * 2].\text{val}$) y se busca en la derecha.
- **Condicion estricta o inversa:** para buscar el primer elemento $\leq v$, almacenar el **minimo** del rango en lugar del maximo y podar cuando $\text{st}[node].\text{val} > v$.

Trampas y errores frecuentes.

- **Retorno de no coincidencia:** si ningun índice cumple la condicion en $[\text{bound}, n - 1]$, la funcion retorna -1 ; el codigo invocador debe contemplar este caso.
- **Olvidar push en query:** al descender en la consulta es imprescindible llamar a `push(node, l, r)` para que los maximos de los hijos reflejen los incrementos pendientes.
- **Criterio de poda con bound:** la condicion `r < bound` descarta rangos situados totalmente a la izquierda de la posicion de inicio. Escribir erroneamente `r > bound` o `l < bound` arruina la exploracion.
- **Actualizacion aditiva en maximo:** en `push`, al sumar un lazy aditivo al maximo del nodo, la actualizacion es `st[node].val += st[node].lazy`; (sin multiplicar por la longitud del rango).

Explicacion linea por linea. Estructura SegmentTree:

- `struct Node { int val = 0, lazy = 0; };` – almacena el maximo del intervalo y el incremento pendiente.
- `SegmentTree(int size)` – ajusta la dimension n a la menor potencia de 2 mayor o igual a `size`.
- `void push(int node, int l, int r)` – propaga el valor `lazy` sumandolo a `val` y transfiriendolo a los hijos si $l \neq r$.
- `st[node].val += st[node].lazy;` – en maximo por rango, el incremento se anade directamente sin multiplicar por $(r - l + 1)$.
- `void add_range(int node, int l, int r, int ql, int qr, int val)` – suma `val` a todos los elementos en $[ql, qr]$.
- `st[node].val = max(st[node*2].val, st[node*2+1].val);` – combina los hijos recalculando el maximo.
- `int query(int node, int l, int r, int val, int bound)` – busca recursivamente el primer índice $i \geq \text{bound}$ con $A[i] \geq \text{val}$.

- `if(r < bound — st[node].val > val) return -1;` — descarta el nodo si queda totalmente a la izquierda de `bound` o su maximo no llega a `val`.
- `if(l == r) return l;` — caso base: si es hoja y no fue podada, es el indice buscado.
- `int left = query(node*2, l, mid, val, bound);` — explora recursivamente la rama izquierda primero.
- `if(left != -1) return left;` — si se halló un indice valido en la izquierda, se retorna de inmediato por minimalidad.
- `return query(node*2+1, mid+1, r, val, bound);` — si no hubo solucion en la izquierda, busca en la rama derecha.
- `int lower_bound_index(int val, int bound)` — funcion publica que inicia la consulta desde la raiz (`[0, n - 1]`).

Codigo. Ver `cpp.tex`, seccion *Segment Tree Add + lower bound index*.

3.4 Segment Tree Lazy Add + Set + Sum

Idea intuitiva. La variante mas completa: combina **add** y **set** en el mismo nodo. Prioridad: el **set** “mata” cualquier **add** previo (porque asignar implica “este es el nuevo valor base”). El **add** se acumula sobre el **set**. El orden importa: **set** primero, **add** despues. La razon algebraica: si representamos el valor final como $\text{base} + \text{delta}$, entonces **set** redefine el **base** y **add** redefine el **delta**; ambos son necesarios para una representacion completa.

Propiedades clave (invariantes).

- Dos lazies: `lazy_set` (sobrescribe) y `lazy_add` (acumula).
- En **push**: primero aplicar **set** (que borra **add** en hijos), despues **add** (que acumula).
- Si `lazy_set` esta activo, ignora `lazy_add` previo.

Codigo clave. Push con dos operaciones:

```
void push(int node, int l, int r) {
    if (l == r) return;
    int m = (l + r) / 2;
    for (int c : {node*2, node*2+1}) {
        if (st[node].hasSet) {
            st[c].val = (c == node*2 ? m-l+1 : r-m) * st[node].lazySet;
            st[c].lazySet = st[node].lazySet;
            st[c].hasSet = true;
            st[c].lazyAdd = 0; // resetear
        }
        if (st[node].lazyAdd != 0) {
            st[c].val += (c == node*2 ? m-l+1 : r-m) * st[node].lazyAdd;
            st[c].lazyAdd += st[node].lazyAdd;
        }
    }
    st[node].hasSet = false; st[node].lazyAdd = 0;
}
```

Complejidad.

- $O(\log n)$.
- Memoria $O(4n)$ pero con campos extra por nodo.

Donde tocar para modificar.

- **Aniadir otro tipo de operacion:** aniadir un nuevo campo y su manejo en **push**.
- **Build desde un arreglo:** ya implementado en el constructor.
- **Chained operations:** aniadir **min**, **max** con sus semanticas.

Trampas y errores frecuentes.

- Olvidar resetear `lazy_add` cuando se aplica `set` a un hijo.
- Aplicar `add` antes de `set` da valores incorrectos.
- El orden de las operaciones en un update mixto debe estar bien definido.

Explicacion linea por linea. Estructura SegmentTree:

- `struct Node` – nodo con tres campos: valor, lazy de suma, lazy de asignacion.
- `int val = 0, lazy_add = 0;` – valor inicial y acumulador.
- `int lazy_set = LLONG_MAX;` – centinela: indica “sin asignacion pendiente”.
- `Node(int x=0)` – constructor del nodo con valor x.
- `val = x; lazy_add = 0; lazy_set = LLONG_MAX;` – inicializa los tres campos.
- `SegmentTree(const vector<int>& a)` – constructor que ajusta n a potencia de 2.
- `while(__builtin_popcount(n) != 1) n++;` – fuerza potencia de 2.
- `st.resize(2*n);` – redimensiona a $2n$ elementos.
- `st[n+i] = Node(a[i]);` – copia el arreglo a las hojas.
- `for(int i=n-1; i>=1; i--) st[i] = merge(st[2*i], st[2*i+1]);` – construye los nodos internos bottom-up.
- `Node merge(Node &left, Node &right)` – combina dos nodos hijos.
- `ans.val = left.val + right.val; return ans;` – suma los valores.
- `void push(int node, int l, int r)` – propaga lazies al nodo actual y a sus hijos.
- `if(st[node].lazy_set != LLONG_MAX){...}` – si hay set pendiente.
- `st[node].val = st[node].lazy_set * (r-l+1);` – aplica el set multiplicado por el tamano.
- `if(l != r){ st[node*2].lazy_set = st[node].lazy_set; st[node*2].lazy_add = 0; }` – propaga set, resetea add en hijos.
- `st[node].lazy_set = LLONG_MAX;` – limpia el set del nodo actual.
- `if(st[node].lazy_add != 0){...}` – si hay add pendiente.
- `st[node].val += st[node].lazy_add * (r-l+1);` – aplica add al nodo.
- `st[node*2].lazy_add += st[node].lazy_add;` – propaga add a hijos.
- `void update(int node, int l, int r, int ql, int qr, int val, int type)` – actualiza un rango.
- `push(node, l, r);` – primero aplica lazy pendiente.
- `if(ql <= l && r <= qr){...}` – caso cubierto totalmente.
- `if(type == 1) st[node].lazy_add += val; else st[node].lazy_set = val;` – elige add o set segun tipo.
- `if(r < ql || qr < l) return;` – caso fuera del rango.
- `int mid = l + (r-l)/2;` – punto medio.
- `update(node*2, l, mid, ql, qr, val, type);` – recursion al hijo izquierdo.
- `update(node*2+1, mid+1, r, ql, qr, val, type);` – recursion al hijo derecho.
- `st[node].val = st[node*2].val + st[node*2+1].val;` – recalcula valor del padre.
- `int query(int node, int l, int r, int ql, int qr)` – query en un rango.
- `if(ql <= l && r <= qr) return st[node].val;` – caso contenido, retorna suma.
- `if(r < ql || qr < l) return 0;` – caso fuera del rango.
- `return query(node*2, l, mid, ql, qr) + query(node*2+1, mid+1, r, ql, qr);` – suma de ambos hijos.
- `void add_range(int l, int r, int val){ update(1, 0, n-1, l, r, val, 1); }` – helper publico para add.

- `void set_range(int l, int r, int val){ update(1, 0, n-1, l, r, val, 2); }` – helper publico para set.
- `int sum(int l, int r){ return query(1, 0, n-1, l, r); }` – helper publico para query.

Codigo. Ver `cpp.tex`, seccion *Segment Tree Lazy Add + Set + Sum*.

3.5 Segment Tree iterativo

Idea intuitiva. Variante no recursiva del segment tree: usa un arreglo de tamaño $2n$ (donde n es potencia de 2). Las hojas estan en $[n, 2n)$; los nodos internos en $[1, n)$. El build llena las hojas y luego construye hacia arriba. Updates: ir a la hoja y propagar hacia arriba. Queries: usar el patron de “dos punteros” que suben/expanden segun la paridad. La ventaja: $\sim 2x$ mas rapido en la practica por el menor overhead de recursion.

Propiedades clave (invariantes).

- `t[i]` para $i \geq n$ son hojas (valores originales).
- `t[i]` para $i < n$ son nodos internos (sumas de sus hijos).
- Update: ir a hoja, modificar, subir hasta la raiz.
- Query: expansion binaria de los rangos cubiertos.

Codigo clave. El loop de query iterativo:

```
long long query(int l, int r) {
    long long res = 0;
    for (l += n, r += n+1; l < r; l >>= 1, r >>= 1) {
        if (l & 1) res += t[l++];
        if (r & 1) res += t[--r];
    }
    return res;
}
```

Complejidad.

- Build $O(n)$, update y query $O(\log n)$.
- Memoria $O(2n)$.

Donde tocar para modificar.

- **Otra operacion:** cambiar el ‘+’ por la operacion (max, min, xor).
- **Lazy propagation:** anadir un arreglo `lz` y aplicar en updates/queries.
- **Resize dinamico:** anadir capacidad de crecer si llega un update fuera de rango.

Ejemplo. Para `a = {1,2,3,4,5}`: `query(1,3) = 2+3+4 = 9`. Tras `set(2, 10)`: `query(1,3) = 2+10+4 = 16`.

Trampas y errores frecuentes.

- El loop usa `r += n+1` (no `r += n`) porque `query` es inclusivo y queremos que el bucle salga correctamente.
- Si n no es potencia de 2, ajustar el tamaño a la potencia siguiente.
- El `build` copia el arreglo sin verificar tamaño; anadir bounds check.

Explicacion linea por linea. Estructura SegTreeIter:

- `struct SegTreeIter\{ int n;\}` – tamaño (potencia de 2).
- `vector<long long> t;` – arreglo interno de tamaño $2n$.
- `SegTreeIter(int n) this->n = n; t.assign(2*n, 0);` – inicializa con ceros.
- `void build(const vector<long long>& a)...` – construye desde un arreglo.
- `for(int i=0;i<(int)a.size();i++) t[n+i] = a[i];` – copia el arreglo a las hojas.
- `for(int i=n-1;i>=1;i--) t[i] = t[2*i] + t[2*i+1];` – construye hacia arriba.
- `void set(int p, long long val)...` – point update.
- `p += n; t[p] = val;` – va a la hoja y asigna.
- `for(p >>= 1; p >= 1; p >>= 1) t[p] = t[2*p] + t[2*p+1];` – recalcula hacia arriba.
- `long long query(int l, int r)...` – query por rango.
- `long long res = 0;` – acumulador.
- `for(l += n, r += n+1; l < r; l >>= 1, r >>= 1)...` – loop principal.
- `if(l & 1) res += t[l++];` – si l es hijo derecho, anadir y avanzar.
- `if(r & 1) res += t[--r];` – si r es hijo derecho (vistio desde atras), anadir y retroceder.
- `return res;` – retorna el resultado.

Programa principal:

- `SegTreeIter st(5);` – crea segtree para 5 elementos.
- `st.build(1, 2, 3, 4, 5);` – construye.
- `cout << st.query(1, 3) << "\n";` – imprime 9.
- `st.set(2, 10);` – cambia $a[2]$ a 10.
- `cout << st.query(1, 3) << "\n";` – imprime 16.

Codigo. Ver `cpp.tex`, seccion *Segment Tree iterativo*.

3.6 Segment Tree 2D

Idea intuitiva. Extension del Segment Tree a dos dimensiones (arbol de arboles de segmentos): el arbol exterior particiona las filas (coordenada x), y cada nodo del arbol exterior contiene un Segment Tree completo que particiona las columnas (coordenada y). La raiz cubre $[0, n - 1] \times [0, m - 1]$. Para actualizar o consultar, se desciende recursivamente primero en x y luego en y , logrando operaciones de punto/subrectangulo en $O(\log n \cdot \log m)$.

Propiedades clave (invariantes).

- `tree[vx][vy]`: suma del subrectangulo $[lx, rx] \times [ly, ry]$ representado por el nodo exterior vx y el nodo interior vy .
- `updateX`: desciende recursivamente en la dimension x hacia la fila x . En cada nivel de x , invoca `updateY` para actualizar la columna y en ese nodo exterior.
- `updateY`: desciende en la dimension y . En las hojas de y ($ly = ry$), si $lx = rx$ asigna el nuevo valor; si $lx \neq rx$, combina los hijos exteriores: `tree[vx*2][vy] + tree[vx*2+1][vy]`.
- `queryX` y `queryY`: descomponen el rectangulo de consulta $[x1, x2] \times [y1, y2]$ en $O(\log n \cdot \log m)$ nodos canonicos.
- Memoria: requiere $4n \times 4m = 16nm$ enteros.

Codigo clave. Actualizacion puntual 2D (arbol de arboles):

```
void updateY(int vx, int lx, int rx, int vy, int ly, int ry, int x, int y, int val) {
    if (ly == ry) {
        if (lx == rx) tree[vx][vy] = val;
        else tree[vx][vy] = tree[vx * 2][vy] + tree[vx * 2 + 1][vy];
    } else {
        int my = (ly + ry) / 2;
        if (y <= my) updateY(vx, lx, rx, vy * 2, ly, my, x, y, val);
        else updateY(vx, lx, rx, vy * 2 + 1, my + 1, ry, x, y, val);
        tree[vx][vy] = tree[vx][vy * 2] + tree[vx][vy * 2 + 1];
    }
}

void updateX(int vx, int lx, int rx, int x, int y, int val) {
    if (lx != rx) {
        int mx = (lx + rx) / 2;
        if (x <= mx) updateX(vx * 2, lx, mx, x, y, val);
        else updateX(vx * 2 + 1, mx + 1, rx, x, y, val);
    }
    updateY(vx, lx, rx, 1, 0, m - 1, x, y, val);
}
```

Complejidad.

- Actualizacion puntual: $O(\log n \cdot \log m)$.
- Consulta en subrectangulo: $O(\log n \cdot \log m)$.
- Memoria: $O(n \cdot m)$ con factor constante 16 ($4n \times 4m$ enteros).

Donde tocar para modificar.

- **Cambiar operacion:** para maximo o minimo, cambiar la suma por **max** o **min** tanto en la combinacion de x como en la de y .
- **Alternativa mas ligera (2D BIT / Fenwick):** si la operacion es suma o XOR y no se requiere lazy, un Fenwick 2D usa solo $O(nm)$ de memoria con la misma complejidad temporal y constante mucho menor.
- **Segment Tree 2D dinamico / disperso:** si la grilla es de hasta $10^9 \times 10^9$ pero hay pocas actualizaciones, se reemplazan los vectores por punteros o indices dinamicos.

Trampas y errores frecuentes.

- **Omitir la recursion en x (updateX):** llamar a **updateY** unicamente desde la raiz de x deja desactualizados los hijos de x y produce resultados en cero. Es indispensable descender primero en x con **updateX** y llamar a **updateY** en cada paso.
- **Consumo excesivo de memoria:** la constante 16 hace que para $N = M = 2000$ se requieran $\approx 16 \times 4 \cdot 10^6 \times 4 \text{ B} \approx 256 \text{ MB}$, al borde del limite de memoria.
- **Indices 0-based vs 1-based:** asegurar que las consultas y actualizaciones respeten los limites $[0, n - 1]$ y $[0, m - 1]$.

Explicacion linea por linea.

- `struct SegTree2D { int n, m; vector<vector<int>> tree; };` – dimensiones y matriz de nodos.
- `tree.assign(4*n, vector<int>(4*m, 0));` – reserva espacio para los $4n \times 4m$ nodos 2D.
- `void updateY(...)` – desciende en la dimension y para el nodo exterior vx .
- `if(ly == ry)` – caso base de y (hoja de columnas).
- `if(lx == rx) tree[vx][vy] = val;` – hoja de filas y hoja de columnas: celda puntual (x, y) .

- `else tree[vx][vy] = tree[vx*2][vy] + tree[vx*2+1][vy];` – nodo interno de filas: combina los dos hijos de x .
- `tree[vx][vy] = tree[vx][vy*2] + tree[vx][vy*2+1];` – combina los dos hijos de y .
- `void updateX(int vx, int lx, int rx, int x, int y, int val)` – desciende en la dimension x .
- `if(lx != rx)` – si no es hoja en x , avanza hacia el hijo que contiene a x .
- `updateY(vx, lx, rx, 1, 0, m-1, x, y, val);` – actualiza el arbol de y para el nodo vx actual.
- `void update(int x, int y, int val) { updateX(1, 0, n-1, x, y, val); }` – punto de entrada para `update`.
- `int queryY(int vx, int vy, int ly, int ry, int y1, int y2)` – consulta el rango $[y1, y2]$ en el nodo vx .
- `if(y1 > ry — y2 | ly)` return 0; — rango disjunto en y .
- `if(y1 <= ly && ry <= y2)` return `tree[vx][vy];` – intervalo totalmente contenido en y .
- `int queryX(int vx, int lx, int rx, int x1, int x2, int y1, int y2)` – consulta el rango $[x1, x2]$ en x .
- `if(x1 <= lx && rx <= x2)` return `queryY(vx, 1, 0, m-1, y1, y2);` – intervalo cubierto en x : consulta el arbol de y .
- `int query(int x1, int y1, int x2, int y2)` – punto de entrada publico para consulta en subrectangulo.

Codigo. Ver `cpp.tex`, seccion *Segment Tree 2D*.

3.7 Segment Tree persistente

Idea intuitiva. Variante donde cada `update` crea una **nueva raiz** en vez de modificar el arbol existente. La estructura es funcional: cada nodo tiene hijos apuntando a versiones anteriores (que no se modifican). Asi, queries sobre versiones antiguas siguen funcionando. La eficiencia: cada `update` crea $O(\log n)$ nodos nuevos (solo el camino afectado). Es util para “consultar el estado del arreglo tras el k -esimo `update`” o problemas de “offline con rollback”.

Propiedades clave (invariantes).

- Cada `update` retorna un nuevo puntero a raiz; el arbol anterior permanece intacto.
- Solo se crean $O(\log n)$ nodos por `update` (el camino afectado).
- Memoria total: $O((N + Q) \log N)$ nodos.
- Las versiones forman un arbol (cada version es un nodo en el “version tree”).

Codigo clave. La recursion de `update` (que crea nuevos nodos):

```
PNode* pUpdate(PNode* node, int l, int r, int pos, long long val) {
    if (l == r) return new PNode(val);
    int m = (l + r) / 2;
    if (pos <= m) return new PNode(0, pUpdate(node->l, l, m, pos, val), node->r);
    else return new PNode(0, node->l, pUpdate(node->r, m+1, r, pos, val));
}
```

Complejidad.

- Update: $O(\log n)$ (creacion de nodos).
- Query: $O(\log n)$.
- Memoria: $O((n + q) \log n)$.

Donde tocar para modificar.

- **Aniadir lazy:** aniadir un campo `lazy` y propagar en `pupdate`.
- **Otra operacion:** cambiar la suma por `max`, `min`, `xor`, etc.
- **Rollback explicito:** aniadir un stack de punteros a raiz por version.
- **Offline queries con tiempo:** guardar punteros a raiz para cada query.

Ejemplo. Tras `a = {1,2,3,4,5}`, hacer `set(2, 10)` crea una nueva version v_2 . La version original (`root`) sigue dando suma 15; la nueva (`v2`) da 22.

Trampas y errores frecuentes.

- Nunca modificar nodos existentes: cada update crea uno nuevo.
- Si hay muchas updates, la memoria crece; considerar limpieza con un `unordered_map` para reusar nodos.
- El destructor debe liberar toda la memoria (recorrer el arbol recursivamente).
- En el constructor de nodo, los hijos se asignan por defecto a `nullptr`.

Explicacion linea por linea. Estructura PNode:

- `struct PNode{\ long long val; PNode *l, *r;}` – nodo con valor y dos hijos.
- `PNode(long long v = 0, PNode* l = nullptr, PNode* r = nullptr): val(v), l(l), r(r)\{\}` – constructor con defaults.
- `PNode* pBuild(int l, int r, const vector<long long>\& a)\{\...\}` – construye el arbol inicial.
- `if(l == r) return new PNode(a[l]);` – caso base: hoja con el valor del arreglo.
- `int m = (l + r) / 2;` – punto medio.
- `return new PNode(0, pBuild(l, m, a), pBuild(m+1, r, a));` – nodo interno con hijos recursivos.
- `PNode* pUpdate(PNode* node, int l, int r, int pos, long long val)\{\...\}` – update que crea nueva version.
- `if(l == r) return new PNode(val);` – caso base: nueva hoja con el valor.
- `if(pos <= m) return new PNode(0, pUpdate(node->l, l, m, pos, val), node->r);` – actualiza hijo izquierdo.
- `else return new PNode(0, node->l, pUpdate(node->r, m+1, r, pos, val));` – actualiza hijo derecho.
- `long long pQuery(PNode* node, int l, int r, int ql, int qr)\{\...\}` – query en una version.
- `if(!node or ql > r or qr < l) return 0;` – caso vacio o fuera de rango.
- `if(ql <= l \&\& r <= qr) return node->val;` – caso contenido.
- `return pQuery(node->l, l, m, ql, qr) + pQuery(node->r, m+1, r, ql, qr);` – recursion.

Programa principal:

- `vector<long long> a = \{1, 2, 3, 4, 5\};` – arreglo inicial.
- `PNode* root = pBuild(0, 4, a);` – construye version inicial.
- `PNode* v2 = pUpdate(root, 0, 4, 2, 10);` – crea version con `a[2] = 10`.
- `cout << pQuery(root, 0, 4, 0, 4) << "\textbackslash n";` – suma version 1: 15.
- `cout << pQuery(v2, 0, 4, 0, 4) << "\textbackslash n";` – suma version 2: 22.

Codigo. Ver `cpp.tex`, seccion *Segment Tree persistente*.

3.4 Otras

3.1 MO

Idea intuitiva. Algoritmo de Mo “puro” (sin updates): queries offline ordenadas por $(L/\sqrt{n}, R)$ (con R alternando entre ascendente y descendente segun el bloque). Mover L o R entre queries consecutivas es $O(1)$; el total es $O((N + Q)\sqrt{n})$. La intuicion: al reordenar las queries, los punteros L y R se mueven “lo menos posible”; cada elemento del array se visita $O(\sqrt{n} \cdot Q/N)$ veces.

Propiedades clave (invariantes).

- Q guarda (l, r, idx) .
- El sort prioriza bloque de L ; dentro del bloque, alterna R ascendente/descendente para mejorar la locality.
- Tres punteros l, r que se mueven paso a paso.
- Tamano de bloque optimo: $\sim N/\sqrt{Q}$.

Codigo clave. El sort clasico de Mo con alternancia:

```
int block = (int)sqrt(n);
sort(queries.begin(), queries.end(), [&](const Q& a, const Q& b) {
    int blockA = a.l / block, blockB = b.l / block;
    if (blockA != blockB) return blockA < blockB;
    return (blockA & 1) ? (a.r > b.r) : (a.r < b.r);
});
```

Complejidad.

- $O((N + Q)\sqrt{n})$.
- Con Hilbert order, $\sim 30\%$ mas rapido en practica.

Donde tocar para modificar.

- **Hilbert order:** alternativa al sort clasico; da mejor locality y es mas rapido en practica.
- **Aniadir la operacion concreta:** el snippet tiene `// add` y `// delete` como comentarios; reemplazarlos con la logica del problema (contar distintos, suma de frecuencias, etc.).
- **Bloque optimo:** calcular `int block = max(1, (int)(n / sqrt(q)))`;

Trampas y errores frecuentes.

- Los indices en el sort son **0-based** (`q[i].l = l - 1`), consistente con el resto del codigo.
- `add` y `remove` deben ser **inversos exactos**; sino, las queries dan respuestas incorrectas.
- Para queries cerradas $[l, r]$, Mo suele usar $[l, r + 1)$ (half-open); aniadir o restar 1 consistentemente.

Explicacion linea por linea. Estructura y programa principal:

- `struct Q{ int l,r,idx; };` – cada query guarda limites e indice original.
- `int main() int n,m; cin >> n >> m;` – lee tamano y numero de queries.
- `vector<int> a(n); for(int i=0 ; i<n ; i++) cin >> a[i];` – lee el arreglo.
- `vector<Q> q(m);` – almacena queries.
- `int l,r; cin >> l >> r; q[i].l = l-1; q[i].r = r-1; q[i].idx = i;` – convierte a 0-based y guarda.
- `int rootN = sqrtl(n);` – tamano de bloque.
- `sort(q.begin(), q.end(), [&](Q x, Q y)...);` – orden por bloque de L , alternando R .
- `int bx = x.l / rootN, by = y.l / rootN;` – calcula bloques.
- `return (bx & 1) ? (x.r > y.r) : (x.r < y.r);` – alterna direccion de R por bloque.

- `vector<int> freq(1000001, 0);` – frecuencias para contar distintos.
- `long long curr = 0;` – respuesta actual.
- `auto add = [&](int pos) int x = a[pos]; if(freq[x] == 0) curr++; freq[x]++; ;` – añade elemento.
- `auto del = [&](int pos) int x = a[pos]; freq[x]--; if(freq[x] == 0) curr--; ;` – quita elemento.
- `int l=0, r=-1;` – punteros iniciales (rango vacío).
- `while(r < qr) add(++r);` – expande R a la derecha.
- `while(r > qr) del(r--);` – contrae R.
- `while(l < ql) del(l++);` – contrae L desde la izquierda.
- `while(l > ql) add(--l);` – expande L hacia la izquierda.
- `ans[q[i].idx] = curr;` – guarda respuesta en el índice original.

Código. Ver `cpp.tex`, sección *MO*.

3.5 Fenwick / BIT

3.1 BIT point update + prefix sum

Idea intuitiva. El **Fenwick Tree** (BIT) almacena sums parciales de manera que `sum(i)` (suma de $[1, i]$) y `add(i, delta)` cuestan $O(\log n)$. La representación es ingeniosa: el nodo i cubre el rango $[i - \text{lowbit}(i) + 1, i]$. Así, `sum(i)` itera $i -= \text{lowbit}(i)$, sumando los nodos que cubren un prefijo creciente, y `add(i, delta)` itera $i += \text{lowbit}(i)$, propagando el cambio a todos los nodos cuyo rango contiene a i . La idea: cada índice i se descompone en una suma de bloques disjuntos de tamaño potencias de 2 (su “lowbit representation”).

Propiedades clave (invariantes).

- Índice **1-based** (no usar $i = 0$).
- $\text{lowbit}(i) = i \& (-i)$.
- Cada nodo cubre exactamente un rango de tamaño `lowbit(i)`.
- La operación debe ser **invertible** (suma, xor, no min/max).

Código clave. Las dos operaciones básicas:

```
void add(int i, long long d) {
    for (; i <= n; i += i & -i) bit[i] += d;
}
long long sum(int i) {
    long long s = 0;
    for (; i > 0; i -= i & -i) s += bit[i];
    return s;
}
```

Complejidad.

- $O(\log n)$ por operación. Memoria $O(n)$.
- Constante muy pequeña (~ 1): en la práctica, $\sim 3x$ más rápido que segment tree.

Donde tocar para modificar.

- **Suma -¿ max/min/xor:** cambiar el += por la operacion (con identidad apropiada). **No** funciona para min de forma natural (no es invertible).
- **Tamano dinamico:** anadir un metodo `resize(newSize)` que preserve valores.
- **Tamano grande:** con $n = 10^7$, mejor comprimir coordenadas.
- **Operador no conmutativo:** usar segment tree en su lugar.

Trampas y errores frecuentes.

- Indices 0 vs 1: el BIT es **1-based**. Si el problema da indices 0-based, anadir +1 en cada llamada.
- El metodo `sum` itera `i -= i & -i`; anadir o quitar 1 rompe el invariante.
- Si el tipo es `int`, posibles overflows; usar `long long`.
- Para rango $[l, r]$, el query es `sum(r) - sum(l-1)` (cuidado con $l = 1$).

Explicacion linea por linea. Estructura del BIT:

- `struct BIT { int n; vector<long long> t; };` – encapsula el Fenwick Tree.
- `BIT(int n): n(n), t(n+1, 0) {}` – constructor: tamaño $n + 1$ para indexacion 1-based.
- `void add(int i, long long delta)` – agrega `delta` en la posicion `i`.
- `for(; i <= n; i += i & -i) t[i] += delta;` – avanza por los ancestros de `i` (`lowbit`), sumando en cada uno.
- `long long sum(int i)` – calcula la suma de `a[1]` a `a[i]`.
- `for(; i > 0; i -= i & -i) r += t[i];` – desciende removiendo el bit menos significativo.
- `long long rangeSum(int l, int r)` – retorna la suma en $[l, r]$ mediante `sum(r) - sum(l - 1)`.

Programa principal:

- `BIT bit(10);` – crea BIT de tamaño 10.
- `bit.add(3, 5);` – suma 5 en la posicion 3.
- `bit.add(7, 2);` – suma 2 en la posicion 7.
- `cout << bit.sum(5) << "\n";` – imprime suma de $[1, 5]$ (que es 5).
- `cout << bit.rangeSum(1, 10) << "\n";` – imprime suma total (7).

Codigo. Ver `cpp.tex`, seccion *BIT point update + prefix sum*.

3.2 BIT range add + point query

Idea intuitiva. Truco de **diferencias**: para sumar d a $[l, r]$, anadir d en l y $-d$ en $r + 1$. Asi, el valor en i es la suma de los prefijos hasta i , lo que se calcula con un BIT normal. La idea algebraica: si mantenemos un arreglo de diferencias Δ donde $a_i = \sum_{j \leq i} \Delta_j$, entonces anadir d a $a[l..r]$ equivale a $\Delta_l += d$ y $\Delta_{r+1} -= d$, y reconstruir a es un prefix sum.

Propiedades clave (invariantes).

- Internamente es un BIT 5.1 con la convencion de diferencias.
- `rangeAdd(l, r, delta)` cuesta $O(\log n)$ (dos `add`).
- `pointQuery(i)` cuesta $O(\log n)$.
- Si $r + 1 > n$, no se anade el $-d$ (fuera del rango).

Codigo clave. El truco de diferencias:

```
void rangeAdd(int l, int r, long long d) {
    add(l, d);
    if (r + 1 <= n) add(r + 1, -d);
}
long long pointQuery(int i) { return sum(i); }
```

Complejidad.

- $O(\log n)$ por operacion.
- Memoria $O(n)$.

Donde tocar para modificar.

- **Persistencia:** aniadir versionado (cada update crea una nueva “copia” del estado; en BIT no es trivial, mejor usar persistent segtree).
- **Coordenada compression:** si n es grande pero los updates son pocos, comprimir.
- **2D range add + point query:** aniadir una dimension con la misma tecnica.

Trampas y errores frecuentes.

- Si $r + 1 > n$, **no** aniadir **-delta** (no hay efecto fuera del rango); el snippet ya lo chequea.
- Para reconstruir el array original tras k range adds, hacer k point queries (no hay shortcut).
- Confundir “suma de diferencias” con “suma acumulada”; son cosas distintas.

Explicacion linea por linea. Estructura BITRangeAdd:

- `struct BITRangeAdd { int n; vector<long long> t; };` – encapsula el Fenwick Tree para range add y point query.
- `BITRangeAdd(int n): n(n), t(n+1, 0) {}` – inicializa el vector con $n + 1$ ceros (1-based).
- `void add(int i, long long delta)` – suma delta a la posicion i .
- `for(; i <= n; i += i & -i) t[i] += delta;` – propaga la diferencia por los lowbits.
- `void rangeAdd(int l, int r, long long delta)` – suma delta al rango $[l, r]$.
- `add(l, delta);` – inicia el efecto sumando en la posicion l .
- `if(r + 1 <= n) add(r + 1, -delta);` – finaliza el efecto restando en $r + 1$.
- `long long pointQuery(int i)` – consulta el valor actual en la celda i .
- `for(; i > 0; i -= i & -i) r += t[i];` – acumula las diferencias prefijas hasta i .

Programa principal:

- `BITRangeAdd bit(10);` – crea BIT para 10 elementos.
- `bit.rangeAdd(2, 5, 3);` – suma 3 al rango $[2, 5]$.
- `cout << bit.pointQuery(1) << "\n";` – antes del rango: 0.
- `cout << bit.pointQuery(3) << "\n";` – dentro del rango: 3.
- `cout << bit.pointQuery(6) << "\n";` – despues del rango: 0.

Codigo. Ver `cpp.tex`, seccion *BIT range add + point query*.

3.3 BIT range add + range sum

Idea intuitiva. Permite realizar actualizaciones de suma en rango y consultas de suma en rango en $O(\log n)$ usando dos arboles de Fenwick (BIT), evitando la sobrecarga de un Segment Tree con lazy propagation. La idea matematica: sea $D[k] = A[k] - A[k-1]$ el arreglo de diferencias (de modo que $A[k] = \sum_{j=1}^k D[j]$). La suma prefija $\sum_{k=1}^x A[k]$ se puede reescribir intercambiando el orden de sumatoria:

$$\sum_{k=1}^x A[k] = \sum_{k=1}^x \sum_{j=1}^k D[j] = \sum_{j=1}^x (x-j+1)D[j] = x \sum_{j=1}^x D[j] - \sum_{j=1}^x (j-1)D[j]$$

Se utilizan dos BITs: t_1 mantiene las diferencias $D[j]$, y t_2 mantiene los productos $D[j] \cdot (j-1)$.

Propiedades clave (invariantes).

- Al sumar d en $[l, r]$:
 - En t_1 : $+d$ en l , y $-d$ en $r+1$.
 - En t_2 : $+d \cdot (l-1)$ en l , y $-d \cdot r$ en $r+1$ (puesto que el indice es $r+1$ y $(r+1)-1 = r$).
- La suma acumulada en $[1, x]$ se evalua directamente como:

$$\text{prefixSum}(x) = x \cdot t_1.\text{sum}(x) - t_2.\text{sum}(x)$$

- La suma en cualquier rango $[l, r]$ se obtiene por diferencia de prefijos: $\text{prefixSum}(r) - \text{prefixSum}(l-1)$.
- Indexacion 1-based: los arboles de Fenwick requieren indices estrictamente positivos ($1 \leq i \leq n$).

Codigo clave. Suma en rango y consulta en rango con dos Fenwick trees:

```
void rangeAdd(int l, int r, long long delta) {
    add(t1, l, delta);
    add(t1, r + 1, -delta);
    add(t2, l, delta * (l - 1));
    add(t2, r + 1, -delta * r);
}

long long prefixSum(int i) {
    return sum(t1, i) * i - sum(t2, i);
}

long long rangeSum(int l, int r) {
    return prefixSum(r) - prefixSum(l - 1);
}
```

Complejidad.

- Actualizacion en rango: $O(\log n)$ (realiza 4 llamadas a **add**).
- Consulta en rango: $O(\log n)$ (realiza 4 llamadas a **sum**).
- Memoria: $O(n)$ ($2n$ enteros de 64 bits en dos vectores continuos, consumo mucho menor que Segment Tree).

Donde tocar para modificar.

- **2D range add + range sum**: se expande la misma logica de sumatorias dobles a 4 BITs en $O(\log n \cdot \log m)$ por operacion.
- **Aritmetica modular**: aplicar $\% \text{MOD}$ y sumas positivas con $+ \text{MOD}$ en cada multiplicacion y acumulacion.
- **Punto a punto**: para consultar el valor actual de un solo elemento $A[i]$, basta con consultar $t_1.\text{sum}(i)$.

Trampas y errores frecuentes.

- **Desbordamiento de 32 bits:** los productos $\delta \times (l - 1)$ y $\delta \times r$ pueden superar con facilidad $2 \cdot 10^9$ (por ejemplo, para $n = 10^5$ y $\delta = 10^5$, el producto es 10^{10}). Ambos arreglos t_1 y t_2 deben ser de tipo `long long`.
- **Indices con base cero:** intentar consultar o actualizar en el índice 0 provoca un bucle infinito en `i -= i & -i` o `i += i & -i`. Siempre usar indexación 1-based o sumar +1 internamente.
- **Límite superior $r + 1$:** la cancelación se aplica en $r + 1$, por lo que los vectores deben reservarse con al menos $n + 2$ elementos (o $n + 1$ si $r \leq n$ estricto).

Explicación línea por línea. Estructura BIT2:

- `struct BIT2 { int n; vector<long long> t1, t2; };` – encapsula dos árboles de Fenwick para `range add` y `range sum`.
- `BIT2(int n): n(n), t1(n+1, 0), t2(n+1, 0) {}` – inicializa ambos arreglos con tamaño $n + 1$ en cero.
- `void add(vector<long long>& t, int i, long long delta)` – suma `delta` a la posición i en el BIT especificado.
- `for(; i <= n; i += i & -i) t[i] += delta;` – propaga la actualización por potencias de 2 hacia arriba (`lowbit`).
- `long long sum(const vector<long long>& t, int i)` – calcula la suma prefija $[1, i]$ en el BIT `t`.
- `for(; i > 0; i -= i & -i) r += t[i];` – desciende eliminando el bit menos significativo.
- `void rangeAdd(int l, int r, long long delta)` – añade `delta` al rango $[l, r]$.
- `add(t1, l, delta); add(t1, r + 1, -delta);` – actualiza las diferencias de primer orden en `t1`.
- `add(t2, l, delta * (1 - 1)); add(t2, r + 1, -delta * r);` – actualiza las correcciones lineales en `t2`.
- `long long prefixSum(int i)` – calcula la suma acumulada $[1, i]$ usando la fórmula $i \cdot t_1 - t_2$.
- `long long rangeSum(int l, int r)` – retorna la suma del intervalo $[l, r]$ restando prefijos.

Programa principal:

- `BIT2 bit(10);` – inicializa la estructura para 10 elementos (índices 1 a 10).
- `bit.rangeAdd(2, 5, 3);` – suma 3 a las posiciones de la 2 a la 5.
- `cout << bit.rangeSum(1, 10) << "\n";` – imprime 12 (3×4 elementos modificados).
- `cout << bit.rangeSum(3, 5) << "\n";` – imprime 9 (3×3 elementos en el rango).

Código. Ver `cpp.tex`, sección *BIT range add + range sum*.

3.6 RMQ y sparse table

3.1 Sparse Table

Idea intuitiva. Estructura para consultas estáticas de rango (Range Minimum Query, RMQ) en $O(1)$ tras un precomputo de $O(n \log n)$. La idea fundamental es precalcular las respuestas para todos los subintervalos cuya longitud es una potencia de dos (2^k). La entrada `st[k][i]` representa la respuesta en el intervalo $[i, i + 2^k - 1]$. Dado un rango de consulta $[l, r]$ de longitud $\text{len} = r - l + 1$, se elige $k = \lfloor \log_2(\text{len}) \rfloor$ de modo que dos bloques de longitud 2^k (uno que comienza en l y otro que termina en r) cubren completamente el intervalo $[l, r]$. Esta combinación directa funciona en $O(1)$ **exclusivamente para operaciones idempotentes** ($x \circ x = x$), como `min`, `max`, `gcd` y operaciones a nivel de bits (`AND/OR`), donde el solapamiento de los dos bloques no altera el resultado.

Propiedades clave (invariantes).

- Caso base: `st[0][i] = a[i]` (intervalos de longitud $2^0 = 1$).
- Relación de recurrencia: `st[k][i] = min(st[k-1][i], st[k-1][i + (1 << (k-1))])`.
- Número de niveles: `LOG = floor(log2 n) + 1` (en GCC: `32 - __builtin_clz(n)`).
- La operación debe ser asociativa e **idempotente** ($f(x, x) = x$).
- Estructura estática: no soporta actualizaciones eficientes (requeriría $O(n)$ por modificación).

Codigo clave. Construccion y consulta $O(1)$ en Sparse Table:

```
struct SparseTable {
    int n, LOG;
    vector<vector<int>>> st;
    vector<int> lg;
    SparseTable(const vector<int>& a) {
        n = a.size();
        LOG = 32 - __builtin_clz(n);
        st.assign(LOG, vector<int>(n));
        st[0] = a;
        for (int k = 1; k < LOG; ++k) {
            for (int i = 0; i + (1 << k) <= n; ++i) {
                st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; ++i) lg[i] = lg[i / 2] + 1;
    }
    int query(int l, int r) {
        int k = lg[r - l + 1]; // o: 31 - __builtin_clz(r - l + 1)
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

Complejidad.

- Construccion: $O(n \log n)$ en tiempo.
- Consulta: $O(1)$ en tiempo.
- Memoria: $O(n \log n)$ enteros.

Donde tocar para modificar.

- **Cambiar operacion:** reemplazar min por max, std::gcd, o bits (&, |) tanto en la construccion como en la consulta.
- **Operaciones no idempotentes (suma, multiplicacion, XOR):** descomponer el rango $[l, r]$ en potencias de 2 disjuntas en $O(\log n)$, o utilizar una **Disjoint Sparse Table** para mantener consultas en $O(1)$.
- **Optimizacion de logaritmo:** la tabla lg precalculada evita la llamada a funciones bitwise; en C++20 tambien se puede usar std::bit_width o std::__lg(len).
- **Localidad espacial de cache:** indexar st[k][i] en lugar de st[i][k] garantiza acceso contiguo en memoria durante los bucles de construccion y reduce notablemente los fallos de cache.

Trampas y errores frecuentes.

- **Rango inclusivo:** la funcion espera $[l, r]$ inclusivo ($0 \leq l \leq r < n$). El inicio del segundo bloque es $r - (1 \ll k) + 1$.
- **Intentar usar para suma:** solapar dos intervalos en una suma contara los elementos centrales dos veces. Para sumas estaticas, usar sumas prefijas ordinarias ($O(1)$ tiempo y $O(n)$ memoria).
- **Off-by-one en limite de bucle:** la condicion $i + (1 \ll k) \leq n$ asegura que el segundo sub-bloque no desborde el tamano n del vector.

Explicacion linea por linea. Estructura SparseTable:

- int n, LOG; – dimension del arreglo y cantidad de niveles de potencias de 2.
- vector<vector<int>>> st; – tabla donde st[k][i] cubre $[i, i + 2^k - 1]$.

- `vector<int> lg;` – tabla precalculada de logaritmos base 2 para consulta en $O(1)$.
- `LOG = 32 - __builtin_clz(n);` – calcula $\lfloor \log_2 n \rfloor + 1$ niveles necesarios.
- `st.assign(LOG, vector<int>(n)); st[0] = a;` – inicializa el nivel base con los elementos de entrada.
- `for(int k=1; k<LOG; k++)` – genera sucesivamente los niveles doblando la longitud de los bloques.
- `st[k][i] = min(st[k-1][i], st[k-1][i + (1<<(k-1))]);` – combina los dos sub-bloques adyacentes de tamaño 2^{k-1} .
- `lg[i] = lg[i/2] + 1;` – calcula $\lfloor \log_2 i \rfloor$ mediante programación dinámica lineal.
- `int k = lg[r - l + 1];` – mayor potencia de dos tal que $2^k \leq r - l + 1$.
- `return min(st[k][l], st[k][r - (1<<k) + 1]);` – combina el bloque que inicia en l y el bloque que termina en r .

Programa principal:

- `SparseTable st({3, 1, 4, 1, 5, 9, 2, 6});` – construye la Sparse Table para el arreglo dado.
- `cout << st.query(0, 4) << "\n";` – consulta el mínimo en $[0, 4]$ (elementos $\{3, 1, 4, 1, 5\}$, mínimo es 1).
- `cout << st.query(2, 7) << "\n";` – consulta el mínimo en $[2, 7]$ (elementos $\{4, 1, 5, 9, 2, 6\}$, mínimo es 1).

Código. Ver `cpp.tex`, sección *Sparse Table*.

3.2 Disjoint Sparse Table

Idea intuitiva. Estructura de datos estática que permite responder consultas sobre cualquier rango $[l, r]$ en tiempo $O(1)$ tras un precomputo de $O(n \log n)$, pero a diferencia de la Sparse Table clásica, **no requiere idempotencia** (funciona para cualquier operación asociativa: suma, multiplicación, XOR, matrices, etc.). La idea: en cada nivel k , el arreglo se divide en bloques de tamaño 2^{k+1} . En cada bloque con punto medio `mid`, se precomputa un sufijo hacia la izquierda desde `mid - 1` y un prefijo hacia la derecha desde `mid`. Para cualquier consulta $[l, r]$, si se toma $k = \lfloor \log_2(l \oplus r) \rfloor$ (el bit más significativo donde l y r difieren), l y r caen en el mismo bloque pero en lados opuestos de su punto medio. Así, el rango $[l, r]$ se descompone exactamente en dos subrangos disjuntos: $[l, \text{mid} - 1]$ y $[\text{mid}, r]$, sin solapamiento alguno.

Propiedades clave (invariantes).

- `st[k][i]`: en el nivel k , si $i < \text{mid}$ almacena la acumulación en el sufijo $[i, \text{mid} - 1]$; si $i \geq \text{mid}$, almacena la acumulación en el prefijo $[\text{mid}, i]$.
- Identificación de nivel en $O(1)$: la condición de pertenencia al mismo bloque y lados opuestos del punto medio equivale a que l y r difieran en el bit k , por lo que $k = 31 - \text{__builtin_clz}(l \oplus r)$.
- La operación debe ser **asociativa**; no requiere ser conmutativa ni idempotente (si no es conmutativa, se respeta el orden estricto: `op(st[k][l], st[k][r])`).

Código clave. Construcción y consulta $O(1)$ con Disjoint Sparse Table:

```
struct DisjointSparseTable {
    int n, LOG;
    vector<vector<int>> st;
    DisjointSparseTable(const vector<int>& a) {
        n = a.size();
        LOG = (n <= 1 ? 1 : 32 - __builtin_clz(n - 1) + 1);
        st.assign(LOG, a);
        for (int k = 1; k < LOG; ++k) {
            int half = 1 << k;
            for (int mid = half; mid < n; mid += 2 * half) {
                for (int i = mid - 2; i >= mid - half; --i)
                    st[k][i] = min(a[i], st[k][i + 1]);
                int lim = min(n, mid + half);
```

```

        for (int i = mid + 1; i < lim; ++i)
            st[k][i] = min(st[k][i - 1], a[i]);
    }
}
}
int query(int l, int r) {
    if (l == r) return st[0][l];
    int k = 31 - __builtin_clz(l ^ r);
    return min(st[k][l], st[k][r]);
}
};

```

Complejidad.

- Construcción: $O(n \log n)$ en tiempo.
- Consulta: $O(1)$ en el peor caso (una sola operación `op` y cálculo de `clz`).
- Memoria: $O(n \log n)$ enteros.

Donde tocar para modificar.

- **Cualquier operación asociativa:** sustituir `min` por suma (+), multiplicación modular, XOR (^) o multiplicación de matrices $O(M^3)$.
- **Operaciones no conmutativas:** asegurar el orden de los argumentos: `op(st[k][l], st[k][r])`.
- **Tabulación de logaritmos:** para evitar `__builtin_clz`, se puede precomputar un arreglo `lg` con $\lfloor \log_2 i \rfloor$.

Trampas y errores frecuentes.

- **Caso base $l = r$:** si $l = r$, entonces $l \oplus r = 0$, para el cual `__builtin_clz(0)` es comportamiento indefinido en C++. Es indispensable tratar $l = r$ por separado retornando `st[0][l]`.
- **Indices 0-based:** la propiedad del bit más significativo requiere que los índices sean 0-based ($0 \leq l \leq r < n$).
- **Límites en el último bloque:** en el bucle derecho, verificar que `lim` no supere n (`min(n, mid + half)`).

Explicación línea por línea. Estructura `DisjointSparseTable`:

- `struct DisjointSparseTable { int n, LOG; vector<vector<int>> st; };` – encapsula la estructura.
- `LOG = (n <= 1 ? 1 : 32 - __builtin_clz(n - 1) + 1);` – calcula $\lceil \log_2 n \rceil + 1$ niveles.
- `st.assign(LOG, a);` – inicializa cada nivel con los valores base del arreglo.
- `for(int k=1; k<LOG; ++k)` – itera sobre cada nivel con bloques de tamaño 2^{k+1} .
- `int half = 1 << k;` – la mitad del bloque actual; el punto de división es `mid`.
- `for(int mid = half; mid < n; mid += 2 * half)` – recorre los puntos medios de cada bloque.
- `st[k][i] = min(a[i], st[k][i + 1]);` – precalcula acumulaciones hacia la izquierda (sufijos).
- `st[k][i] = min(st[k][i - 1], a[i]);` – precalcula acumulaciones hacia la derecha (prefijos).
- `int query(int l, int r)` – resuelve la consulta en tiempo constante $O(1)$.
- `if(l == r) return st[0][l];` – evita $l \oplus r = 0$ y retorna el elemento único.
- `int k = 31 - __builtin_clz(l ^ r);` – nivel cuyo punto medio separa a l de r .
- `return min(st[k][l], st[k][r]);` – combina el sufijo $[l, \text{mid} - 1]$ y el prefijo $[\text{mid}, r]$.

Programa principal:

- `DisjointSparseTable dst({3, 1, 4, 1, 5, 9, 2, 6});` – construye la DST para el arreglo.
- `cout << dst.query(0, 4) << "\n";` – consulta el rango $[0, 4]$, resultado es 1.
- `cout << dst.query(2, 7) << "\n";` – consulta el rango $[2, 7]$, resultado es 1.

Codigo. Ver `cpp.tex`, seccion *Disjoint Sparse Table*.

3.7 Trees y DP en arbol

3.1 Binary Lifting / LCA

Idea intuitiva. El **Binary Lifting** (levantamiento binario) es una tecnica que precalcula para cada nodo v de un arbol enraizado sus ancestros a distancias exponenciales $2^0, 2^1, 2^2, \dots, 2^k$. Como cualquier entero positivo d se descompone de forma unica en una suma de potencias de 2 (su representacion binaria), se puede ascender d niveles en el arbol en tiempo $O(\log d) \leq O(\log n)$. Para encontrar el **LCA** (Lowest Common Ancestor, o ancestro comun mas bajo) de dos nodos u y v :

1. Si estan a distinta profundidad, se asciende el nodo mas profundo hasta que ambos queden en el mismo nivel.
2. Si tras igualar profundidades coinciden ($u = v$), dicho nodo es el LCA.
3. En caso contrario, se realizan saltos binarios en paralelo desde el bit mas significativo hacia abajo, subiendo ambos nodos siempre que sus ancestros sigan siendo distintos. Al terminar, quedan inmediatamente debajo del LCA, por lo que este es su padre directo (`up[0][u]`).

Propiedades clave (invariantes).

- `up[k][v]`: ancestro a distancia 2^k del nodo v . Recurrencia:

$$\text{up}[k][v] = \text{up}[k-1][\text{up}[k-1][v]]$$

- Numero de niveles: $\text{LOG} = \lfloor \log_2 n \rfloor + 1$ (en GCC: `32 - __builtin_clz(n + 1)`).
- Para la raiz se define `up[0][root] = -1` (o la raiz misma con profundidad 0).
- Distancia entre nodos en el arbol: $\text{dist}(u, v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{lca}(u, v)]$.

Codigo clave. Precalculo DFS y consulta de LCA en $O(\log n)$:

```
void dfs(int v, int p) {
    up[0][v] = p;
    for (int k = 1; k < LOG; ++k) {
        if (up[k-1][v] != -1)
            up[k][v] = up[k-1][up[k-1][v]];
    }
    for (int u : adj[v]) {
        if (u == p) continue;
        depth[u] = depth[v] + 1;
        dfs(u, v);
    }
}

int lca(int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    int diff = depth[a] - depth[b];
    for (int k = 0; k < LOG; ++k)
        if (diff & (1 << k)) a = up[k][a];
    if (a == b) return a;
    for (int k = LOG - 1; k >= 0; --k) {
        if (up[k][a] != up[k][b]) {
            a = up[k][a];
            b = up[k][b];
        }
    }
    return up[0][a];
}
```

Complejidad.

- Precomputo (DFS + tabla): $O(n \log n)$ en tiempo.
- Consulta de LCA o k -ésimo ancestro: $O(\log n)$ en el peor caso.
- Memoria: $O(n \log n)$ enteros para la tabla `up`.

Donde tocar para modificar.

- **K -ésimo ancestro:** para subir k niveles desde v , iterar sobre los bits de k : `if (k & (1 << j)) v = up[j][v];`.
- **Camino ponderado (max/min/suma de aristas):** añadir una tabla paralela `val[k][v]` que guarde la operación en el camino de longitud 2^k : `val[k][v] = op(val[k-1][v], val[k-1][up[k-1][v]])`.
- **Bosque de arboles:** si el grafo no es conexo, invocar `dfs(root, -1)` para cada componente conexa.

Trampas y errores frecuentes.

- **Omitir el chequeo $a == b$:** tras igualar profundidades, si $a = b$ (uno es ancestro del otro), se debe retornar a inmediatamente. Si se entra al segundo bucle, se intentará subir por encima del LCA.
- **Acceso fuera de rango en raíz:** al construir `up[k][v]`, si `up[k-1][v] == -1`, no acceder a `up[k-1][-1]`; verificar `if (up[k-1][v] != -1)`.
- **Condicion de la segunda fase:** la condicion debe ser `up[k][a] != up[k][b]` para saltar juntos solo mientras sigan siendo distintos, asegurando que convergen a los hijos directos del LCA.

Explicacion linea por linea. Estructura LCA:

- `struct LCA { int n, LOG; vector<vector<int>> up, adj; vector<int> depth; };` – encapsula la estructura de Binary Lifting.
- `LOG = 32 - __builtin_clz(n + 1);` – calcula $\approx \lceil \log_2 n \rceil + 1$, cantidad de niveles necesarios.
- `void addEdge(int u, int v)` – inserta una arista bidireccional en el arbol.
- `void dfs(int v, int p)` – recorre el arbol calculando profundidades y ancestros en potencias de 2.
- `up[0][v] = p;` – el ancestro a distancia $2^0 = 1$ es el padre directo en el arbol.
- `up[k][v] = up[k-1][up[k-1][v]];` – aplica la transitividad de saltos: $2^k = 2^{k-1} + 2^{k-1}$.
- `void build(int root)` – inicia el precalculo con DFS desde la raíz dada con padre -1 .
- `int lca(int a, int b)` – responde a la consulta del ancestro comun mas bajo en $O(\log n)$.
- `if(depth[a] < depth[b]) swap(a, b);` – asegura sin perdida de generalidad que a este a mayor o igual profundidad.
- `if(diff & (1<<k)) a = up[k][a];` – asciende a la diferencia de niveles para emparejar profundidades.
- `if(a == b) return a;` – si los nodos coinciden, b era ancestro de a y es el LCA.
- `if(up[k][a] != up[k][b]) { a = up[k][a]; b = up[k][b]; }` – ascienden simultaneamente mientras sigan en ramas distintas.
- `return up[0][a];` – al terminar, se encuentran justo debajo del LCA; se retorna su padre.

Programa principal:

- `LCA lca(5);` – inicializa el arbol para 5 nodos (0 a 4).
- `lca.addEdge(0, 1); ...` – inserta las aristas del arbol.
- `lca.build(0);` – construye las tablas asignando el nodo 0 como raíz.
- `cout << lca.lca(2, 4) << "\n";` – consulta el LCA entre los nodos 2 y 4 (resultado es 1).

Codigo. Ver `cpp.tex`, seccion *Binary Lifting / LCA*.

3.2 Heavy-Light Decomposition (HLD)

Idea intuitiva. Descomponer un arbol en **cadenas pesadas** (caminos maximales formados por aristas pesadas, donde cada nodo no hoja se conecta con su hijo de mayor tamaño de subarbol). Cada cadena se mapea a un rango contiguo en un arreglo base mediante un recorrido DFS modificado que visita primero a los hijos pesados. Así, consultas y actualizaciones sobre **caminos** $u \rightarrow v$ se descomponen en $O(\log n)$ intervalos contiguos resueltos con un Segment Tree o Fenwick Tree. La garantía teórica radica en que cualquier hijo liviano tiene un subarbol de tamaño a lo sumo la mitad ($\leq |T|/2$); por ende, cualquier camino hacia la raíz cruza a lo sumo $\log_2 n$ aristas livianas y un camino arbitrario entre dos nodos cruza a lo sumo $2\log_2 n$ cadenas pesadas.

Propiedades clave (invariantes).

- $\text{heavy}[v]$ = hijo pesado de v (el de mayor subarbol).
- $\text{head}[v]$ = cabeza de la cadena que contiene a v .
- $\text{pos}[v]$ = posición en el arreglo base.
- Cada nodo pertenece a exactamente una cadena.

Código clave. El query sobre un path $u \rightarrow v$:

```
long long pathQuery(int u, int v) {
    long long res = 0;
    while (head[u] != head[v]) {
        if (depth[head[u]] < depth[head[v]]) swap(u, v);
        res += seg.query(pos[head[u]], pos[u]);
        u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    res += seg.query(pos[u], pos[v]);
    return res;
}
```

Complejidad.

- Precomputo / Build: $O(n)$ en tiempo.
- Query / Update sobre camino ($u \rightarrow v$): $O(\log^2 n)$ usando Segment Tree o Fenwick Tree (un camino atraviesa a lo sumo $O(\log n)$ cadenas pesadas, y cada consulta sobre el rango continuo de una cadena toma $O(\log n)$).
- Query / Update sobre subarbol: $O(\log n)$ con Segment Tree o BIT (el subarbol completo corresponde a un único rango continuo en el arreglo base).
- Memoria: $O(n)$ para los vectores de la descomposición.

Donde tocar para modificar.

- **Subtree queries:** guardar el tamaño del subarbol $\text{sz}[v]$ durante el DFS. Como la descomposición numera primero todo el subarbol de cada nodo, el subarbol de v abarca el rango $[\text{pos}[v], \text{pos}[v] + \text{sz}[v] - 1]$.
- **Valores en aristas:** asociar cada arista al nodo más profundo. Para consultas sobre caminos, el LCA se omite modificando el tramo final a $[\text{pos}[u] + 1, \text{pos}[v]]$ (o ignorándolo si $u = v$).
- **Estructura externa:** conectar los rangos generados por `pathSegments` con un Segment Tree o BIT para ejecutar consultas y actualizaciones puntuales o en rangos.

Trampas y errores frecuentes.

- **Cadenas ligeras vs pesadas:** un camino entre u y v cruza a lo sumo $2\log_2 n$ cadenas; olvidar el bucle `while (head[u] != head[v])` rompe la descomposición.
- **Swap de nodos por profundidad de cabeza:** al ascender, siempre se debe mover el nodo cuya **cabeza de cadena** sea más profunda ($\text{depth}[\text{head}[u]] < \text{depth}[\text{head}[v]]$), NO simplemente $\text{depth}[u] < \text{depth}[v]$.

- **Ultimo segmento:** tras el bucle, u y v pertenecen a la misma cadena ($\text{head}[u] == \text{head}[v]$). Se debe asegurar que $\text{pos}[u] \leq \text{pos}[v]$ (usando $\text{if}(\text{depth}[u] > \text{depth}[v]) \text{swap}(u, v)$) y agregar el segmento final $[\text{pos}[u], \text{pos}[v]]$.

Explicacion linea por linea. Estructura HLD:

- `struct HLD { ... };` – estructura que encapsula Heavy-Light Decomposition.
- `int n, curPos;` – cantidad de nodos y contador para asignar indices contiguos en el arreglo base.
- `vector<int> parent, depth, heavy, head, pos;` – almacena padre, profundidad, hijo pesado, cabeza de cadena y posicion asignada en el arreglo base.
- `vector<vector<int>> adj;` – lista de adyacencia del arbol.
- `HLD(int n): n(n)` – constructor: reserva memoria e inicializa vectores con valores por defecto (padre -1 , hijo pesado -1).
- `void addEdge(int u, int v)` – inserta una arista bidireccional entre los nodos u y v .
- `int dfs(int v)` – calcula profundidades, padres y selecciona para cada nodo su hijo pesado (el de mayor tamaño de subarbol).
- `if(sub > maxSub) { maxSub = sub; heavy[v] = u; }` – identifica al hijo con el subarbol mas grande.
- `void decompose(int v, int h)` – numera los nodos en $\text{pos}[v]$ colocando cadenas continuas en memoria; visita primero al hijo pesado.
- `if(heavy[v] != -1) decompose(heavy[v], h);` – extiende la cadena actual con la misma cabeza h .
- `decompose(u, u);` – para los hijos ligeros, inicia una nueva cadena con ellos mismos como cabeza.
- `void build(int root = 0)` – ejecuta el DFS y la descomposicion desde la raiz elegida.
- `vector<pair<int,int>> pathSegments(int u, int v)` – descompone el camino $u \rightarrow v$ en $O(\log n)$ rangos contiguos en el arreglo base.
- `if(depth[head[u]] < depth[head[v]]) swap(u, v);` – garantiza subir la cadena cuya cabeza este a mayor profundidad.
- `segs.push_back({pos[head[u]], pos[u]}); u = parent[head[u]];` – anade el tramo de la cadena actual y salta al padre de la cabeza.
- `if(depth[u] > depth[v]) swap(u, v);` – dentro de la misma cadena, ordena los extremos por profundidad.
- `segs.push_back({pos[u], pos[v]});` – anade el ultimo tramo del camino (entre el ancestro comun y el otro nodo).

Programa principal:

- `HLD hld(5);` – instancia la descomposicion para 5 nodos.
- `hld.addEdge(0, 1); ...` – construye la topologia del arbol.
- `hld.build(0);` – ejecuta el precalculo fijando el nodo 0 como raiz.
- `auto segs = hld.pathSegments(2, 4);` – genera los segmentos contiguos para consultar el camino entre 2 y 4.

Codigo. Ver `cpp.tex`, seccion *Heavy-Light Decomposition (HLD)*.

3.3 Centroid Decomposition

Idea intuitiva. Descomponer recursivamente un arbol mediante sus **centroides**. Un centroeide es un vertice cuya eliminacion divide al arbol en componentes conexas, cada una con un tamaño de a lo sumo $\lfloor n/2 \rfloor$ nodos. En cualquier arbol siempre existe al menos un centroeide (y a lo sumo dos, que serian adyacentes). Al seleccionar el centroeide como raiz y repetir el proceso recursivamente sobre cada componente resultante, se obtiene un **arbol de centroides** de altura a lo sumo $O(\log n)$.

La propiedad medular de esta descomposicion es que **todo camino simple entre dos nodos u y v pasa por un unico centroeide comun C que es su LCA en el arbol de centroides**, cumpliendose que $\text{dist}(u, v) = \text{dist}(u, C) + \text{dist}(C, v)$.

Propiedades clave (invariantes).

- **Cota de tamaño:** cada subarbol en el nivel de recursion k tiene tamaño $\leq n/2^k$.
- **Altura logaritmica:** el arbol de centroides tiene una profundidad maxima de $\lfloor \log_2 n \rfloor + 1$.
- **removed[v]:** marca los vertices fijados como centroides para desconectar de facto las componentes sin modificar la lista de adyacencia.
- **centroidParent[c]:** almacena el padre del nodo c en el arbol de centroides (para la raiz del arbol de centroides es -1).

Codigo clave. Búsqueda del centroide en una componente de tamaño total:

```
int dfsCentroid(int v, int p, int total) {
    for (int u : adj[v]) {
        if (u != p && !removed[u] && subSize[u] > total / 2) {
            return dfsCentroid(u, v, total);
        }
    }
    return v;
}
```

Complejidad.

- **Construccion (Build):** $O(n \log n)$ en tiempo. En cada uno de los $O(\log n)$ niveles de profundidad, cada vertice es visitado $O(1)$ veces para calcular su tamaño y localizar el centroide.
- **Consultas / Actualizaciones dinamicas:** tipicamente $O(\log n)$ o $O(\log^2 n)$ por evento, iterando sobre los $O(\log n)$ ancestros en el arbol de centroides.
- **Memoria:** $O(n)$ para listas de adyacencia y vectores auxiliares.

Donde tocar para modificar.

- **Caminos de longitud K o con propiedad X :** en cada centroide C , procesar las contribuciones de sus subarboles combinando los caminos que van de los nodos a C (usando dos punteros, frecuencias o estructuras como multiset/map), restando luego las combinaciones dentro del mismo subarbol para evitar falsos caminos.
- **Actualizaciones dinamicas (nodo coloreado mas cercano):** para cada nodo modificado, subir por sus ancestros en el arbol de centroides actualizando la mejor respuesta en la estructura asociada a cada ancestro.
- **Distancia rapida entre nodos:** precalcular distancias o LCA en el arbol original (mediante Binary Lifting en $O(1)$ o $O(\log n)$) para consultar $\text{dist}(u, v)$ rapidamente.

Trampas y errores frecuentes.

- **Calcular total localmente:** en cada llamada a `build(v, p)` se DEBE invocar primero `dfsSize(v, -1)` para obtener el tamaño real del componente conexo actual. Usar el N global provocara recursiones incorrectas o búsquedas fallidas.
- **Ignorar nodos con removed[u]:** tanto en `dfsSize` como en `dfsCentroid` y en la recursion de `build`, es imprescindible omitir vertices con `removed[u] == true`.
- **No desmarcar removed:** a diferencia de recorridos con backtracking, `removed[c]` se mantiene en `true` durante toda la descomposicion para aislar las componentes.

Explicacion linea por linea. Estructura CentroidDecomp:

- `struct CentroidDecomp { ... };` – encapsula la descomposicion por centroides.
- `int n;` – cantidad total de vertices.
- `vector<vector<int>> adj;` – lista de adyacencia del arbol.
- `vector<int> subSize;` – almacena el tamaño de subarbol en la componente actual.

- `vector<bool> removed;` – bandera que indica si un nodo ya fue elegido como centroide previo.
- `vector<int> centroidParent;` – padre de cada nodo en el arbol de centroides generado.
- `CentroidDecomp(int n): n(n)` – constructor: dimensiona vectores e inicializa `centroidParent` con `-1`.
- `void addEdge(int u, int v)` – inserta una arista bidireccional entre u y v .
- `int dfsSize(int v, int p)` – recorre la componente actual calculando tamanos ignorando nodos removidos.
- `int dfsCentroid(int v, int p, int total)` – desciende por la rama que contiene estrictamente mas de $\lfloor \text{total}/2 \rfloor$ nodos.
- `return v;` – si ningun hijo supera la mitad del tamano, v es el centroide de la componente.
- `void build(int v = 0, int p = -1)` – funcion recursiva que construye el arbol de centroides.
- `int total = dfsSize(v, -1);` – calcula el tamano exacto de la componente conexa activa.
- `int c = dfsCentroid(v, -1, total);` – encuentra el centroide c de la componente.
- `centroidParent[c] = p; removed[c] = true;` – conecta c a su padre en el arbol de centroides y lo desactiva.
- `for(int u : adj[c]) if(!removed[u]) build(u, c);` – descompone recursivamente cada subcomponente restante.

Programa principal:

- `CentroidDecomp cd(5);` – inicializa la estructura para un arbol de 5 nodos.
- `cd.addEdge(0, 1); cd.addEdge(1, 2); ...` – inserta las aristas del arbol.
- `cd.build();` – ejecuta la descomposicion recursiva.
- `cout << cd.centroidParent[0] << "\n";` – imprime el padre del nodo 0 en el arbol de centroides.

Codigo. Ver `cpp.tex`, seccion *Centroid Decomposition*.

3.4 Euler Tour + range queries

Idea intuitiva. Un **Euler Tour** (primer-visita DFS) asigna a cada nodo v un tiempo de entrada `tin[v]` y un tiempo de salida `tout[v]`. El subarbol de v corresponde exactamente al **rango contiguo** $[\text{tin}[v], \text{tout}[v]]$ en el arreglo base. La razon del rango contiguo: el DFS entra al subarbol de v y no sale hasta haber visitado todos sus descendientes; por ende, las posiciones $\text{tin}[v], \text{tin}[v] + 1, \dots, \text{tout}[v]$ corresponden exactamente a los nodos del subarbol en orden de visita.

Esto permite resolver queries y actualizaciones sobre subarboles con cualquier estructura de rango (BIT, Segment Tree) sobre el arreglo base de tamano n , con la correspondencia directa: operar sobre el subarbol de v es equivalente a operar sobre el rango $[\text{tin}[v], \text{tout}[v]]$.

Propiedades clave (invariantes).

- `tin[v]`: indice en el arreglo base al momento de **entrar** al nodo v (se asigna antes de visitar los hijos).
- `tout[v]`: indice del **ultimo descendiente** de v visitado (se asigna despues de recursionar en todos los hijos).
- Tamano del subarbol de v : $\text{tout}[v] - \text{tin}[v] + 1$.
- Verificacion de ancestro en $O(1)$: u es ancestro de v si y solo si $\text{tin}[u] \leq \text{tin}[v]$ y $\text{tout}[v] \leq \text{tout}[u]$.

Codigo clave. El DFS que construye el Euler Tour:

```
void dfs(int v, int p) {
    tin[v] = timer++;
    for (int u : adj[v])
        if (u != p) dfs(u, v);
    tout[v] = timer - 1;
}
```

Complejidad.

- **Construccion:** $O(n)$ en tiempo y memoria.
- **Query / Update sobre subarbol:** $O(\log n)$ con BIT o Segment Tree sobre el arreglo base.
- **Verificacion de ancestro:** $O(1)$ sin estructuras adicionales.

Donde tocar para modificar.

- **Valor por nodo:** durante el DFS, guardar el valor del nodo v en la posicion $\text{tin}[v]$ del arreglo base. Luego usar BIT/Segment Tree sobre ese arreglo para responder queries de rango.
- **Queries en caminos:** el Euler Tour de primera visita **no** resuelve queries sobre caminos arbitrarios $u \rightarrow v$; para eso usar HLD o LCA con Binary Lifting.
- **LCA via Euler Tour de aristas:** existe una variante en que se registra cada nodo tanto al entrar como al salir (generando un arreglo de tamaño $2n - 1$); sobre este arreglo se puede calcular LCA con RMQ. Esta variante es distinta de la implementacion actual.
- **Bosque (grafo no conexo):** invocar `build(r)` para cada raiz r de cada componente conexa.

Trampas y errores frecuentes.

- **Confusion entre tin y posicion en arreglo:** $\text{tin}[v]$ es el indice donde se ubica v en el arreglo base, NO el instante de tiempo en sentido reloj. El arreglo base de tamaño n tiene exactamente una entrada por nodo.
- **Formula de tout:** en esta implementacion $\text{tout}[v] = \text{timer} - 1$ (la posicion del ultimo hijo procesado), de modo que el subarbol es $[\text{tin}[v], \text{tout}[v]]$ **inclusive**. Si se usa $\text{tout}[v] = \text{timer}$ sin restar, el rango seria $[\text{tin}[v], \text{tout}[v])$ semiincluyente — mezclar convenciones causa bugs sutiles.
- **Timer inicial:** el timer comienza en 0 y los rangos son 0-indexados; al usar un BIT 1-indexado hay que sumar 1 a todos los indices.

Explicacion linea por linea. Estructura EulerTour:

- `struct EulerTour { ... };` – encapsula el Euler Tour de primera visita.
- `int n, timer;` – cantidad de nodos y contador global de posiciones asignadas.
- `vector<int> tin, tout;` – tiempos de entrada y salida de cada nodo.
- `vector<vector<int>> adj;` – lista de adyacencia del arbol.
- `EulerTour(int n): n(n)` – constructor: inicializa los vectores y pone `timer = 0`.
- `void addEdge(int u, int v)` – inserta arista bidireccional entre u y v .
- `void dfs(int v, int p)` – DFS que asigna $\text{tin}[v]$ al entrar y $\text{tout}[v]$ al salir.
- `tin[v] = timer++;` – asigna la posicion actual a v y avanza el contador.
- `for(int u : adj[v]) if(u != p) dfs(u, v);` – recursa en los hijos (excluye al padre p).
- `tout[v] = timer - 1;` – el ultimo indice asignado dentro del subarbol de v es `timer - 1`.
- `void build(int root = 0)` – inicia el DFS desde `root` con padre ficticio `-1`.

Programa principal:

- `EulerTour et(5);` – crea la estructura para un arbol de 5 nodos.
- `et.addEdge(0, 1); et.addEdge(1, 2); ...` – construye la topologia del arbol.
- `et.build(0);` – ejecuta el DFS con el nodo 0 como raiz.
- `cout << "tin[1]=" << et.tin[1] << " tout[1]=" << et.tout[1] << "\n";` – imprime el rango del subarbol del nodo 1.

Codigo. Ver `cpp.tex`, seccion *Euler Tour + range queries*.

3.8 BST balanceado

3.1 Treap implicito

Idea intuitiva. Un **treap implicito** es un arbol binario de busqueda aleatorizado donde cada nodo tiene un **valor**, una **prioridad aleatoria** y un **tamano de subarbol**. La invariante es: heap por prioridad (el padre tiene mayor prioridad que sus hijos), y el **indice implicito** de un nodo en la secuencia viene dado por el tamano del subarbol izquierdo; no se almacena una clave explicita.

Las dos primitivas fundamentales son:

- **split(*t*, *k*, *l*, *r*)**: divide el treap *t* en *l* (primeros *k* elementos, rango $[0, k)$) y *r* (el resto, rango $[k, n)$).
- **merge(*l*, *r*)**: une dos treaps manteniendo el orden (todos los de *l* antes que los de *r*), eligiendo como raiz al nodo de mayor prioridad para preservar la propiedad de heap.

La altura esperada del arbol es $O(\log n)$ gracias a la aleatoriedad de las prioridades; no se necesita rebalanceo explicito.

Propiedades clave (invariantes).

- **sz(*t*)**: tamano del subarbol de *t* (0 si *t* == nullptr).
- Propiedad de heap: *t*->pr >= *t*->l->pr y *t*->pr >= *t*->r->pr.
- Invariante de posicion: el indice de *t* en la secuencia es **sz(*t* → *l*)**.
- Toda operacion comienza con **upd(*t*)** al retornar para mantener **sz** correcto.

Codigo clave. Split por posicion (base del treap implicito):

```
void split(Node* t, int k, Node*& l, Node*& r) {
    if (!t) { l = r = nullptr; return; }
    if (sz(t->l) >= k) {
        split(t->l, k, l, t->l);
        r = t; upd(r);
    } else {
        split(t->r, k - sz(t->l) - 1, t->r, r);
        l = t; upd(l);
    }
}
```

Complejidad.

- Todas las operaciones (split, merge, insert, erase): $O(\log n)$ **esperado** en promedio sobre las prioridades aleatorias.
- El peor caso teorico es $O(n)$, pero la probabilidad de alcanzarlo decrece exponencialmente.
- Memoria: $O(n)$ nodos; cada nodo ocupa espacio constante.

Donde tocar para modificar.

- **Lazy propagation**: anadir un campo **lazy** al nodo y aplicarlo en una funcion **push(*t*)** antes de acceder a los hijos en **split** y **merge**.
- **Operacion de reversa en rango**: con lazy, se puede implementar una operacion que invierta un subrango $[l, r]$ en $O(\log n)$: hacer split/split, marcar lazy, y merge/merge.
- **Aggregate en rango**: anadir campo **agg** al nodo. Recalcularlo en **upd** con **op(*l*->agg, val, *r*->agg)**.
- **Mejor generador aleatorio**: sustituir **rand()** por **mt19937 rng(...)** con semilla de **chrono::steady_clock**, usando **rng()** en el constructor del nodo.
- **Treap por clave (no implicito)**: guardar la clave en el nodo y cambiar la condicion de split por comparacion de clave en lugar de tamano de subarbol izquierdo.

Trampas y errores frecuentes.

- **Olvidar upd al retornar:** tras modificar cualquier hijo, se debe llamar `upd(t)` antes de retornar; de lo contrario el campo `sz` queda desactualizado y todas las operaciones subsiguientes calculan índices incorrectos.
- **Semantica de `split(t, k, l, r)`:** produce l con exactamente k elementos (índices $[0, k)$) y r con los restantes (índices $[k, n)$). Confundir si el rango es abierto o cerrado genera off-by-one en insert y erase.
- **Memory leak con new:** en contests es aceptable, pero en producción es necesario un destructor o un allocator de pool de nodos para evitar fugas.
- **Prioridades iguales:** si dos nodos tienen la misma prioridad, el treap sigue siendo correcto (la propiedad de heap se cumple con $\geq$), pero la altura esperada puede degradarse ligeramente con `rand()` de baja calidad.

Explicación línea por línea. Estructura Treap:

- `struct Treap { struct Node { int val, pr, sz; Node *l, *r; }; };` – estructura exterior con nodo interno.
- `Node(int v): val(v), pr(rand()), sz(1), l(nullptr), r(nullptr){}` – nuevo nodo: valor dado, prioridad aleatoria, tamaño 1, hijos nulos.
- `Node* root = nullptr;` – puntero a la raíz del treap (árbol vacío inicialmente).
- `int sz(Node* t)` – retorna `t->sz` o 0 si `t` es nulo; evita acceder a punteros nulos.
- `void upd(Node* t)` – recalcula `t->sz = 1 + sz(t->l) + sz(t->r)`; se llama al retornar de cada operación.
- `void split(Node* t, int k, Node*& l, Node*& r)` – divide t en l (k elementos) y r (resto).
- `if(sz(t->l) >= k)` – el corte cae dentro del subárbol izquierdo: se recurre en el izquierdo.
- `split(t->r, k - sz(t->l) - 1, t->r, r);` – el corte está en el derecho; se descuenta el subárbol izquierdo y la raíz actual (-1).
- `Node* merge(Node* l, Node* r)` – une l y r manteniendo prioridades de heap.
- `if(l->pr > r->pr)` – l tiene mayor prioridad: es la raíz; su hijo derecho absorbe el resto.
- `void insert(int pos, int val)` – inserta un nuevo nodo con `val` en la posición `pos`: `split` → `merge` con nuevo nodo → `merge`.
- `void erase(int pos)` – elimina el elemento en posición `pos`: `split`, `split` de 1 elemento, `delete`, `merge`.

Programa principal:

- `Treap t;` – instancia un treap vacío.
- `t.insert(0, 10); t.insert(1, 20); t.insert(2, 30);` – inserta tres elementos en posiciones 0, 1 y 2.
- `t.erase(1);` – elimina el elemento en la posición 1 (el valor 20).

Código. Ver `cpp.tex`, sección *Treap implícito*.

3.9 Miscelaneas de estructuras

3.1 Mo con updates

Idea intuitiva. Extensión del algoritmo de Mo clásico para queries offline que intercalan **updates puntuales**. Cada query se caracteriza por un triple (L, R, T) donde T es el número de updates ocurridos antes de responderla. Las queries se ordenan por $(L/B, R/B, T)$ con $B = n^{2/3}$, de modo que los punteros L , R y T se mueven un total de $O((n + q) \cdot n^{2/3})$ pasos.

La clave es que los updates deben ser **invertibles**: `applyUpdate` y `revertUpdate` son inversos exactos, permitiendo avanzar y retroceder el tiempo arbitrariamente.

Propiedades clave (invariantes).

- Tres punteros: `curL`, `curR` (rango activo) y `curT` (tiempo activo).
- `addPos(x)` y `removePos(x)` son inversos exactos: agregar y luego quitar x debe dejar el estado invariante.
- `applyUpdate(u)` y `revertUpdate(u)` son inversos exactos.
- Tamaño de bloque óptimo: $B = \lceil n^{2/3} \rceil$. Con Q queries y U updates, la complejidad es $O((n + Q) \cdot n^{2/3})$.

Codigo clave. Comparador de ordenamiento con tres coordenadas:

```
int blockSize = (int)pow(n, 2.0/3.0);
sort(queries.begin(), queries.end(),
    [&](const Query& A, const Query& B) {
        int bA = A.l / blockSize, bB = B.l / blockSize;
        if (bA != bB) return bA < bB;
        int bAr = A.r / blockSize, bBr = B.r / blockSize;
        if (bAr != bBr) return bAr < bBr;
        return A.t < B.t;
    });
```

Complejidad.

- **Total:** $O((n + Q) \cdot n^{2/3})$ con bloque optimo $B = n^{2/3}$.
- **Movimientos de L y R :** dentro de un bloque fijo de L , R puede recorrer $O(n)$; hay $O(n^{1/3})$ bloques de R , y hay $O(n^{1/3})$ bloques de $L \Rightarrow O(n \cdot n^{1/3} \cdot n^{1/3}) = O(n^{5/3})$ pasos de L y R .
- **Movimientos de T :** dentro de un bloque de L y de R , T es monotono $\Rightarrow$ a lo sumo Q pasos por par de bloques; hay $O(n^{2/3})$ pares de bloques $\Rightarrow O(Q \cdot n^{2/3})$ pasos de T .
- Funciona bien en la practica hasta $n, Q \sim 10^5$.

Donde tocar para modificar.

- **Logica del problema:** implementar `addPos`, `removePos`, `applyUpdate`, `revertUpdate` segun el enunciado (ej. contar elementos distintos, sumas de frecuencias, etc.).
- **Update que afecta el rango activo:** en `applyUpdate` y `revertUpdate`, verificar si `u.pos` esta dentro de `[curL, curR]` para actualizar `curAns` adecuadamente.
- **Orden de movimiento de punteros:** mover L y R antes que T es importante para mantener la consistencia del estado al aplicar o revertir updates.

Trampas y errores frecuentes.

- **No implementar `revertUpdate`:** es el error mas comun. `revertUpdate` debe ser el inverso **exacto** de `applyUpdate`; no basta con re-aplicar la operacion si esta no es su propia inversa.
- **Orden de los punteros:** en el bucle principal, primero se mueven L y R , luego T . Mover T antes puede generar estados inconsistentes donde un update se aplica al rango equivocado.
- **Inicializacion:** `curL = 0`, `curR = -1`, `curT = -1` asegura que el rango inicial este vacio y el tiempo sea anterior a cualquier update.
- **Bloque muy pequeno:** si $B \ll n^{2/3}$, los movimientos de T dominan; si $B \gg n^{2/3}$, los movimientos de L y R dominan. El optimo es exactamente $n^{2/3}$.

Explicacion linea por linea. Estructura `MoWithUpdates`:

- `struct MoWithUpdates { ... };` – estructura que encapsula `Mo` con updates.
- `struct Query { int l, r, t, idx; };` – query con extremo izquierdo, derecho, tiempo y su indice original.
- `struct Update { int pos, val; };` – update puntual con posicion y nuevo valor.
- `vector<int> a;` – arreglo base sobre el que se aplican los updates.
- `long long curAns = 0;` – respuesta acumulada del estado actual del rango `[curL, curR]`.
- `void addPos(int x)` – agrega la posicion x al rango activo; debe actualizar `curAns`.
- `void removePos(int x)` – quita la posicion x del rango activo; inverso exacto de `addPos`.
- `void applyUpdate(const Update& u)` – avanza el tiempo aplicando el update u ; si `u.pos` esta en el rango, actualiza `curAns`.

- `void revertUpdate(const Update& u)` – retrocede el tiempo revirtiendo el update u ; inverso exacto de `applyUpdate`.
- `int blockSize = pow(n, 2.0/3.0);` – tamaño óptimo de bloque para la complejidad $O((n + Q) \cdot n^{2/3})$.
- `int curL = 0, curR = -1, curT = -1;` – punteros iniciales: rango vacío, tiempo previo a todo update.
- `while(curL > qry.l) addPos(--curL);` – expande el rango hacia la izquierda.
- `while(curR < qry.r) addPos(++curR);` – expande el rango hacia la derecha.
- `while(curL < qry.l) removePos(curL++);` – contrae el rango por la izquierda.
- `while(curR > qry.r) removePos(curR--);` – contrae el rango por la derecha.
- `while(curT < qry.t) applyUpdate(updates[++curT]);` – avanza el tiempo aplicando updates.
- `while(curT > qry.t) revertUpdate(updates[curT--]);` – retrocede el tiempo revirtiendo updates.
- `ans[qry.idx] = curAns;` – guarda la respuesta en la posición original de la query.

Programa principal:

- `MoWithUpdates mwu;` – instancia el esqueleto.
- Las cuatro funciones `addPos`, `removePos`, `applyUpdate`, `revertUpdate` deben implementarse según el problema específico antes de llamar a `mwu.solve()`.

Código. Ver `cpp.tex`, sección *Mo con updates*.

3.2 sqrt-decomposition generico

Idea intuitiva. Dividir un arreglo de n elementos en **bloques** disjuntos contiguos de tamaño $B \approx \sqrt{n}$. Para cada bloque se mantiene una variable agregada con el resumen de sus elementos (suma, mínimo, máximo, etc.).

Las consultas en rango $[l, r]$ se descomponen en tres partes:

1. La cola del bloque izquierdo (elementos sueltos desde l hasta el fin de su bloque).
2. Los bloques intermedios completos, cuyo valor precalculado se acumula en $O(1)$ por bloque.
3. La cabeza del bloque derecho (elementos sueltos desde el inicio de su bloque hasta r).

Si l y r pertenecen al mismo bloque, se itera directamente entre ellos. Con $B \approx \sqrt{n}$, tanto la cantidad de bloques como el tamaño de cada bloque es $O(\sqrt{n})$, garantizando que cualquier consulta o actualización tome tiempo $O(\sqrt{n})$.

Propiedades clave (invariantes).

- Tamaño de bloque: $B = \lfloor \sqrt{n} \rfloor + 1$, lo que garantiza $B^2 > n$.
- Cantidad de bloques: $\lceil n/B \rceil \leq B$.
- El elemento en índice i pertenece al bloque con índice $\lfloor i/B \rfloor$.
- `bucket[b]`: resumen acumulado de todos los elementos pertenecientes al bloque b .
- Toda consulta sobre $[l, r]$ inspecciona a lo sumo dos bloques parciales ($\leq 2B$ elementos) y $O(n/B)$ bloques completos.

Código clave. Consulta por rango combinando bloques completos y elementos individuales:

```
T rangeSum(int l, int r) {
    T res = 0;
    int bl = l / B, br = r / B;
    if (bl == br) {
        for (int i = l; i <= r; ++i) res += a[i];
    } else {
        for (int i = l; i < (bl + 1) * B; ++i) res += a[i];
        for (int i = bl + 1; i < br; ++i) res += bucket[i];
        for (int i = br * B; i <= r; ++i) res += a[i];
    }
    return res;
}
```

Complejidad.

- **Construccion:** $O(n)$ calculando el resumen de cada bloque en una sola pasada.
- **Actualizacion puntual:** $O(1)$ actualizando el elemento $a[p]$ y ajustando $\text{bucket}[p / B]$.
- **Consulta de rango:** $O(\sqrt{n})$ en el peor caso ($< 2B + n/B$ operaciones).
- **Memoria:** $O(n)$ para el arreglo base y $O(\sqrt{n})$ para el arreglo de bloques.

Donde tocar para modificar.

- **Minimo / Maximo (RMQ):** cambiar la operacion aditiva por `std::min` o `std::max`. Para actualizaciones puntuales en RMQ, si un valor se incrementa/reduce en contra del extremo, puede ser necesario recomputar el bloque completo en $O(B) = O(\sqrt{n})$.
- **Actualizaciones en rango:** anadir un arreglo `lazy[b]`. Los bloques completos hacen `lazy[b] += val` en $O(1)$; los parciales se actualizan en $O(B)$.
- **Balance de complejidad:** si hay muchas mas consultas que actualizaciones (o viceversa), fijar B distinto de $\sqrt{n}$ para balancear los costos totales.

Trampas y errores frecuentes.

- **Mismo bloque:** olvidar el caso especial `bl == br` provoca que los bucles de los extremos se solapen o que se acceda a rangos invertidos.
- **Dimension del vector bucket:** reservar espacio para $\lceil n/B \rceil$ bloques (en GCC/C++: $(n + B - 1) / B$ o $n / B + 1$). Con $B = \lfloor \sqrt{n} \rfloor + 1$, dimensionar en B es suficiente pues $B^2 > n$, pero cambiar B sin ajustar la dimension causara accesos fuera de rango (*Out of Bounds*).
- **Limite superior del ultimo bloque:** en `i < (bl + 1) * B`, si $(bl + 1) * B > n$ en el ultimo bloque parcial, asegurarse de no acceder a indices fuera de $[0, n - 1]$ (en consultas con $r < n$ la condicion `bl != br` previene esto, pero ante rangos mal acotados puede ocurrir).

Explicacion linea por linea. Estructura SqrtDecomp:

- `template<class T> struct SqrtDecomp { ... };` – estructura generica parametrizada por el tipo de dato.
- `int n, B;` – tamano del arreglo y tamano elegido para cada bloque.
- `vector<T> a, bucket;` – copia del arreglo original y arreglo de agregados por bloque.
- `SqrtDecomp(const vector<T>& v): a(v)` – constructor: almacena el arreglo v .
- `B = (int)sqrt(n) + 1;` – fija el tamano de bloque garantizando $B^2 > n$.
- `bucket.assign(B, 0);` – inicializa los agregados de cada bloque en 0.
- `for(int i = 0; i < n; i++) bucket[i / B] += v[i];` – precarga las sumas de cada bloque.
- `void pointUpdate(int p, T val)` – actualiza puntualmente en $O(1)$: resta el valor viejo y suma el nuevo a su bloque.
- `T rangeSum(int l, int r)` – responde la suma en el rango $[l, r]$ en tiempo $O(\sqrt{n})$.
- `if(bl == br)` – si ambos extremos estan en el mismo bloque, recorre linealmente el subrango.
- `for(int i = l; i < (bl+1)*B; i++) res += a[i];` – acumula los elementos sueltos del bloque izquierdo.
- `for(int i = bl + 1; i < br; i++) res += bucket[i];` – suma los bloques completos intermedios de forma directa en $O(1)$ cada uno.
- `for(int i = br * B; i <= r; i++) res += a[i];` – acumula los elementos sueltos del bloque derecho.

Programa principal:

- `SqrtDecomp<int> sd({1, 2, 3, 4, 5, 6, 7, 8, 9, 10});` – crea la estructura para un arreglo de 10 enteros.
- `sd.rangeSum(2, 7);` – consulta la suma entre indices 2 y 7 inclusive (resultado 33).
- `sd.pointUpdate(4, 100);` – cambia el elemento en posicion 4 a 100.
- `sd.rangeSum(2, 7);` – recalcula la suma reflejando el cambio (resultado 128).

Codigo. Ver `cpp.tex`, seccion *sqrt-decomposition generico*.

4 Strings

4.1 Hashing

4.1 Rolling Hash (doble modulo)

Idea intuitiva. Un **rolling hash** (o hash polinomial) asigna a cada prefijo de una cadena un valor numerico asociando los caracteres a los coeficientes de un polinomio evaluado en una base A modulo M :

$$H(s[0 \dots k-1]) = \sum_{i=0}^{k-1} s[i] \cdot A^{k-1-i} \pmod{M}$$

La ventaja crucial es que el hash de cualquier subsecuencia contigua $s[l \dots r]$ (1-indexada) se obtiene en tiempo $O(1)$ restando los hashes de los prefijos con el debido escalamiento:

$$\text{hash}(s[l \dots r]) = (h[r] - h[l-1] \cdot A^{r-l+1}) \pmod{M}$$

Para mitigar colisiones accidentales o intencionales (anti-hash tests en competencias), se calcula simultaneamente con **dos modulos primos distintos** (doble hash), reduciendo la probabilidad de colision entre dos cadenas distintas a aproximadamente $1/(M_1 \cdot M_2) \approx 10^{-18}$.

Propiedades clave (invariantes).

- $h[i]$: hash del prefijo de longitud i (con $h[0] = (0,0)$).
- $p[i]$: potencia A^i para ambos modulos precalculada en $O(n)$.
- Comparacion $O(1)$: dos subcadenas son identicas con probabilidad abrumadora si sus pares de hashes coinciden en ambos modulos.
- Indexacion 1-based: la subcadena que va desde el caracter l hasta r inclusive abarca longitud $r - l + 1$.

Codigo clave. Extraccion del hash de subcadena en $O(1)$ con ajuste para diferencias negativas:

```
pair<int, int> getHash(int l, int r) {
    pair<int, int> ans;
    ans.first = (h[r].first - 1LL * h[l-1].first * p[r-l+1].first) % MOD1;
    ans.second = (h[r].second - 1LL * h[l-1].second * p[r-l+1].second) % MOD2;
    ans.first = (ans.first + MOD1) % MOD1;
    ans.second = (ans.second + MOD2) % MOD2;
    return ans;
}
```

Complejidad.

- **Precomputo:** $O(n)$ en tiempo y memoria (construccion de tablas h y p).
- **Consulta de hash:** $O(1)$ por cada intervalo $[l, r]$.
- **Memoria:** $O(n)$ para almacenar los pares de enteros de tamaño $n+1$.

Donde tocar para modificar.

- **Cambiar alfabeto o caracteres no ASCII:** la formula suma $s[i-1]$ como entero directo; si el alfabeto contiene mayusculas, minusculas o digitos, puede usarse el valor `char` directo o normalizar con $s[i-1] - 'a' + 1$ (evitando el valor 0 para no colapsar caracteres nulos con prefijos).
- **Anti-hack (Base aleatoria):** en plataformas como Codeforces, las bases fijas como $A = 31$ o $A = 53$ pueden ser hackeadas con pruebas precalculadas de Thue-Morse. Se puede generar A de forma aleatoria en el rango $[256, M-1]$ usando `mt19937`.

- **Concatenar cadenas por hash:** el hash de $S + T$ se calcula como $(H(S) \cdot A^{|T|} + H(T)) \pmod{M}$.
- **Hash inverso / Palindromos:** construir un objeto H sobre la cadena invertida para comprobar si $s[l \dots r]$ es palindromo comparando su hash directo con su hash reflejado en $O(1)$.

Trampas y errores frecuentes.

- **Resta negativa en modulo:** en C++, el operador % sobre enteros negativos da un resultado negativo. Es imperativo sumar el modulo (+ MOD) antes de volver a aplicar modulo.
- **Desbordamiento en el producto:** la multiplicacion `h[l-1] * p[len]` puede desbordar un entero de 32 bits con signo. Se debe castear a `long long` (por ejemplo multiplicando por 1LL).
- **Mapeo de caracter 0:** si un caracter se codifica como 0, las cadenas “a”, “aa”, “aaa” tendrian hashes identicos al multiplicarse por potencias de la base. El valor sumado por caracter debe ser siempre ≥ 1 .

Explicacion linea por linea. Estructura pair_hash:

- `struct pair_hash` – functor que empaqueta dos enteros de 32 bits en un `uint64_t` mediante desplazamiento binario y XOR, permitiendo usar `pair<int,int>` en `unordered_set` o `unordered_map`.

Estructura H:

- `vector<pair<int,int>> h, p;` – tablas de hashes acumulados y potencias de la base para cada longitud.
- `const int MOD1 = 998244353, MOD2 = 1e9 + 7;` – modulos primos coprimos grandes elegidos para minimizar la probabilidad de colision conjunta.
- `const int A = 31;` – base polinomial (debe ser mayor que el rango del alfabeto).
- `H(string &a)` – constructor: dimensiona h y p a $n + 1$, e inicializa $p[0] = (1, 1)$.
- `h[i].F = ((h[i-1].F * A) % MOD1 + s[i-1]) % MOD1;` – aplica la recurrencia polinomial acumulando el caracter actual en el hash del primer modulo.
- `p[i].F = (p[i-1].F * A) % MOD1;` – precalcula $A^i \pmod{MOD1}$.
- `pair<int,int> getHash(int l, int r)` – retorna el doble hash de la subcadena en el rango 1-indexado $[l, r]$.
- `ans.F = (h[r].F - (h[l-1].F * p[r-l+1].F) % MOD1) % MOD1;` – subtrae la contribucion del prefijo previo escalado por la potencia correspondiente.
- `ans.F = (ans.F + MOD1) % MOD1;` – normaliza el valor al rango no negativo $[0, MOD1 - 1]$.

Programa principal:

- `H h(s);` – precalcula las tablas sobre el string base.
- `auto a = h.getHash(1, 3);` – extrae el doble hash de la subcadena de indices 1 a 3 (“ban”).
- `cout << (a == b ? "iguales" : "distintos");` – compara igualdad exacta entre ambos hashes en tiempo constante.

Codigo. Ver `cpp.tex`, seccion *Rolling Hash (doble modulo)*.

4.2 Patrones

4.1 KMP + LPS

Idea intuitiva. El algoritmo **Knuth-Morris-Pratt (KMP)** busca todas las apariciones de un patron P de longitud m dentro de un texto T de longitud n en tiempo lineal $O(n + m)$.

La clave teorica reside en la tabla **LPS** (Longest Proper Prefix which is Suffix), donde `lps[i]` almacena la longitud del prefijo propio mas largo de $P[0 \dots i]$ que coincide exactamente con un sufijo propio de dicho subsegmento. Cuando ocurre una discrepancia (*mismatch*) entre el texto y el patron en la posicion j , no es necesario retroceder el puntero del texto i ; el conocimiento de la tabla LPS permite saltar el puntero del patron directamente a $j = \text{lps}[j - 1]$, conservando los caracteres coincidentes previos.

Propiedades clave (invariantes).

- $\text{lps}[i]$: longitud del prefijo mas largo de P que es a la vez sufijo propio de $P[0 \dots i]$.
- El puntero i en el texto nunca retrocede: solo avanza o permanece en su lugar mientras j retrocede, garantizando $O(n + m)$.
- Periodo minimo: si $n \pmod{n - \text{lps}[n - 1]} == 0$, entonces la subcadena de longitud $n - \text{lps}[n - 1]$ es el periodo minimo repetido de la cadena completa.
- Autómata KMP: $\text{lps}[j - 1]$ representa el estado de falla del autómata finito determinista para coincidencia de cadenas.

Codigo clave. Construcción de la tabla LPS y búsqueda en el texto:

```
vector<int> create_lps(string &needle) {
    int n = needle.size(), i = 1, prevLPS = 0;
    vector<int> lps(n, 0);
    while (i < n) {
        if (needle[i] == needle[prevLPS]) {
            lps[i++] = ++prevLPS;
        } else if (prevLPS == 0) {
            lps[i++] = 0;
        } else {
            prevLPS = lps[prevLPS - 1];
        }
    }
    return lps;
}
```

Complejidad.

- **Construcción LPS:** $O(m)$ en tiempo y memoria, donde $m = |P|$.
- **Busqueda:** $O(n)$ en tiempo, donde $n = |T|$.
- **Total:** $O(n + m)$ en tiempo y $O(m)$ en memoria adicional para la tabla LPS.

Donde tocar para modificar.

- **Guardar posiciones:** en $j == \text{sz_pattern}$, el match inicia en $i - \text{sz_pattern}$; se registra con `ans.push_back(i - sz_p`
- **Contar ocurrencias de cada prefijo:** calculando la frecuencia de cada valor en LPS y propagando de derecha a izquierda: `ans[lps[i]-1] += ans[i]`.
- **Compresion de cadena / Periodo:** el periodo mas corto de un string S de longitud n es $p = n - \text{lps}[n - 1]$. Si $n \% p == 0$, la cadena es periodica con n/p repeticiones.
- **DP sobre automatas:** para problemas que piden contar cadenas que no contienen a P , se precomputan las transiciones del automata: `aut[state][c]` para cada caracter.

Trampas y errores frecuentes.

- **Acceso en $j == 0$:** al ocurrir un mismatch, verificar siempre si $j == 0$ antes de indexar `lps[j - 1]` para no causar lecturas fuera de rango.
- **Continuar tras un match:** al hallar $j == \text{sz_pattern}$, no reiniciar $j = 0$; hacer $j = \text{lps}[j - 1]$ para detectar ocurrencias solapadas (ej. “ana” en “anana”).
- **Caso base en $i = 0$:** por definicion de sufijo **propio**, $\text{lps}[0] = 0$. El bucle de construcción de LPS siempre inicia en $i = 1$.

Explicacion linea por linea. Funcion `create_lps`:

- `vector<int> create_lps(string &needle)` – construye la tabla de prefijos-sufijos del patron `needle`.
- `int n = needle.size(); vector<int> lps(n);` – dimensiona el vector con ceros; `lps[0] = 0`.
- `int i = 1, prevLPS = 0;` – `i` recorre el patron desde el segundo caracter; `prevLPS` guarda el largo del prefijo coincidente previo.
- `if(needle[i] == needle[prevLPS])` – si el caracter actual extiende el prefijo previo coincidente.
- `lps[i] = prevLPS + 1; i++; prevLPS++;` – guarda el nuevo largo e incrementa ambos punteros.
- `else if(prevLPS == 0) { lps[i] = 0; i++; }` – no hay coincidencia previa: el prefijo-sufijo de longitud maxima es 0.
- `else prevLPS = lps[prevLPS - 1];` – intenta emparejar con un prefijo-sufijo mas corto ya precomputado sin avanzar `i`.

Funcion `solve` (busqueda en texto):

- `vector<int> lps = create_lps(p);` – precalcula la tabla para el patron `p`.
- `int i = 0, j = 0;` – `i` indexa el texto `s`; `j` indexa el patron `p`.
- `if(s[i] == p[j]) { i++; j++; }` – caracteres coinciden: avanza ambos punteros.
- `else if(j == 0) i++;` – mismatch al inicio del patron: avanza unicamente en el texto.
- `else j = lps[j - 1];` – mismatch parcial: retrocede el patron al mayor prefijo comun previo sin retroceder en el texto.
- `if(j == sz_pattern) { cnt++; j = lps[j - 1]; }` – patron completamente encontrado: cuenta la ocurrencia y retrocede `j` para admitir solapamientos.

Codigo. Ver `cpp.tex`, seccion *KMP + LPS*.

4.2 Z-algorithm

Idea intuitiva. El **algoritmo Z** calcula para cada posicion i de una cadena S de longitud n el valor $Z[i]$, definido como la longitud del prefijo comun mas largo (**LCP**) entre la cadena completa S y el sufijo que inicia en la posicion i :

$$Z[i] = \max\{k \geq 0 \mid S[0 \dots k-1] = S[i \dots i+k-1]\}$$

Por definicion, $Z[0] = n$ (o se deja en 0 segun la convencion de implementacion).

Para lograr tiempo $O(n)$, el algoritmo mantiene una ventana o segmento $[l, r]$ que representa el intervalo con el mayor indice r alcanzado tal que $S[l \dots r]$ coincide con el prefijo $S[0 \dots r-l]$. Al evaluar la posicion i :

- Si $i \leq r$, la subcadena en i ya fue comparada contra el prefijo; por ende, $Z[i]$ es al menos $\min(r-i+1, Z[i-l])$.
- Si la coincidencia llega hasta el borde r (o si $i > r$), se expande caracter por caracter incrementando $Z[i]$ y actualizando el nuevo intervalo $[l, r]$.

Propiedades clave (invariantes).

- **r**: extremo derecho del prefijo coincidente mas lejano hacia la derecha.
- Cada caracter del texto se compara con exito a lo sumo una vez para extender r , garantizando $O(n)$ en total.
- Coincidencia de patrones: concatenando $P + \text{'\#'} + T$ (donde **'\#'** es un caracter centinela inexistente en P y T), toda posicion $i > |P|$ con $Z[i] = |P|$ senala una aparicion exacta de P en T .

Codigo clave. Construcción del arreglo Z en $O(n)$:

```
vector<int> zFunction(const string& s) {
    int n = s.size(), l = 0, r = 0;
    vector<int> z(n, 0);
    for (int i = 1; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}
```

Complejidad.

- **Construcción:** $O(n)$ en tiempo estricto en el peor caso.
- **Memoria:** $O(n)$ para el vector retornado.

Donde tocar para modificar.

- **Contar apariciones de cada prefijo:** usar un arreglo de diferencias (*diff array*) sobre $Z[i]$ para contar cuantas veces aparece cada prefijo $S[0 \dots k-1]$ en $O(n)$.
- **Detectar bordes y periodos:** si $i + Z[i] == n$, el prefijo de longitud $Z[i]$ es también sufijo de S (un borde). El periodo mínimo de S es i para el menor i tal que $i + Z[i] == n$ y $n \% i == 0$.
- **Palindromos:** calculando la Z-function sobre $S + \# + \text{reverse}(S)$, se obtienen las longitudes de los palindromos impares o prefijos que son palindromos.
- **Patron en texto:** concatenar $P + \# + T$; el índice en el texto original es $i - |P| - 1$.

Trampas y errores frecuentes.

- **Separador inadecuado:** el caracter centinela (como '#' o '\$') NO debe pertenecer al alfabeto del texto ni del patron; si aparece, permitira falsas coincidencias a traves del delimitador.
- **Convencion en $Z[0]$:** la implementacion asigna $Z[0] = 0$. En problemas de bordes o autorreferencia, recordar que el prefijo completo tiene coincidencia trivial de longitud n .
- **Off-by-one en indices de patron:** en $P + \# + T$, la posicion de inicio en T es $i - |P| - 1$. Olvidar restar el centinela desfasa las coincidencias en 1.

Explicacion linea por linea. Funcion zFunction:

- `int n = s.size(); vector<int> z(n, 0);` – inicializa el vector Z de tamaño n en 0.
- `int l = 0, r = 0;` – extremos del intervalo $[l, r]$ con mayor r alcanzado.
- `for(int i = 1; i < n; i++)` – el calculo inicia en $i = 1$ ($Z[0]$ se mantiene en 0 por definicion del vector).
- `if(i <= r) z[i] = min(r - i + 1, z[i - l]);` – si i cae dentro del bloque $[l, r]$, copia el valor ya computado en $i - l$, truncado al borde derecho restante.
- `while(i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;` – expande trivialmente mientras los caracteres sigan coincidiendo mas alla del bloque previo.
- `if(i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }` – si el nuevo match sobrepasa el extremo derecho anterior r , actualiza la ventana $[l, r]$.

Aplicaciones en main:

- `best = max(best, z[i]);` – calcula el LCP maximo entre la cadena s y cualquiera de sus sufijos propios.
- `prefCnt[1]++; prefCnt[z[i] + 1]--;` – utiliza un arreglo de diferencias para propagar en $O(n)$ el conteo de apariciones de cada prefijo de longitud k .

- `auto zp = zFunction(s + "#" + rs);` – calcula palindromos impares concatenando la cadena con su reversa.
- `if(z2[i] == (int)p.size())` – en $P + \# + T$, detecta aparición exacta del patron en el índice $i - |P| - 1$ de T .

Codigo. Ver `cpp.tex`, seccion *Z-algorithm*.

4.3 Rabin-Karp (single hash)

Idea intuitiva. El algoritmo de **Rabin-Karp** busca ocurrencias de un patron P de longitud m dentro de un texto T de longitud n deslizando una ventana de tamaño fijo m y manteniendo el valor de su hash polinomial.

Al avanzar la ventana un caracter hacia la derecha, el nuevo hash se calcula en $O(1)$ removiendo el caracter saliente $T[i]$ (multiplicado por BASE^{m-1}) y anadiendo el nuevo caracter entrante $T[i + m]$:

$$H_{\text{sig}} = ((H_{\text{act}} - T[i] \cdot \text{BASE}^{m-1}) \cdot \text{BASE} + T[i + m]) \pmod{M}$$

Cuando el hash de la ventana coincide con el hash del patron ($tHash == pHash$), se realiza una verificación carácter por carácter para garantizar que no se trata de una colisión. Dado un módulo primo $M \sim 10^9$, la probabilidad de colisión aleatoria es insignificante ($1/M$), alcanzando una complejidad lineal esperada $O(n + m)$.

Propiedades clave (invariantes).

- Si los hashes son distintos, las cadenas son garantizadamente distintas.
- Si los hashes coinciden, se verifica la igualdad exacta para evitar falsos positivos.
- Factor multiplicativo $h = \text{BASE}^{m-1} \pmod{M}$: pondera el carácter más significativo de la ventana de longitud m .
- Desplazamiento en $O(1)$: mantener la ventana no requiere rehacer la suma polinomial completa.

Codigo clave. Desplazamiento del hash de la ventana y verificación:

```
for (int i = 0; i <= n - m; ++i) {
    if (pHash == tHash) {
        bool match = true;
        for (int j = 0; j < m; ++j)
            if (text[i + j] != pattern[j]) { match = false; break; }
        if (match) occ.push_back(i);
    }
    if (i < n - m) {
        tHash = ((tHash - text[i] * h % MOD + MOD) * BASE + text[i + m]) % MOD;
    }
}
```

Complejidad.

- **Tiempo esperado:** $O(n + m)$ (la verificación $O(m)$ se ejecuta casi exclusivamente cuando hay una coincidencia real).
- **Peor caso:** $O(n \cdot m)$ si los datos de entrada generan colisiones constantes (por ejemplo cadenas con caracteres todos iguales bajo un módulo vulnerable).
- **Memoria:** $O(1)$ adicional (más el vector de posiciones de salida).

Donde tocar para modificar.

- **Doble hash o base aleatoria:** para evitar casos maliciosos (*anti-hash test cases*), generar **BASE** aleatoriamente o usar dos módulos independientes sin verificación manual posterior.
- **Busqueda de multiples patrones de igual longitud:** almacenar los hashes de los patrones en un `unordered_set<long long>` para buscarlos todos simultaneamente en una sola pasada $O(n)$.
- **Matching en 2D (matrices):** calcular hashes por filas y luego por columnas con ventanas de tamaño fijo $R \times C$.

Trampas y errores frecuentes.

- **Negativos en el operador %:** al restar `text[i] * h`, el valor puede ser negativo. Se debe sumar `+ MOD` antes de multiplicar por `BASE` y aplicar modulo.
- **Desbordamiento de 64 bits:** la expresion `(tHash - ...) * BASE` puede desbordar si no se aplica modulo intermedio. Usar `long long` para todas las variables involucradas.
- **Caso $|P| > |T|$:** si $m > n$, el bucle no debe ejecutarse; se debe retornar vacio inmediatamente al inicio de la funcion.

Explicacion linea por linea. Funcion `rabinKarp`:

- `const long long BASE = 91138233LL, MOD = 1000000007LL;` – base prima elegida para dispersion uniforme y modulo primo grande de 32 bits.
- `if(m > n) return {};` – descarta trivialmente cuando el patron es mas largo que el texto.
- `for(int i = 0; i < m - 1; i++) h = h * BASE % MOD;` – calcula $h = \text{BASE}^{m-1} \pmod{\text{MOD}}$, el coeficiente del caracter inicial de la ventana.
- `pHash = (pHash * BASE + pattern[i]) % MOD;` – evalua el hash del patron completo.
- `tHash = (tHash * BASE + text[i]) % MOD;` – evalua el hash de la primera ventana de longitud m en el texto.
- `if(pHash == tHash)` – detecta posible coincidencia entre la ventana actual y el patron.
- `for(int j = 0; j < m; j++) if(text[i+j] != pattern[j]) ...` – verificacion caracter a caracter para garantizar ausencia de falsos positivos por colision.
- `if(match) occ.push_back(i);` – registra el indice de inicio de la ocurrencia confirmada.
- `tHash = ((tHash - text[i] * h % MOD + MOD) * BASE + text[i+m]) % MOD;` – actualiza el hash de la ventana en $O(1)$: elimina $T[i]$ a la izquierda y anade $T[i+m]$ a la derecha.

Programa principal:

- `auto occ = rabinKarp("hello world", "world");` – busca el patron “world” dentro de “hello world”.
- `cout << "found at " << pos;` – imprime la posicion encontrada (indice 6).

Codigo. Ver `cpp.tex`, seccion *Rabin-Karp (single hash)*.

4.4 Aho-Corasick (basic)

Idea intuitiva. El algoritmo de **Aho-Corasick** es una generalizacion simultanea del algoritmo KMP y de un **Trie**, disenado para buscar un conjunto de multiples patrones $\{P_1, P_2, \dots, P_k\}$ dentro de un texto T en tiempo lineal $O(|T| + \sum |P_i|)$.

Construye un trie sobre todos los patrones y establece **enlaces de fallo** (*suffix/failure links*): para cada nodo v , su enlace `link` apunta al nodo del trie que representa el sufijo propio mas largo de la cadena correspondiente a v . Durante la construccion por BFS, el trie se transforma en un autómata determinista finito (DFA) completando las transiciones faltantes hacia los enlaces de fallo (*trie graph*). Así, el recorrido por el texto consiste en una simple transicion $O(1)$ por caracter.

Propiedades clave (invariantes).

- `link[u]`: estado del trie correspondiente al sufijo propio mas largo de u .
- `out[u]`: lista de identificadores de los patrones que terminan en u o en cualquiera de sus ancestros en el arbol de fallos.
- Transiciones completas: `next[v][c]` siempre apunta al siguiente estado valido del autómata, sin requerir bucles `while` durante la busqueda.
- Calculo por niveles (BFS): los enlaces de fallo de profundidad d solo dependen de los enlaces de profundidad $< d$.

Codigo clave. Construcción de enlaces de fallo y del autómata mediante BFS:

```
while (!q.empty()) {
    int v = q.front(); q.pop();
    for (int k = 0; k < 26; ++k) {
        int u = t[v].next[k];
        if (u != -1) {
            t[u].link = t[t[v].link].next[k];
            for (int pid : t[t[u].link].out) t[u].out.push_back(pid);
            q.push(u);
        } else {
            t[v].next[k] = t[t[v].link].next[k];
        }
    }
}
```

Complejidad.

- **Construcción:** $O(\Sigma \cdot \sum |P_i|)$ donde $\Sigma = 26$ es el tamaño del alfabeto.
- **Busqueda:** $O(|T| + \text{matches})$ recorriendo exactamente un estado por cada carácter del texto.
- **Memoria:** $O(\Sigma \cdot \sum |P_i|)$ enteros para la tabla de transiciones del trie.

Donde tocar para modificar.

- **Propagación eficiente de conteos:** si la cantidad total de ocurrencias es muy grande (por ejemplo $O(|T|\sqrt{\sum |P_i|})$), no se debe copiar el vector `out` por BFS. En su lugar, se acumula la frecuencia en cada estado visitado y al final se propagan los valores en orden inverso de BFS o en el árbol de enlaces de fallo (*failure tree*).
- **Patrones con pesos o costos:** asociar a cada nodo del trie una ganancia o costo acumulado desde su enlace de fallo.
- **Alfabeto general:** cambiar el tamaño de `next` y la conversión `c - 'a'` para admitir dígitos, mayúsculas o caracteres arbitrarios.

Trampas y errores frecuentes.

- **Hijos de la raíz:** en la raíz ($v = 0$), las aristas inexistentes rebotan a 0 (`t[0].next[k] = 0`), y los hijos directos tienen enlace a la raíz (`t[u].link = 0`).
- **Explosión de memoria en out:** hacer `t[u].out.push_back` de todos los patrones de los enlaces de fallo puede consumir $O(V \cdot K)$ memoria si hay muchos patrones contenidos unos dentro de otros. Para contests con K grande, usar enlaces de salida directos (*dictionary links*) o conteos escalares.
- **Identificadores repetidos:** si se insertan patrones duplicados, el identificador debe manejarse apropiadamente (acumulando o ignorando según el problema).

Explicación línea por línea. Estructura Node:

- `int next[26]; int link; vector<int> out;` – matriz de transiciones para las 26 letras, enlace de fallo y lista de IDs de patrones que finalizan en el nodo.
- `fill(begin(next), end(next), -1); link = -1;` – inicializa todas las aristas y el enlace en -1 .

Estructura AhoCorasick:

- `AhoCorasick() { t.push_back(Node()); }` – crea el trie con el nodo 0 como raíz.
- `void addString(const string& s, int id)` – inserta el patrón en el trie asignándole el identificador `id` en el nodo final.
- `void build()` – transforma el trie en un autómata determinista calculando enlaces de fallo por BFS.

- `t[0].link = 0;` – el enlace de fallo de la raíz apunta a si misma.
- `if(t[0].next[k] != -1) { t[t[0].next[k]].link = 0; q.push(...); }` – los hijos de la raíz tienen enlace de fallo a la raíz y se encolan primero.
- `else t[0].next[k] = 0;` – las ramas inexistentes desde la raíz rebotan a la misma raíz.
- `t[u].link = t[t[v].link].next[k];` – el fallo del hijo se obtiene siguiendo la transición del carácter k desde el fallo del padre.
- `for(int pid : t[t[u].link].out) t[u].out.push_back(pid);` – propaga los patrones detectados por el sufijo común hacia el nodo actual.
- `else t[v].next[k] = t[t[v].link].next[k];` – atajo del autómata: si no hay arista real, transiciona directamente al estado correspondiente del enlace de fallo.
- `vector<int> countOccurrences(const string& text, int numPatterns)` – procesa el texto carácter por carácter avanzando por el autómata y contando coincidencias.

Programa principal:

- `ac.addString("he", 0); ac.addString("she", 1); ac.addString("his", 2);` – registra los patrones.
- `ac.build();` – construye la estructura completa del autómata.
- `ac.countOccurrences("ushers", 3);` – devuelve la cantidad de veces que cada patrón aparece en el texto.

Código. Ver `cpp.tex`, sección *Aho-Corasick (basic)*.

4.5 Aho-Corasick DP for forbidden patterns

Idea intuitiva. El problema de contar o encontrar cadenas de longitud L sobre un alfabeto Σ que **no contengan** ningún patrón de un conjunto prohibido se modela de forma natural como una programación dinámica sobre el autómata de **Aho-Corasick**.

Cada estado del autómata representa la clase de equivalencia del sufijo más largo que coincide con un prefijo de algún patrón. Un estado u es **prohibido** (terminal) si `!t[u].out.empty()`, es decir, si representa la finalización de al menos un patrón prohibido. La DP calcula:

$$dp[i][v] = \text{cantidad de cadenas de longitud } i \text{ válidas que terminan en el estado } v$$

Para construir la cadena de longitud $i + 1$, desde cada estado válido v se transiciona por cada carácter $k \in \Sigma$ hacia el estado $u = \text{next}[v][k]$. Si u no es un estado terminal, se acumula:

$$dp[i + 1][u] = \sum_{v \rightarrow_k u, u \notin \text{prohibidos}} dp[i][v] \pmod{M}$$

Propiedades clave (invariantes).

- Espacio de estados reducido: dp solo necesita el nivel previo y el nuevo (dp y ndp), usando memoria $O(|S|)$.
- Estado inicial: $dp[0] = 1$ (cadena vacía en la raíz), siempre que la raíz no contenga la cadena vacía como patrón prohibido.
- Criterio de validez: un estado u es válido si y solo si `ac.t[u].out.empty()`.
- Para L grande ($L \leq 10^{18}$) y $|S| \leq 100$, la transición es lineal y fija, pudiéndose resolver en $O(|S|^3 \log L)$ mediante **exponenciación de matrices**.

Código clave. Paso de programación dinámica con optimización de memoria:

```
for (int i = 0; i < L; ++i) {
    fill(ndp.begin(), ndp.end(), 0);
    for (int v = 0; v < S; ++v) {
        if (dp[v] == 0) continue;
        for (int k = 0; k < 26; ++k) {
            int u = ac.t[v].next[k];
```

```

        if (ac.t[u].out.empty()) {
            ndp[u] = (ndp[u] + dp[v]) % AC_MOD;
        }
    }
}
dp = ndp;
}

```

Complejidad.

- **Tiempo:** $O(L \cdot |S| \cdot |\Sigma|)$ donde $|S|$ es la cantidad de nodos del trie de Aho-Corasick y $|\Sigma| = 26$.
- **Memoria:** $O(|S|)$ utilizando solo dos vectores de estados (dp y ndp).

Donde tocar para modificar.

- **Exponenciación de matrices** ($L \leq 10^{18}$): definir una matriz de adyacencia $M[v][u]$ que cuente cuantas transiciones validas $k \in \Sigma$ llevan de v a u . Elevar M^L mediante exponenciación binaria.
- **Maximizar puntaje / Longitud mínima:** cambiar la suma modular por operaciones $(\max, +)$ o $(\min, +)$ para encontrar la cadena con máximo puntaje o menor costo.
- **Patrones obligatorios en lugar de prohibidos:** añadir una máscara de bits a los estados de la DP para rastrear que subconjunto de patrones ya ha aparecido en la cadena.

Trampas y errores frecuentes.

- **Requisito de construcción previa:** Aho-Corasick debe haberse construido completamente con `ac.build()` para que las transiciones `next[v][k]` y la propagación de salidas `out` estén completas antes de ejecutar la DP.
- **Patrón vacío prohibido:** si alguno de los patrones prohibidos es la cadena vacía, la raíz misma es terminal y la respuesta para cualquier L es 0.
- **Módulo en cada transición:** aplicar módulo en cada suma dentro del bucle interno para evitar desbordamientos aritméticos.

Explicación línea por línea. Función `countAvoiding`:

- `int S = (int)ac.t.size();` – obtiene la cantidad total de estados $|S|$ en el autómata.
- `vector<long long> dp(S, 0), ndp(S, 0);` – vectores para almacenar las cantidades de cadenas en el nivel actual y siguiente.
- `if(ac.t[0].out.empty()) dp[0] = 1;` – caso base: si la raíz no contiene patrones prohibidos, hay 1 forma de iniciar con longitud 0.
- `for(int i = 0; i < L; i++)` – itera sobre cada longitud de 0 a $L - 1$.
- `if(dp[v] == 0) continue;` – poda: omite estados inalcanzables en el nivel actual.
- `int u = ac.t[v].next[k];` – calcula el siguiente estado al añadir el carácter k .
- `if(ac.t[u].out.empty()) ndp[u] = (ndp[u] + dp[v]) % AC_MOD;` – si el estado resultante no completa ningún patrón prohibido, acumula las formas.
- `dp = ndp;` – actualiza el vector para avanzar a la siguiente longitud.
- `for(long long x : dp) total = (total + x) % AC_MOD;` – suma las formas de todos los estados válidos al alcanzar longitud L .

Programa principal:

- `ac.addString("ab", 0); ac.build();` – inserta el patrón prohibido “ab” y compila el autómata.
- `countAvoiding(ac, 3);` – calcula cadenas de longitud 3 sobre el alfabeto $[a-z]$ que no contienen la subcadena “ab” (26^3 – cadenas con “ab”).

Codigo. Ver `cpp.tex`, seccion *Aho-Corasick DP for forbidden patterns*.

4.3 Estructuras avanzadas

4.1 Suffix Array

Idea intuitiva. El **Suffix Array** (Arreglo de Sufijos) p de una cadena S de longitud n contiene los indices de inicio de todos los sufijos de S ordenados lexicograficamente:

$$S[p[0] \dots n-1] < S[p[1] \dots n-1] < \dots < S[p[n-1] \dots n-1]$$

La tecnica de **doubling** (duplicacion de prefijos) construye el arreglo en $O(n \log n)$: en la iteracion k , los sufijos estan clasificados segun sus primeros 2^k caracteres. Para pasar a 2^{k+1} , cada sufijo de longitud 2^{k+1} se interpreta como un par de clases de equivalencia de longitud 2^k :

$$\text{par}(i) = (c[i], c[(i + 2^k) \pmod n])$$

Como la segunda mitad ya esta ordenada en la permutacion previa, restar 2^k y aplicar un **counting sort** estable sobre la primera mitad ordena los pares completos en $O(n)$ por paso, repitiendo $\lceil \log_2 n \rceil$ veces.

Propiedades clave (invariantes).

- $p[i]$: indice de inicio en S del i -esimo sufijo en orden lexicografico.
- $c[i]$: clase de equivalencia (rango normalizado) del sufijo que inicia en la posicion i . Si dos sufijos comparten los primeros 2^k caracteres, tienen el mismo valor c .
- Busqueda binaria sobre el SA: cualquier subcadena P aparece como prefijo de un rango contiguo $[first, last)$ en el arreglo de sufijos. Puede localizarse en $O(|P| \log n)$.

Codigo clave. Transicion de doubling $k \rightarrow k+1$ con ordenamiento por conteo:

```
while ((1 << k) < n) {
    for (int i = 0; i < n; ++i)
        p[i] = (p[i] - (1 << k) + n) % n;
    count_sort(p, c);
    vector<int> c_new(n);
    c_new[p[0]] = 0;
    for (int i = 1; i < n; ++i) {
        pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << k)) % n]};
        pair<int, int> now = {c[p[i]], c[(p[i] + (1 << k)) % n]};
        c_new[p[i]] = c_new[p[i-1]] + (now != prev);
    }
    c = c_new;
    k++;
}
```

Complejidad.

- **Construccion:** $O(n \log n)$ en tiempo ($O(\log n)$ etapas con counting sort $O(n)$).
- **Memoria:** $O(n)$ enteros para los arreglos p , c y auxiliares.
- **Busqueda de patron:** $O(|P| \log n)$ por busqueda binaria en el SA.

Donde tocar para modificar.

- **Multiples cadenas (Generalized Suffix Array):** concatenar las cadenas separadas por caracteres delimitadores unicos y menores que cualquier caracter del alfabeto (ej. $S_1 + '$' + S_2 + '#'$).
- **Contar subcadenas distintas:** complementado con el arreglo LCP (ver seccion LCP), la cantidad de subcadenas distintas es $\frac{n(n+1)}{2} - \sum \text{LCP}[i]$.
- **k -esima subcadena lexicografica:** acumular las longitudes de los sufijos descontando los LCP adyacentes para saltar al bloque que contiene la k -esima subcadena.

Trampas y errores frecuentes.

- **Modulo circular % n**: el algoritmo trata las cadenas como ciclicas para simplificar los limites; asegurar que el caracter delimitador final (si se agrega '\$') sea lexicograficamente menor que todo el alfabeto para que los sufijos reales no se solapen con prefijos circulares.
- **Counting sort estable**: el counting sort DEBE ser estable para preservar el orden de la segunda componente obtenido con la resta modular.
- **Tamanos de alfabetos grandes**: en la etapa base ($k = 0$), ordenar con `std::sort` maneja cualquier valor ASCII sin necesidad de predimensionar un vector de frecuencias enorme.

Explicacion linea por linea. Estructura SuffArr:

- `vector<int> p, c`; – `p` es el arreglo de sufijos (posiciones); `c` son las clases de equivalencia.
- `SuffArr(string &a)` – constructor: guarda la cadena, dimensiona vectores e invoca `build()`.
- `void count_sort(vector<int> &p, vector<int> &c)` – ordena los elementos de `p` en $O(n)$ segun sus clases de equivalencia `c` usando un arreglo de frecuencias y posiciones acumuladas.
- `vector<pair<int,int>> tmp(n)`; – etapa base ($k = 0$): asocia cada caracter individual con su posicion original.
- `sort(all(tmp))`; – ordena los caracteres iniciales lexicograficamente.
- `c[p[i]] = c[p[i-1]] + (tmp[i].F != tmp[i-1].F)`; – asigna clases iniciales de 1 caracter.
- `p[i] = (p[i] - (1<<k) + n) % n`; – desplaza cada indice hacia atras por 2^k , quedando preordenados por su segunda mitad.
- `count_sort(p, c)`; – aplica ordenamiento por conteo sobre la primera mitad, completando el ordenamiento lexicografico de longitud 2^{k+1} .
- `pair<int,int> now = {c[p[i]], c[(p[i]+(1<<k))%n]}`; – extrae el par de clases que define el bloque de tamano 2^{k+1} .
- `c_new[p[i]] = c_new[p[i-1]] + (now != prev)`; – incrementa la clase de equivalencia si el par difiere del sufijo previo.

Uso en main (Pattern Matching):

- `sa.s.compare(suf, p.size(), p) < 0`; – comparador que contrasta el prefijo del sufijo contra el patron.
- `while(lo < hi) { ... }` – primera busqueda binaria para hallar el extremo inferior del rango en el SA.
- `string pNext = p + "{"`; – anade un caracter lexicograficamente mayor que todo el alfabeto para encontrar el extremo superior en una segunda busqueda binaria.
- `int last - first`; – cantidad exacta de apariciones del patron en el texto.

Codigo. Ver `cpp.tex`, seccion *Suffix Array*.

4.2 LCP Array (Kasai)

Idea intuitiva. El arreglo **LCP** (Longest Common Prefix) almacena la longitud del prefijo comun mas largo entre pares de sufijos consecutivos en el arreglo de sufijos (SA):

$$\text{LCP}[k] = \text{lcp}(S[\text{sa}[k] \dots n - 1], S[\text{sa}[k + 1] \dots n - 1])$$

El algoritmo de **Kasai et al.** calcula este arreglo en tiempo lineal $O(n)$ procesando los sufijos en el orden en que aparecen en la cadena original ($i = 0, 1, \dots, n - 1$) en lugar del orden del SA. La observacion fundamental es que al pasar del sufijo que inicia en i al sufijo que inicia en $i + 1$, simplemente se elimina el primer caracter; por ende, si el sufijo i compartia h caracteres con su predecesor en el SA, el sufijo $i + 1$ compartira al menos $h - 1$ caracteres con su correspondiente predecesor:

$$h_{i+1} \geq h_i - 1$$

Esto garantiza que la variable h se decrementa a lo sumo n veces, y por ende solo puede incrementarse a lo sumo $2n$ veces en total, resultando en tiempo $O(n)$.

Propiedades clave (invariantes).

- Arreglo inverso `rank_[i]`: posición en el SA del sufijo que inicia en i (`rank_[sa[k]] = k`).
- Tamaño del arreglo: $n - 1$ elementos para una cadena de longitud n .
- LCP de dos sufijos arbitrarios $sa[i]$ y $sa[j]$ con $i < j$: corresponde al mínimo elemento en el rango:

$$\text{LCP}(sa[i], sa[j]) = \min_{i \leq k < j} \text{lcp}[k]$$

calculable en $O(1)$ tras un precomputo RMQ con Sparse Table sobre el arreglo LCP.

Código clave. Construcción del arreglo LCP con el algoritmo de Kasai:

```
vector<int> buildLCP(const string& s, const vector<int>& sa) {
    int n = s.size(), h = 0;
    vector<int> rank_(n), lcp(max(0, n - 1), 0);
    for (int i = 0; i < n; ++i) rank_[sa[i]] = i;
    for (int i = 0; i < n; ++i) {
        if (rank_[i] > 0) {
            int j = sa[rank_[i] - 1];
            while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
            lcp[rank_[i] - 1] = h;
            if (h > 0) h--;
        }
    }
    return lcp;
}
```

Complejidad.

- **Tiempo:** $O(n)$ estricto (h se incrementa como máximo $2n$ veces y se decrementa como máximo n veces).
- **Memoria:** $O(n)$ para el arreglo de retorno y el vector auxiliar `rank_`.

Donde tocar para modificar.

- **Subcadenas distintas:** la cantidad total de subcadenas únicas en una cadena de longitud n es:

$$\text{distintas} = \frac{n(n+1)}{2} - \sum_{k=0}^{n-2} \text{lcp}[k]$$

- **Subcadena repetida más larga:** es el valor máximo del arreglo LCP, obtenido con `*max_element(all(lcp))`.
- **Subcadena común más larga de dos cadenas:** concatenar $A + '$' + B$, calcular SA y LCP, y buscar el máximo $\text{lcp}[k]$ tal que $sa[k]$ y $sa[k+1]$ pertenezcan a cadenas distintas.
- **Consultas LCP entre pares:** montar una Sparse Table sobre `lcp` para responder $\text{LCP}(u, v)$ en $O(1)$.

Trampas y errores frecuentes.

- **Sufijo lexicográficamente menor:** el sufijo con `rank_[i] == 0` no tiene predecesor en el SA; se debe omitir con `if (rank_[i] > 0)`.
- **Cadena de longitud 1 o vacía:** para $n \leq 1$, el arreglo LCP tiene tamaño 0. Usar `max(0, n - 1)` evita accesos negativos.
- **Olvido de `h--`:** omitir `if (h > 0) h--`; degrada la complejidad a $O(n^2)$.

Explicacion linea por linea. Funcion `buildLCP`:

- `vector<int> rank_(n);` – vector de permutacion inversa: ubica donde quedo cada sufijo original dentro del SA.
- `for(int i=0;i<n;i++) rank_[sa[i]] = i;` – llena la tabla de rangos: el sufijo que inicia en `sa[i]` esta en la posicion `i`.
- `vector<int> lcp(max(0, n - 1), 0);` – crea el vector para almacenar las $n - 1$ adyacencias.
- `int h = 0;` – longitud coincidente inicial garantizada.
- `for(int i = 0; i < n; i++)` – itera sobre los sufijos en orden de aparicion en el texto.
- `if(rank_[i] > 0)` – evalua unicamente si tiene un vecino inmediatamente anterior en el orden lexicografico del SA.
- `int j = sa[rank_[i] - 1];` – `j` es el indice de inicio del sufijo vecino anterior en el SA.
- `while(i + h < n && j + h < n && s[i+h] == s[j+h]) h++;` – extiende la coincidencia desde el desplazamiento `h` ya garantizado.
- `lcp[rank_[i] - 1] = h;` – guarda el LCP entre dicho vecino y el sufijo actual.
- `if(h > 0) h--;` – invariante de Kasai: el siguiente sufijo $i + 1$ compartira al menos $h - 1$ caracteres con su predecesor.

Programa principal:

- `auto lcp = buildLCP(s, sa);` – calcula las longitudes de prefijos comunes adyacentes para “banana” (resultado: 0 1 3 0 0 2).

Codigo. Ver `cpp.tex`, seccion *LCP Array (Kasai)*.

4.3 Suffix Automaton

Idea intuitiva. El **Autómata de Sufijos** (Suffix Automaton o SAM) es el autómata determinista finito (DFA) mínimo que reconoce todas las subcadenas (y en particular todos los sufijos) de una cadena S de longitud n . Posee la propiedad excepcional de requerir a lo sumo $2n - 1$ estados y a lo sumo $3n - 4$ transiciones para $n \geq 2$.

Cada estado del SAM representa una clase de equivalencia de subcadenas que comparten exactamente el mismo conjunto de posiciones finales de aparicion (**endpos**). Para cada estado v :

- Las subcadenas de la clase tienen longitudes contiguas en el intervalo $(\text{len}[\text{link}[v]], \text{len}[v]]$.
- La cantidad de subcadenas distintas representadas por el estado v es exactamente $\text{len}[v] - \text{len}[\text{link}[v]]$.
- El **suffix link** (`link[v]`) apunta al estado que contiene el sufijo propio mas largo que pertenece a un conjunto estrictamente mayor de posiciones finales. Los suffix links forman un arbol (*link tree*) correspondiente al arbol de sufijos de la cadena invertida.

Propiedades clave (invariantes).

- `len[v]`: longitud de la subcadena mas larga que llega al estado v .
- `link[v]`: enlace de sufijo hacia la clase con el conjunto *endpos* inmediatamente mas amplio.
- `occ[v]`: cantidad de veces que aparecen las subcadenas de esta clase en el texto original.
- Al anadir un caracter mediante `extend(c)`, se anade a lo sumo 1 estado original y a lo sumo 1 estado clon, preservando la cota lineal $O(n)$ en tiempo y espacio.

Codigo clave. Paso de extension incremental del autómata:

```
void extend(char c) {
    int k = c - 'a', cur = st.size(), p = last;
    st.push_back(State());
    st[cur].len = st[last].len + 1;
    st[cur].occ = 1;
```

```

while (p != -1 && st[p].next[k] == -1) {
    st[p].next[k] = cur;
    p = st[p].link;
}
if (p == -1) {
    st[cur].link = 0;
} else {
    int q = st[p].next[k];
    if (st[p].len + 1 == st[q].len) {
        st[cur].link = q;
    } else {
        int clone = st.size();
        st.push_back(st[q]);
        st[clone].len = st[p].len + 1;
        st[clone].occ = 0;
        while (p != -1 && st[p].next[k] == q) {
            st[p].next[k] = clone;
            p = st[p].link;
        }
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}

```

Complejidad.

- **Construccion:** $O(n \cdot \Sigma)$ en tiempo con transiciones en arreglo fijo de tamaño $\Sigma = 26$.
- **Memoria:** $O(n \cdot \Sigma)$ ya que hay a lo sumo $2n$ estados.
- **Consultas:** $O(|P|)$ para verificar si un patron P es subcadena y contar sus apariciones.

Donde tocar para modificar.

- **Contar subcadenas distintas:** sumar $\text{len}[v] - \text{len}[\text{link}[v]]$ para todos los estados $v > 0$.
- **Frecuencia de aparicion de un patron:** recorrer el autómata con los caracteres de P ; si se alcanza un estado v , la respuesta es $\text{occ}[v]$ (precalculada propagando por orden decreciente de longitud).
- **Subcadena comun mas larga de dos cadenas (LCS):** procesar la segunda cadena manteniendo el estado actual y la longitud coincidente, retrocediendo por `link` cuando no haya transicion.
- **k -esima subcadena lexicografica:** calcular con DP en el DAG el numero de caminos desde cada nodo y hacer un recorrido greedy.

Trampas y errores frecuentes.

- **Capacidad reservada:** dimensionar el vector de estados con `st.reserve(2 * maxLen)`; subestimar la memoria provocara realocaciones costosas.
- **Ocurrencias del clon:** el estado `clone` solo divide clases de equivalencia, por lo que inicia con `st[clone].occ = 0`. Asignarle 1 sobrecontara apariciones.
- **Orden topologico:** para propagar `occ` o resolver DP, se puede ordenar los estados por `len` decreciente (o usar counting sort en $O(V)$) en lugar de un DFS topologico general.

Explicacion linea por linea. Estructura State:

- `int next[26]; int link, len; long long occ;` – almacena las 26 transiciones, el enlace de sufijo, la longitud maxima y el conteo de apariciones del estado.

- `fill(begin(next), end(next), -1); link = -1; len = 0; occ = 0;` – inicializa el estado como nodo aislado.

Metodo extend:

- `st[cur].len = st[last].len + 1; st[cur].occ = 1;` – crea el nuevo estado para el prefijo extendido con longitud incrementada.
- `while(p != -1 && st[p].next[k] == -1) { st[p].next[k] = cur; p = st[p].link; }` – redirige todos los sufijos que no tenian transicion por c hacia el nuevo estado.
- `if(p == -1) st[cur].link = 0;` – si ningun sufijo tenia transicion por c , el enlace de sufijo apunta a la raiz (0).
- `if(st[p].len + 1 == st[q].len) st[cur].link = q;` – transicion continua: la clase de q no necesita dividirse.
- `int clone = st.size(); st.push_back(st[q]);` – transicion discontinua: clona q para separar las subcadenas que extienden a p .
- `st[clone].len = st[p].len + 1; st[clone].occ = 0;` – el clon asume longitud ajustada y conteo 0.
- `while(p != -1 && st[p].next[k] == q) ...` – redirige hacia el clon las transiciones que antes apuntaban a q .
- `st[q].link = st[cur].link = clone;` – tanto q como el nuevo estado cur enlazan al estado clonado.

Metodos build, countDistinct y longestCommonSubstring:

- `sort(order.begin(), order.end(), ...)` – ordena estados por longitud descendente para propagar ocurrencias de hijos a padres en el arbol de links.
- `ans += st[i].len - st[st[i].link].len;` – suma las longitudes de subcadenas unicas generadas por cada estado.
- `while(v != 0 && st[v].next[k] == -1) { v = st[v].link; l = st[v].len; }` – en caso de discordancia al buscar LCS, retrocede por el link ajustando la longitud coincidente.

Codigo. Ver `cpp.tex`, seccion *Suffix Automaton*.

4.4 Lyndon factorization (Duval)

Idea intuitiva. Una palabra o cadena de **Lyndon** es una cadena no vacia que es estrictamente menor lexicograficamente que todos sus sufijos propios (lo que equivale a ser estrictamente menor que todas sus rotaciones no triviales).

El **Teorema de Chen-Fox-Lyndon** establece que toda cadena S admite una **factorizacion unica** como concatenacion de palabras de Lyndon no crecientes:

$$S = w_1 w_2 \dots w_k \quad \text{con } w_1 \geq w_2 \geq \dots \geq w_k \text{ en orden lexicografico}$$

El algoritmo de **Duval** encuentra esta factorizacion en tiempo lineal $O(n)$ y memoria auxiliar $O(1)$ manteniendo tres punteros: i (inicio de la factorizacion pendiente), j (puntero de coincidencia periodica) y k (explorador hacia adelante). Mantiene la invariante de que la subcadena $S[i \dots k-1]$ es pre-Lyndon (de la forma $u^p v$ donde u es una palabra de Lyndon y v es un prefijo propio de u).

Propiedades clave (invariantes).

- Unicidad: la descomposicion en palabras de Lyndon no crecientes existe y es unica para cualquier cadena.
- La subcadena $S[i \dots k-1]$ se mantiene como $u^p v$:
 - Si $S[k] == S[j]$: el periodo se extiende; se incrementa j y k .
 - Si $S[k] > S[j]$: la periodicidad se rompe favorablemente, fusionando todo en una palabra de Lyndon mas larga; se reinicia $j = i$.
 - Si $S[k] < S[j]$ o fin de cadena ($k == n$): se extraen p copias completas de la palabra de Lyndon u de longitud $\text{len} = k - j$, avanzando i .
- Rotacion minima lexicografica: ejecutando Duval sobre $S + S$, el inicio del factor de Lyndon que sobrepasa o alcanza la posicion n senala el punto de partida de la **menor rotacion ciclica** de S en $O(n)$.

Codigo clave. Algoritmo de Duval para extraer los cortes de la factorizacion:

```
vector<int> cuts;
int i = 0;
while (i < n) {
    int j = i, k = i + 1;
    while (k < n && s[j] <= s[k]) {
        if (s[j] < s[k]) j = i;
        else j++;
        k++;
    }
    int len = k - j;
    while (i <= j) {
        cuts.push_back(i + len - 1);
        i += len;
    }
}
```

Complejidad.

- **Tiempo:** $O(n)$ estricto (cada caracter se inspecciona como maximo dos veces: una durante la extension y otra al avanzar i).
- **Memoria:** $O(1)$ auxiliar (ademas del vector de cortes de retorno).

Donde tocar para modificar.

- **Minima rotacion ciclica:** ejecutar Duval sobre $T = S + S$. El indice i al inicio del factor que cruza n corresponde al punto de partida de la rotacion minima en $O(n)$.
- **Obtener los factores explicitos:** en lugar de almacenar unicamente el indice final ($i + \text{len} - 1$), extraer la subcadena con `s.substr(i, len)`.
- **Maxima rotacion ciclica:** invertir el comparador de orden lexicografico ($s[j] \geq s[k]$).

Trampas y errores frecuentes.

- **Condicion de corte** $i \leq j$: el bucle de confirmacion de factores debe iterar mientras $i \leq j$ y avanzar $i += \text{len}$, extrayendo todos los periodos completos u antes de reanudar el analisis con el resto v .
- **Longitud del periodo:** el periodo de la palabra pre-Lyndon se calcula como $\text{len} = k - j$ (la distancia entre el explorador y el seguidor del prefijo).

Explicacion linea por linea. Funcion lyndonFactorization:

- `int n = s.size(); vector<int> cuts;` – almacena los indices finales de cada factor de Lyndon.
- `int i = 0;` – i marca el inicio del proximo sufijo aun no descompuesto.
- `int j = i, k = i + 1;` – j recorre el prefijo de referencia y k explora caracteres hacia adelante.
- `while(k < n && s[j] <= s[k])` – mientras la cadena se mantenga no decreciente periodicamente.
- `if(s[j] < s[k]) j = i;` – caracter mayor que el periodo: fusiona todo en un nuevo y mas largo factor de Lyndon, reiniciando la referencia a i .
- `else j++;` – caracter identico: extiende la periodicidad de la palabra pre-Lyndon.
- `k++;` – avanza el puntero explorador.
- `int len = k - j;` – calcula la longitud del factor irreducible u .
- `while(i <= j) { cuts.push_back(i + len - 1); i += len; }` – extrae cada repeticion completa del factor u y actualiza i .

Programa principal:

- `auto cuts = lyndonFactorization("ababa");` – para “ababa”, los factores son “ab”, “ab”, “a”, guardando los cortes en 0, 2 y 4 (o en sus extremos correspondientes).

Codigo. Ver `cpp.tex`, seccion *Lyndon factorization (Duval)*.

4.4 Palindromos

4.1 Manacher

Idea intuitiva. El algoritmo de **Manacher** calcula en tiempo estrictamente lineal $O(n)$ el radio del palindromo mas largo centrado en cada posicion de una cadena S de longitud n , distinguiendo tanto palindromos de longitud impar (centrados en un caracter i) como de longitud par (centrados entre los caracteres $i - 1$ e i).

Mantiene una ventana $[l, r]$ que representa el palindromo que alcanza la posicion mas lejana a la derecha. Al evaluar un nuevo centro i :

- Si $i \leq r$, el centro i tiene un reflejo simetrico $i' = l + r - i$ respecto al centro de la ventana. Por simetria, el radio en i es al menos $\min(\text{radio}[i'], r - i + 1)$.
- Luego se intenta expandir mas alla del limite r caracter a caracter, actualizando la ventana $[l, r]$ cuando se sobrepasa el extremo derecho previo.

Propiedades clave (invariantes).

- **Convencion del codigo:**
 - `odd[i] = k` ($k \geq 1$): palindromo impar centrado en i , abarca $S[i - k + 1 \dots i + k - 1]$ con longitud $2k - 1$. Representa tambien la cantidad de subpalindromos impares con centro en i .
 - `even[i] = k` ($k \geq 0$): palindromo par centrado entre $i - 1$ e i , abarca $S[i - k \dots i + k - 1]$ con longitud $2k$. Representa tambien la cantidad de subpalindromos pares centrados entre $i - 1$ e i .
- Consultas en $O(1)$: determinar si una subcadena $S[l \dots r]$ es un palindromo se resuelve en tiempo constante verificando si el radio registrado en su centro cubre la longitud total.

Codigo clave. Calculo de radios de palindromos impares y pares:

```
// Impares
for (int i = 0; i < n; ++i) {
    int k = (i > r) ? 1 : min(odd[l + r - i], r - i + 1);
    while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
    odd[i] = k;
    if (i + k - 1 > r) { l = i - k + 1; r = i + k - 1; }
}
// Pares
l = 0; r = -1;
for (int i = 0; i < n; ++i) {
    int k = (i > r) ? 0 : min(even[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
    even[i] = k;
    if (i + k - 1 > r) { l = i - k; r = i + k - 1; }
}
```

Complejidad.

- **Construccion:** $O(n)$ en tiempo; cada caracter extiende el extremo derecho r a lo sumo una vez.
- **Consultas:** $O(1)$ por subcadena para consultas de verificacion con `isPalindrome`.
- **Memoria:** $O(n)$ para almacenar los vectores `odd` y `even`.

Donde tocar para modificar.

- **Contar cantidad total de subpalindromos:** sumar todos los elementos de `odd` y `even` (implementado en `countPalindromes`).
- **Subcadena palindromica mas larga:** buscar el maximo entre $2 \cdot \text{odd}[i] - 1$ y $2 \cdot \text{even}[i]$ (implementado en `longestPalindrome`).
- **Palindromos en arboles o caminos:** combinable con HLD / hashing para verificar palindromos en caminos de arboles.
- **Version unificada con caracteres centinela:** intercalar '#' entre cada letra transforma todos los palindromos a impares sobre una cadena de tamano $2n + 1$.

Trampas y errores frecuentes.

- **Centro par desfasado:** para longitud par, el centro cae entre $i - 1$ e i . En `isPalindrome`, el centro se calcula como $c = (l + r + 1) / 2$, y se evalua `even[c] >= len / 2`.
- **Desfase en la formula del espejo:** para impares el espejo es $l + r - i$; para pares el espejo es $l + r - i + 1$. Mezclar ambas expresiones corrompe el algoritmo.
- **Radio vs Longitud:** recordar que la longitud es $2k - 1$ para impares y $2k$ para pares; confundir k con el diametro es un error habitual.

Explicacion linea por linea. Funcion manacher:

- `vector<int> odd(n, 0); int l = 0, r = -1;` – inicializa el vector de impares y la ventana vacia.
- `int k = (i > r) ? 1 : min(odd[l + r - i], r - i + 1);` – aprovecha el radio del nodo espejo reflejado en la ventana actual.
- `while(i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;` – expande hacia ambos lados mientras haya simetria.
- `if(i + k - 1 > r) { l = i - k + 1; r = i + k - 1; }` – avanza la ventana $[l, r]$ si el nuevo palindromo rebasa el borde derecho previo.
- `vector<int> even(n, 0); l = 0; r = -1;` – reinicia la ventana para el calculo de palindromos pares.
- `int k = (i > r) ? 0 : min(even[l + r - i + 1], r - i + 1);` – radio par garantizado por el espejo simetrico.
- `while(i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;` – expande comparando caracteres equidistantes del corte entre $i - 1$ e i .

Funciones auxiliares:

- `longestPalindrome` – recorre los centros comparando $2 \cdot \text{odd}[i] - 1$ y $2 \cdot \text{even}[i]$, retornando la subcadena de maxima longitud.
- `countPalindromes` – suma todos los radios de `odd` y `even` obteniendo el numero total de subcadenas palindromicas.
- `isPalindrome(l, r, odd, even)` – segun la paridad de la longitud $r - l + 1$, ubica el centro y verifica en $O(1)$ si el radio es suficiente para cubrir $[l, r]$.

Codigo. Ver `cpp.tex`, seccion *Manacher*.

5 Grafos

5.1 Caminos minimos

5.1 Dijkstra

Idea intuitiva. Encuentra el shortest path desde un origen s a todos los vertices en $O((V + E) \log V)$ asumiendo pesos no negativos. La idea: usar una cola de prioridad (min-heap) ordenada por distancia tentativa. En cada

paso, extraer el nodo con menor distancia; si ya fue procesado, saltar. Si al actualizar un vecino la distancia mejora, insertar/actualizar en la cola. La demostracion: cuando un nodo se extrae por primera vez, no hay forma de llegar a el con menor distancia (cualquier camino alternativo deberia pasar por nodos no procesados, con distancia $\geq$ la actual).

Propiedades clave (invariantes).

- Asume pesos ≥ 0 ; con pesos negativos usar Bellman-Ford.
- Greedy correctness: la primera vez que se extrae un nodo, su distancia es la definitiva.
- Si se quiere reconstruir el camino, guardar el `parent[]`.
- Cada nodo se procesa a lo sumo V veces (mas relajations, pero solo V definitivas).

Codigo clave. El bucle principal de Dijkstra:

```
priority_queue<pair<long long, int>, vector<...>, greater<...>> pq;
pq.push({0, source});
while (!pq.empty()) {
    auto [d, v] = pq.top(); pq.pop();
    if (d > dist[v]) continue; // entrada vieja
    for (auto [w, u] : adj[v])
        if (dist[v] + w < dist[u]) {
            dist[u] = dist[v] + w;
            pq.push({dist[u], u});
        }
}
```

Complejidad.

- $O((V + E) \log V)$ con heap binario, $O(V + E)$ con heap de Fibonacci.
- En la practica, $\sim 1.5 \cdot 10^{-7}$ segundos por operacion para $V = 10^5$.

Donde tocar para modificar.

- **Camino mas corto entre todos los pares:** si se necesita desde todos los origenes, usar Floyd o ejecutar Dijkstra V veces.
- **Shortest path con aristas negativas limitadas:** Bellman-Ford.
- **Reconstruir camino:** anadir `parent[v]` actualizado junto a `dist[v]`.
- **Dijkstra con potentials (Johnson):** para aristas negativas si se puede reescalar.
- **Multi-source:** anadir varios nodos fuentes iniciales al heap.

Trampas y errores frecuentes.

- Distancias no inicializadas a INF; el snippet usa `1e18`.
- El `if(dist[adjN] != 1e18)` para borrar de la cola es importante si usas `set` (no asi con `priority_queue`).
- Con pesos grandes, `dist[v] + w` puede overflow; usar `LLONG_MAX/2` como INF.
- No usar con pesos negativos: el algoritmo puede dar respuestas incorrectas.

Codigo. Ver `cpp.tex`, seccion *Dijkstra*.

5.2 Bellman-Ford

Idea intuitiva. Encuentra shortest path desde s incluso con **pesos negativos** en $O(VE)$. La idea: relajar todas las aristas $V - 1$ veces. Cada relajacion propaga la mejor distancia a “un paso mas”. Despues de $V - 1$ pasadas, si alguna arista aun se puede relajar, hay un **ciclo negativo** alcanzable. La demostracion: el camino mas corto tiene a lo sumo $V - 1$ aristas (sin ciclos); tras $V - 1$ relajaciones todas las distancias estan en su valor optimo.

Propiedades clave (invariantes).

- Funciona con pesos negativos (sin ciclos negativos).
- Tras $V - 1$ relajaciones, las distancias son definitivas (si no hay ciclo negativo).
- Una relajación adicional detecta ciclos negativos: si $\text{dist}[u] + w < \text{dist}[v]$, hay ciclo negativo alcanzable desde s .
- Cada relajación solo mejora distancias, así que termina (no oscila).

Código clave. El doble bucle de relajación:

```
for (int i = 0; i < n - 1; ++i)
    for (auto [u, v, w] : edges)
        if (dist[u] + w < dist[v])
            dist[v] = dist[u] + w;
// Detectar ciclo negativo
for (auto [u, v, w] : edges)
    if (dist[u] + w < dist[v]) { /* ciclo negativo */ }
```

Complejidad.

- $O(VE)$.
- SPFA (con cola) en promedio es mucho más rápido, peor caso igual.

Donde tocar para modificar.

- **SPFA**: optimización que relaja solo nodos cuya distancia cambió; en el peor caso sigue siendo $O(VE)$.
- **Detectar ciclo negativo global**: ejecutar Bellman-Ford desde un super-origen conectado a todos.
- **Camino más corto con K aristas exactas**: K iteraciones de Bellman-Ford.
- **Johnson's**: reescalar pesos con potentials para usar Dijkstra.

Trampas y errores frecuentes.

- Overflow si los pesos son $\geq 2^{52}$; usar `long long` y `__int128` si hace falta.
- Si el grafo es denso ($E \approx V^2$), $O(VE)$ es prohibitivo.
- El chequeo de ciclo negativo debe hacerse tras las $V - 1$ iteraciones, no durante.
- Distancias no inicializadas a INF; el primer nodo (source) tiene distancia 0.

Código. Ver `cpp.tex`, sección *Bellman-Ford*.

5.3 Floyd-Warshall

Idea intuitiva. Shortest path entre **todos los pares** de vértices en $O(V^3)$. DP: $d[i][j]$ = shortest path usando solo nodos intermedios en $\{0, \dots, k\}$. Transición: $d_{k+1}[i][j] = \min(d_k[i][j], d_k[i][k+1] + d_k[k+1][j])$. La demostración: tras k iteraciones, $d[i][j]$ es el shortest path con intermedios en $\{0, \dots, k-1\}$; añadir el nodo k da la recurrencia clásica.

Propiedades clave (invariantes).

- Detecta ciclos negativos: tras el algoritmo, $d[i][i] < 0$ indica ciclo negativo alcanzable desde i .
- Funciona con pesos negativos (sin ciclos negativos).
- Memoria $O(V^2)$; con $V \leq 400$ es OK, con más conviene Dijkstra V veces.
- Es la única opción cuando se quieren shortest paths entre todos los pares y el grafo es denso.

Codigo clave. El triple bucle clasico:

```
for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
// detectar ciclo negativo
for (int i = 0; i < n; ++i) if (d[i][i] < 0) tiene_ciclo_negativo = true;
```

Complejidad.

- $O(V^3)$ tiempo, $O(V^2)$ memoria.
- Para $V \leq 400$: $\sim 6.4 \cdot 10^7$ operaciones (factible).

Donde tocar para modificar.

- **Reconstruir camino:** anadir `next[i][j]` y `backtrack`.
- **Transitividad:** el algoritmo tambien calcula transitividad (`d[i][j] != INF` significa alcanzable).
- **Pesos doubles:** anadir tolerancia (`d[i][j] > d[i][k] + d[k][j] - eps`).
- **Grafo no denso:** usar Dijkstra V veces en vez de Floyd.

Trampas y errores frecuentes.

- Para detectar ciclo negativo correctamente, anadir un chequeo `if (ans[i][i] < 0)` tras el triple bucle.
- El orden de los bucles `k`, `i`, `j` es importante; invertir da resultados diferentes.
- Con `double`, usar tolerancia `< eps` en vez de `<` para evitar loops infinitos.

Codigo. Ver `cpp.tex`, seccion *Floyd-Warshall*.

5.4 0-1 BFS

Idea intuitiva. Variante de BFS para grafos con **pesos 0 o 1**. Usa un **deque**: aristas de peso 0 se encolan al frente (`push_front`), aristas de peso 1 al final (`push_back`). Asi, la primera vez que se extrae un nodo, su distancia es la definitiva, igual que en Dijkstra pero en $O(V + E)$. La intuicion: el deque mantiene los nodos ordenados por distancia; las aristas de peso 0 no cambian la distancia (entran al frente), las de peso 1 la incrementan (entran al final).

Propiedades clave (invariantes).

- Asume pesos **solo** 0 o 1.
- Mejor caso que Dijkstra (sin factor $\log V$).
- No funciona con pesos ≥ 2 .
- La cola mantiene los nodos ordenados por distancia no decreciente.

Codigo clave. El deque doble:

```
deque<int> q;
q.push_front(source);
dist[source] = 0;
while (!q.empty()) {
    int v = q.front(); q.pop_front();
    for (auto [w, u] : adj[v]) {
        if (dist[v] + w < dist[u]) {
            dist[u] = dist[v] + w;
            if (w == 0) q.push_front(u);
        }
    }
}
```

```

        else q.push_back(u);
    }
}
}

```

Complejidad.

- $O(V + E)$.
- Cada nodo se inserta a lo sumo $O(1)$ veces (con check de distancia).

Donde tocar para modificar.

- **Pesos en $[0, k]$ small:** usar small k-ary BFS o Dijkstra.
- **Reconstruir camino:** añadir `parent[]`.
- **Pesos negativos:** no funciona, usar Bellman-Ford.

Trampas y errores frecuentes.

- Si los pesos son otro rango, el algoritmo da respuestas incorrectas. Verificar que las aristas son solo 0/1.
- El chequeo `dist[v] + w < dist[u]` es crucial; sino, el algoritmo procesa nodos multiples veces.
- Si una arista tiene peso ≥ 2 , añadir verificaciones o cambiar a Dijkstra.

Codigo. Ver `cpp.tex`, seccion 0-1 BFS.

5.2 Componentes y cortes

5.1 Kosaraju SCC

Idea intuitiva. Encuentra las **componentes fuertemente conexas** (SCCs) en $O(V + E)$. Algoritmo en 3 pasos:

- DFS en G rellenando una pila en **post-orden** (los nodos que terminan tarde quedan arriba).
- Construir G^T (grafo transpuesto).
- DFS en G^T procesando nodos en orden de la pila (de mayor a menor tiempo); cada DFS da una SCC.

La demostracion: dos nodos u, v estan en la misma SCC sii hay camino $u \rightsquigarrow v$ y $v \rightsquigarrow u$; Kosaraju captura exactamente esto al explorar G^T desde los nodos con mayor tiempo de finalizacion en G .

Propiedades clave (invariantes).

- Cada SCC es maximal fuertemente conexas.
- El **grafo de SCCs** (metagraf) es un DAG.
- Kosaraju y Tarjan dan el mismo resultado; Tarjan es “mas rapido” en practica (un solo DFS).
- Cada nodo pertenece a exactamente una SCC.

Codigo clave. La estructura de Kosaraju:

```

vector<int> order;
vector<bool> used(n);
function<void(int)> dfs1 = [&](int v) {
    used[v] = true;
    for (int u : g[v]) if (!used[u]) dfs1(u);
    order.push_back(v); // post-orden
};
function<void(int, int)> dfs2 = [&](int v, int root) {
    comp[v] = root;

```

```

    for (int u : gt[v]) if (comp[u] == -1) dfs2(u, root);
};
// 1: DFS en G para llenar order
// 2: DFS en G^T en orden inverso para asignar SCCs

```

Complejidad.

- $O(V + E)$.
- Constante 2 (dos DFS + el grafo transpuesto).

Donde tocar para modificar.

- **2-SAT**: SCC sobre grafo de implicaciones.
- **Construir el DAG de SCCs**: añadir aristas entre SCCs diferentes.
- **Tamaño de cada SCC**: añadir contador durante el segundo DFS.
- **Grafo grande**: usar Tarjan para evitar construir G^T .

Trampas y errores frecuentes.

- El snippet usa `vector<...> adj[n]` (VLA); reemplazable por `vector<vector<pair<int,int>>>` para portabilidad.
- El orden del segundo DFS debe ser **inverso** al de finalización del primero.
- Si el grafo no es conexo, añadir un loop sobre todos los nodos en el primer DFS.
- Confundir SCC con componentes conexas (las primeras son para dirigidos, las segundas para no dirigidos).

Código. Ver `cpp.tex`, sección *Kosaraju SCC*.

5.2 Tarjan (puentes)

Idea intuitiva. Encuentra los **puentes** (bridges): aristas cuya remoción **desconecta** el grafo. Usa un solo DFS manteniendo `tin[v]` (tiempo de descubrimiento) y `low[v]` (el menor `tin` alcanzable desde v via back-edges). Una arista $v - u$ es puente si $low[u] > tin[v]$. La demostración: si $low[u] > tin[v]$, entonces u (y su subarbol) no pueden “escapar” de v ; quitar la arista $v - u$ parte el grafo.

Propiedades clave (invariantes).

- $low[v] = \min(tin[v], tin[w])$ para todo back-edge (v, w) , y $\min(low[u])$ para hijos en el DFS.
- Solo se consideran las aristas del **arbol DFS** (no back-edges) para `low`.
- Cada nodo tiene exactamente un `tin`; cada back-edge se procesa una vez.

Código clave. El DFS de Tarjan para bridges:

```

void dfs(int v, int pEdge) {
    used[v] = true;
    tin[v] = low[v] = timer++;
    for (auto [u, id] : adj[v]) {
        if (id == pEdge) continue; // no contar el edge al padre
        if (used[u]) {
            low[v] = min(low[v], tin[u]); // back-edge
        } else {
            dfs(u, id);
            low[v] = min(low[v], low[u]);
            if (low[u] > tin[v]) bridges.push_back(id); // es puente
        }
    }
}

```

Complejidad.

- $O(V + E)$.
- Una sola pasada de DFS.

Donde tocar para modificar.

- **Bridge tree**: contractar cada 2-edge-connected component a un nodo; el resultado es un arbol.
- **Articulation points**: variante con condicion diferente (ver el siguiente modulo).
- **Grafo no dirigido**: solo funciona en no dirigidos; en dirigidos hay conceptos analogos (strong bridges, dominator tree).
- **Reconstruir componentes 2-edge-connected**: BFS desde cada nodo excluyendo bridges.

Trampas y errores frecuentes.

- No confundir el parent con un back-edge al padre; chequear `if (adjN == parent) continue;` y contar solo las veces que se “visita” como hijo.
- Para multigrafos, los edges deben tener IDs unicos para distinguirlos.
- Si el grafo no es conexo, anadir loop externo sobre todos los nodos.

Codigo. Ver `cpp.tex`, seccion *Tarjan (puentes)*.

5.3 Tarjan (puntos de articulacion)

Idea intuitiva. Encuentra los **puntos de articulacion** (cut vertices): vertices cuya remocion **desconecta** el grafo. Mismo setup que para bridges: `tin[v]`, `low[v]`. Un nodo v (no raiz) es de articulacion si existe un hijo u con $low[u] \geq tin[v]$. La **raiz** es de articulacion si tiene ≥ 2 hijos en el DFS tree. La razon: para v no raiz, la condicion $\geq$ indica que el subarbol de u depende de v ; quitar v los desconecta. Para la raiz, perder ≥ 2 hijos parte el DFS en ≥ 2 componentes.

Propiedades clave (invariantes).

- $low[u] \geq tin[v]$ significa que el subarbol de u no tiene back-edge “por encima” de v .
- La raiz es caso especial: tiene que perder mas de un hijo para que el grafo se parta.
- Cada nodo se evalua una sola vez.

Codigo clave. La condicion de articulacion:

```
void dfs(int v, int p) {
    used[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
    for (int u : adj[v]) {
        if (u == p) continue;
        if (used[u]) low[v] = min(low[v], tin[u]);
        else {
            dfs(u, v);
            low[v] = min(low[v], low[u]);
            if (low[u] >= tin[v] && p != -1) // no raiz
                isArt[v] = true;
            ++children;
        }
    }
    if (p == -1 && children > 1) isArt[v] = true; // raiz especial
}
```

Complejidad.

- $O(V + E)$.
- Una sola pasada de DFS.

Donde tocar para modificar.

- **Block-cut tree:** estructura bipartita con BCCs (2-vertex-connected components) y articulation points.
- **Verificar biconectividad:** chequear que no hay puntos de articulacion.
- **Componentes biconnected:** extender Tarjan para enumerar BCCs.

Trampas y errores frecuentes.

- No olvidar el caso especial de la raiz.
- La condicion usa $\geq$, no $>$; un hijo con `low[u] == tin[v]` tambien cuenta.
- Multigrafos pueden tener “falsos” articulation points; verificar caso a caso.

Codigo. Ver `cpp.tex`, seccion *Tarjan (puntos de articulacion)*.

5.4 Tarjan SCC

Idea intuitiva. Variante del SCC que usa **un solo DFS** con una pila explicita. Mantiene `disc[v]`, `low[v]`, y una pila de nodos “en la SCC actual”. Cuando `low[v] == disc[v]`, v es la raiz de una SCC; popear hasta v inclusive. La demostracion: `low[v] == disc[v]` indica que v no tiene back-edges a ancestros; entonces v y los nodos en la pila hasta v forman una SCC maximal.

Propiedades clave (invariantes).

- Mas eficiente que Kosaraju (un solo DFS).
- `low[v] = min(low[u], disc[w])` para back-edges o recursion.
- La pila asegura que solo los nodos en la SCC actual se popean.
- Cada nodo se pushea y popea a lo sumo una vez.

Codigo clave. El DFS de Tarjan SCC:

```
void dfs(int v) {
    disc[v] = low[v] = timer++;
    st.push(v); inStack[v] = true;
    for (int u : g[v]) {
        if (disc[u] == -1) { dfs(u); low[v] = min(low[v], low[u]); }
        else if (inStack[u]) low[v] = min(low[v], disc[u]);
    }
    if (low[v] == disc[v]) {
        // popear hasta v
        while (true) {
            int u = st.top(); st.pop(); inStack[u] = false;
            comp[u] = current_scc;
            if (u == v) break;
        }
        ++current_scc;
    }
}
```

Complejidad.

- $O(V + E)$.
- Constante similar a Kosaraju pero sin construir G^T .

Donde tocar para modificar.

- **Tamano de cada SCC:** anadir contador durante el pop.
- **Grafo condensation:** anadir aristas entre SCCs en el DAG resultante.
- **2-SAT:** anadir la construccion del grafo de implicaciones.

Trampas y errores frecuentes.

- `inStack[u]` debe chequearse antes de usar `low[v] = min(low[v], disc[u])` en back-edges.
- La pila global es importante; sin ella no se puede extraer la SCC.
- Si el grafo es muy grande, recursion puede overflow; anadir version iterativa.

Codigo. Ver `cpp.tex`, seccion *Tarjan SCC*.

5.3 Matching

5.1 Kuhn (bipartite matching)

Idea intuitiva. Encuentra un **matching maximo** en un grafo bipartito en $O(VE)$. Para cada nodo u en L , intentar encontrar un vecino v en R que este libre o cuyo match se pueda “reorganizar” recursivamente. Si se encuentra, matchear $u - v$. La idea: si u quiere un vecino v ya ocupado, intentamos “mover” el match de v a otro nodo; recursivamente, esto encuentra un augmenting path si existe.

Propiedades clave (invariantes).

- `matchR[v]` = el nodo de L matcheado a v (o -1 si libre).
- `vis[]` se resetea **cada intento** de u .
- Si el matching es perfecto, $|L| = |R|$ = tamano del matching.
- Kuhn no garantiza maximalidad greedy; puede haber augmenting paths no encontrados.

Codigo clave. La recursion del augmenting path:

```
bool tryKuhn(int v) {
    if (vis[v]) return false;
    vis[v] = true;
    for (int u : g[v]) {
        if (matchR[u] == -1 || tryKuhn(matchR[u])) {
            matchR[u] = v;
            return true;
        }
    }
    return false;
}
// Por cada v en L:
fill(vis, vis+n, false);
if (tryKuhn(v)) ++matches;
```

Complejidad.

- $O(VE)$ en el peor caso.
- Con Hopcroft-Karp: $O(\sqrt{V} \cdot E)$, mucho mejor para grafos densos.

Donde tocar para modificar.

- **Matching bipartito minimo (min vertex cover):** Konig's theorem.
- **Matching general:** blossom algorithm (no incluido).
- **Aniadir capacidades:** si los nodos de R tienen capacidad > 1 , partir cada uno en capacidad copias.
- **Output del matching:** el array `matchR` da los pares finales.

Trampas y errores frecuentes.

- Olvidar resetear `vis` por intento; eso es lo que evita “ciclos infinitos” en la recursion.
- Para grafos grandes ($V > 10^4$), Kuhn puede ser muy lento; usar Hopcroft-Karp.
- Si el matching es maximo pero no perfecto, $|L|$ o $|R|$ puede ser mayor; el matching es solo maximo.

Codigo. Ver `cpp.tex`, seccion *Kuhn (bipartite matching)*.

5.2 Hopcroft-Karp

Idea intuitiva. Matching bipartito maximo en $O(\sqrt{V}E)$. Algoritmo en **fases**: en cada fase, hacer BFS para encontrar los **caminos de aumento mas cortos** desde nodos libres de L ; luego DFS solo por esos caminos. Cada fase aumenta el matching en ≥ 1 y toma $O(E)$. La demostracion: tras una fase, todos los augmenting paths tienen longitud $\geq$ la nueva minima; eso sucede cada $\sqrt{V}$ fases (longitud minima crece geometricamente).

Propiedades clave (invariantes).

- `dist[u]` = distancia en niveles desde un nodo libre de L .
- BFS termina cuando no hay mas camino a un nodo libre de R .
- DFS respeta los niveles (solo sigue `dist[matchV[v]] == dist[u] + 1`).
- Cada fase aumenta el matching en ≥ 1 arista.

Codigo clave. El BFS de Hopcroft-Karp:

```
bool bfs() {
    queue<int> q;
    for (int v : L) {
        if (matchU[v] == -1) { dist[v] = 0; q.push(v); }
        else dist[v] = -1;
    }
    bool found = false;
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : g[v]) {
            if (matchV[u] != -1 && dist[matchV[u]] == -1) {
                dist[matchV[u]] = dist[v] + 1;
                q.push(matchV[u]);
            }
            if (matchV[u] == -1) found = true; // llegamos a libre
        }
    }
    return found;
}
```

Complejidad.

- $O(\sqrt{V}E)$.
- Para $V = 10^4$, $E = 10^5$: $\sim 3 \cdot 10^7$ operaciones.

Donde tocar para modificar.

- **Reconstruir el matching:** ya viene en `matchU/matchV`.
- **Aniadir pesos:** Hungarian algorithm.
- **Densidad alta:** si $E \approx V^2$, sigue siendo util pero verificar constantes.
- **Aniadir capacidades:** partir los nodos con capacidad > 1 en varias copias.

Trampas y errores frecuentes.

- `dist[matchV[v]] == dist[u] + 1` (no `==` a secas); sin esto, el DFS no respeta los niveles y se pierde la complejidad.
- El BFS debe marcar los niveles correctamente; resetear `dist` para nodos alcanzables solo via matching.
- El DFS debe respetar los niveles; cualquier “atajo” invalida la cota.

Codigo. Ver `cpp.tex`, seccion *Hopcroft-Karp*.

5.4 MST

5.1 Kruskal

Idea intuitiva. Encuentra el **arbol de expansion minima** (MST) en $O(E \log E)$. Ordena las aristas por peso ascendente y las ania al MST si no forman ciclo (chequear con DSU). El arbol tiene exactamente $V - 1$ aristas. La demostracion (“Cut Property”): para cualquier corte, la arista minima cruzando el corte pertenece a *algun* MST; por lo tanto, elegir la minima disponible que no forma ciclo es optimo.

Propiedades clave (invariantes).

- Algoritmo **greedy** correcto (Cut Property).
- Funciona en grafos **no conexos**; produce un **minimum spanning forest**.
- Si las aristas tienen pesos unicos, el MST es unico.
- El MST tiene exactamente $V - 1$ aristas (o menos si el grafo es desconexo).

Codigo clave. Kruskal con DSU:

```
sort(edges.begin(), edges.end());
DSU dsu(n);
long long mst = 0;
int edgesUsed = 0;
for (auto [w, u, v] : edges)
    if (dsu.unite(u, v)) {
        mst += w;
        if (++edgesUsed == n - 1) break;
    }
```

Complejidad.

- $O(E \log E)$ (ordenamiento) o $O(E \log V)$ con DSU optimizations.
- DSU es casi $O(1)$ amortizado, asi que el sort domina.

Donde tocar para modificar.

- **Second best MST:** enumerar aristas del MST, sacar una, aniadir una no-MST; recalcular.
- **Kruskal randomizado:** si el grafo es casi-completo, agregar shuffle.
- **Aniadir info al DSU:** guardar el peso maximo en el camino, etc.
- **DSU con rollback:** si necesitas Kruskal reversible (para queries offline).

Trampas y errores frecuentes.

- El DSU debe estar limpio para cada invocacion.
- Si las aristas tienen pesos negativos, Kruskal sigue funcionando.
- Si el grafo es desconexo, el “MST” es un bosque con varias componentes.
- El check `if (++edgesUsed == n - 1) break;` optimiza la salida temprana.

Codigo. Ver `cpp.tex`, seccion *Kruskal*.

5.2 Prim (priority queue)

Idea intuitiva. Otra forma de calcular el MST: empezar desde un nodo y, en cada paso, anadir la arista de menor peso que conecta el conjunto construido con el resto. Usando priority queue, es $O((V + E) \log V)$. La demostracion: cada vez que se anade una arista, es la minima cruzando el “corte” entre el conjunto actual y el resto (Cut Property); eso preserva optimalidad.

Propiedades clave (invariantes).

- Similar a Dijkstra pero sin acumular distancias, solo eligiendo minimo por arista.
- Funciona mejor que Kruskal en **grafos densos** ($E \approx V^2$).
- Si el grafo es desconexo, genera el bosque (solo llega a los alcanzables).
- Cada nodo se inserta al heap una vez; cada arista se procesa una vez.

Codigo clave. Prim clasico con heap:

```
priority_queue<pair<int, int>, vector<...>, greater<...>> pq;
vector<bool> used(n, false);
vector<int> minEdge(n, INF);
minEdge[0] = 0;
pq.push({0, 0});
while (!pq.empty()) {
    auto [w, v] = pq.top(); pq.pop();
    if (used[v]) continue;
    used[v] = true;
    mst += w;
    for (auto [peso, u] : adj[v])
        if (!used[u] && peso < minEdge[u]) {
            minEdge[u] = peso;
            pq.push({peso, u});
        }
}
```

Complejidad.

- $O((V + E) \log V)$.
- Con matriz de adyacencia: $O(V^2)$ sin heap.

Donde tocar para modificar.

- **Prim con matriz de adyacencia:** $O(V^2)$ sin heap, util en grafos densos pequenos.
- **Reconstruir el MST:** anadir `parent[]`.
- **Prim en streaming:** variante para grafos que llegan incrementalmente.

Trampas y errores frecuentes.

- El `if (used[v]) continue;` es crucial: si el nodo ya fue procesado, saltar.
- En la version matriz, elije el minimo por linea (no heap).
- Si el grafo es desconexo, anadir un loop externo sobre todos los nodos no usados.

Codigo. Ver `cpp.tex`, seccion *Prim (priority queue)*.

5.5 Basicos

5.1 DFS / BFS / componentes

Idea intuitiva. **DFS** (Depth-First Search) y **BFS** (Breadth-First Search) son los dos recorridos basicos de grafos. DFS usa una pila (o recursion) y explora tan profundo como puede antes de retroceder; BFS usa una cola y explora por “niveles” de distancia. La cantidad de **componentes conexas** se cuenta iniciando un BFS/DFS desde cada nodo no visitado. La diferencia clave: BFS da el camino mas corto en aristas; DFS da un orden de finalizacion util para problemas topologicos.

Propiedades clave (invariantes).

- BFS da la **distancia en aristas** mas corta desde el origen.
- DFS da un **orden topologico** (si se anaden al final).
- Ambos son $O(V + E)$.
- Un grafo con C componentes conexas satisface $\sum \text{subtree_size}_v = V$ y $\sum E_v = 2E$.
- DFS en arbol da un spanning tree; las “back edges” corresponden a ciclos.

Codigo clave. BFS clasico:

```
queue<int> q;
q.push(source);
dist[source] = 0;
while (!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v])
        if (dist[u] == -1) {
            dist[u] = dist[v] + 1;
            q.push(u);
        }
}
```

Complejidad.

- $O(V + E)$ tiempo, $O(V)$ memoria.
- Cada arista se procesa exactamente 2 veces (ida y vuelta) en no dirigidos.

Donde tocar para modificar.

- **Aniadir pesos:** si las aristas tienen peso y quieres shortest path, BFS ya no sirve; usar Dijkstra.
- **Grafo dirigido:** en DFS, anadir `check if (state[u] == 1)` para detectar ciclos.
- **Bipartite check:** BFS alternando colores (ver el siguiente modulo).
- **Componentes fuertemente conexas (dirigidos):** usar Tarjan o Kosaraju.
- **BFS en matriz:** anadir offsets por direccion para grid 2D.

Trampas y errores frecuentes.

- Stack overflow con DFS recursivo para $V > 10^5$; usar version iterativa con pila explicita.
- En BFS, olvidar marcar `dist[u] = dist[v] + 1` antes de pushear da visitas duplicadas.
- Para grafos dirigidos, “componente conexas” no esta definida; usar SCC.

Codigo. Ver `cpp.tex`, seccion *DFS / BFS / componentes*.

5.2 Topological sort (Kahn + DFS)

Idea intuitiva. Un **orden topologico** de un DAG es una permutacion de los vertices tal que toda arista $u \rightarrow v$ tiene u antes que v . Dos algoritmos clasicos:

- **Kahn:** BFS sobre nodos con `indeg == 0`; al “consumir” un nodo, decrementar el indeg de sus vecinos; los que lleguen a 0 se encolan.
- **DFS:** hacer DFS; al terminar un nodo (estado = 2), aniadirlo al final; el orden es el reverso.

La demostracion: en un DAG siempre hay al menos un nodo con indeg 0; quitarlo mantiene el DAG; por induccion, se obtiene el orden.

Propiedades clave (invariantes).

- Existe sii el grafo es un DAG.
- Si al final `order.size() < V`, hay un ciclo.
- Kahn da el orden “natural” (los que pueden ir primero); DFS da el orden “al revés” (los que pueden ir al final primero).
- La cantidad de ordenes topologicos puede ser exponencial.

Codigo clave. Kahn con priority queue (orden lexicografico):

```
priority_queue<int, vector<int>, greater<int>> pq;
for (int v = 0; v < n; ++v) if (indeg[v] == 0) pq.push(v);
while (!pq.empty()) {
    int v = pq.top(); pq.pop();
    order.push_back(v);
    for (int u : adj[v]) {
        if (--indeg[u] == 0) pq.push(u);
    }
}
```

Complejidad.

- $O(V + E)$ ambos.
- Con priority queue, $O((V + E) \log V)$.

Donde tocar para modificar.

- **Detectar ciclos:** comparar `order.size()` con V .
- **Orden lexicografico minimo:** usar priority queue en Kahn.
- **Topological sort con pesos:** combinar con shortest path en DAG (un solo pase en orden topologico).
- **Encontrar todos los ordenes:** DP sobre subsets (solo para V pequeno).

Trampas y errores frecuentes.

- En el snippet, Kahn retorna orden vacío si hay ciclo (útil para detectar).
- En DFS, no olvidar `reverse(order)`.
- Confundir `indeg` con `outdeg`; el orden Kahn quita primero los nodos “fuente”.
- Para grafos no DAG, el algoritmo termina con `order.size() < V`.

Código. Ver `cpp.tex`, sección *Topological sort (Kahn + DFS)*.

5.3 Bipartite check + 2-coloring

Idea intuitiva. Un grafo es **bipartito** si sus vértices se pueden dividir en dos conjuntos L y R tales que toda arista va de L a R . Esto equivale a decir que se puede **2-colorear** sin que dos vértices adyacentes tengan el mismo color. BFS alternando colores detecta esto: si en algún momento dos adyacentes tienen el mismo color, no es bipartito. La demostración: por cada componente, fijar el color de un nodo determina todos los demás; si en algún paso hay conflicto, hay un ciclo impar (no bipartito).

Propiedades clave (invariantes).

- Grafo bipartito $\Leftrightarrow$ sin ciclos impares.
- Conexo: si un componente no es bipartito, todo el grafo no lo es.
- Si hay **múltiple componentes**, cada uno se colorea independientemente.
- La 2-coloración es única salvo swap de colores.

Código clave. BFS con 2-coloreo:

```
vector<int> color(n, -1);
for (int v = 0; v < n; ++v) if (color[v] == -1) {
    color[v] = 0;
    queue<int> q; q.push(v);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int w : adj[u]) {
            if (color[w] == -1) { color[w] = 1 - color[u]; q.push(w); }
            else if (color[w] == color[u]) return false;
        }
    }
}
return true;
```

Complejidad.

- $O(V + E)$.
- Memoria $O(V)$.

Donde tocar para modificar.

- **Colorear desde múltiples fuentes:** BFS desde cada nodo no coloreado.
- **Bipartito + matching:** Hopcroft-Karp.
- **Grafo bipartito ponderado:** Hungarian algorithm.
- **Verificar bipartito en subgrafo:** solo aplicar BFS a los nodos relevantes.

Trampas y errores frecuentes.

- Verificar **todos** los componentes, no solo el primero.
- Para grafos con auto-loops, un auto-loop hace al grafo no bipartito.
- En multigrafos, las aristas multiples no afectan el chequeo de bipartito.

Codigo. Ver `cpp.tex`, seccion *Bipartite check + 2-coloring*.

5.6 Flujos

5.1 Edmonds-Karp (max flow)

Idea intuitiva. Implementacion BFS del algoritmo de **Ford-Fulkerson**. En cada iteracion, hacer BFS desde s hasta t siguiendo aristas con capacidad residual > 0 ; si se llega a t , se encontro un **camino de aumento**. Enviar el minimo de las capacidades en el camino; actualizar capacidades residuales. La demostracion de $O(VE^2)$: cada BFS encuentra el augmenting path mas corto, y la longitud minima crece monotona; como hay $\leq E$ longitudes posibles, hay $\leq E$ fases.

Propiedades clave (invariantes).

- BFS garantiza caminos de aumento mas cortos.
- Capacidad residual: forward edge baja, backward edge sube.
- Complejidad $O(VE^2)$.
- El flujo maximo respeta la conservacion en nodos ($\text{in} = \text{out}$ salvo s, t).

Codigo clave. BFS + augmenting path:

```
while (bfs(s, t, parent)) { // bfs encuentra camino o retorna false
    int pathFlow = INF;
    for (int v = t; v != s; v = parent[v])
        pathFlow = min(pathFlow, capacity[parent[v]][v]);
    for (int v = t; v != s; v = parent[v]) {
        capacity[parent[v]][v] -= pathFlow;
        capacity[v][parent[v]] += pathFlow;
    }
    maxFlow += pathFlow;
}
```

Complejidad.

- $O(VE^2)$.
- En la practica, mas lento que Dinic para grafos grandes.

Donde tocar para modificar.

- **Grafo bipartito:** $O(\sqrt{V}E)$ (equivalente a Hopcroft-Karp).
- **Dinic** es preferible en la mayoria de los casos.
- **Capacidades doubles:** usar tolerancia para comparacion.
- **Min-cost flow:** anadir costo por arista y modificar el algoritmo.

Trampas y errores frecuentes.

- Las capacidades deben ser > 0 para entrar a la cola (no ≥ 0); sino, ciclos infinitos.
- Para grafos bipartitos, Dinic o Hopcroft-Karp son mejores.
- Si el flujo maximo es muy grande, los `int` pueden overflow; usar `long long`.

Codigo. Ver `cpp.tex`, seccion *Edmonds-Karp (max flow)*.

5.2 Dinic (max flow)

Idea intuitiva. Algoritmo de flujo mas eficiente que Edmonds-Karp: combina **BFS para level graph** con **DFS bloqueante** (multiple augmenting paths en una sola pasada). Cada fase cuesta $O(E)$ y hay $O(V)$ fases en el peor caso; para bipartitos, $O(\sqrt{V}E)$. La razon: cada fase “satura” al menos una arista, asi que en $O(E)$ fases el flujo maximo se alcanza; con $O(V)$ niveles en el BFS, hay $O(VE)$ operaciones totales.

Propiedades clave (invariantes).

- `level[v]` = distancia desde s en el level graph (solo aristas con `cap > 0`).
- `it[v]` = iterador actual para evitar reprocesar aristas.
- DFS solo sigue aristas con `level[e.to] == level[v] + 1`.
- Back-edges tienen `rev` apuntando a la forward edge original.

Codigo clave. El DFS bloqueante de Dinic:

```
long long dfs(int v, int t, long long f) {
    if (v == t) return f;
    for (int& i = it[v]; i < adj[v].size(); ++i) {
        Edge& e = adj[v][i];
        if (e.cap > 0 && level[e.to] == level[v] + 1) {
            long long ret = dfs(e.to, t, min(f, e.cap));
            if (ret > 0) {
                e.cap -= ret;
                adj[e.to][e.rev].cap += ret;
                return ret;
            }
        }
    }
    return 0;
}
```

Complejidad.

- $O(V^2E)$ general, $O(\sqrt{V}E)$ para bipartitos.
- En la practica, mucho mas rapido que Edmonds-Karp.

Donde tocar para modificar.

- **Dinic con scaling:** anadir factor de escala para capacidades grandes.
- **Lower bounds:** anadir un super-source/sink para forzar flujo minimo.
- **Matching bipartito:** el grafo se construye como source $-i$ L $-i$ R $-i$ sink, todos con capacidad 1.
- **Push-relabel:** alternativa para grafos muy densos.

Trampas y errores frecuentes.

- `it[v]` debe **incrementarse** en cada iteracion; sino, el DFS se queda atascado en la misma arista.
- Si el BFS no encuentra t , el flujo es maximo; terminar.
- Las back-edges son cruciales para “cancelar” elecciones previas.

Codigo. Ver `cpp.tex`, seccion *Dinic (max flow)*.

5.3 Min-Cost Max-Flow

Idea intuitiva. Encuentra el flujo maximo con **costo minimo** (asumiendo costos no negativos). Usa **SPFA** (o Dijkstra con potentials) para encontrar el shortest path en el **costo** desde s hasta t considerando capacidades residuales. Cada augmenting path aporta su costo $\times$ flujo. La idea: los potentials reescalan las aristas para que todas tengan costo no negativo, permitiendo usar Dijkstra.

Propiedades clave (invariantes).

- $\text{pot}[]$ = potentials para evitar aristas negativas (Johnson's trick).
- $\text{dist}[v]$ = costo minimo desde s con potentials.
- Solo se aumentan aristas con $\text{cap} > 0$ (forward) y costo es $+\text{cost}$ (forward) o $-\text{cost}$ (backward).
- Tras cada augmenting, los potentials se actualizan para mantener la consistencia.

Codigo clave. El ciclo de min-cost flow:

```
while (spfa(s, t, dist, parent)) {
    int pathFlow = INF;
    for (int v = t; v != s; v = parent[v])
        pathFlow = min(pathFlow, capacity[parent[v]][v]);
    for (int v = t; v != s; v = parent[v]) {
        capacity[parent[v]][v] -= pathFlow;
        capacity[v][parent[v]] += pathFlow;
        cost += pathFlow * edge_cost[parent[v]][v];
    }
}
```

Complejidad.

- $O(F \cdot VE)$ con SPFA, $O(F \cdot E \log V)$ con Dijkstra + potentials.
- F = flujo maximo total.

Donde tocar para modificar.

- **Dijkstra + potentials:** si los costos son enteros no negativos, esto es mas estable.
- **Costos negativos en aristas:** requieren Bellman-Ford inicial o reorden.
- **Flujo minimo con costo:** encontrar el minimo flujo posible y su costo.
- **Costo por unidad:** si los costos son muy grandes, anadir verificacion.

Trampas y errores frecuentes.

- El $\text{pot}[]$ se actualiza **al final** de cada iteracion (no durante).
- Las back-edges tienen **costo negativo**, esencial para corregir elecciones previas.
- Si los costos son muy grandes, considerar modular arithmetic.
- Para problemas con flujo exacto, anadir verificacion post-algoritmo.

Codigo. Ver `cpp.tex`, seccion *Min-Cost Max-Flow*.

5.7 Especiales

5.1 2-SAT (implication graph + SCC)

Idea intuitiva. Resuelve formulas booleanas en **CNF** con clausulas de **2 literales**: $(l_1 \vee l_2)$. Cada variable x se representa con dos nodos: x (verdadero) y $\neg x$ (falso). Cada clausula $l_1 \vee l_2$ se descompone en dos implicaciones: $\neg l_1 \rightarrow l_2$ y $\neg l_2 \rightarrow l_1$. Si en el grafo de implicaciones $\neg x$ y x estan en la misma SCC, la formula es **UNSAT**. La demostracion: si $\neg x \rightarrow x$ en el grafo, entonces $\neg x$ implica x , asi que $\neg x$ debe ser falso y x verdadero; pero entonces $\neg x$ es falso implica una contradiccion.

Propiedades clave (invariantes).

- Cada literal es un nodo; total $2n$ nodos.
- Kosaraju/Tarjan dan las SCCs.
- Si SAT: asignar $x =$ (topological order del condensation).
- Clausulas `setTrue(x)` se modelan como `implies(!x, x)`.
- La formula es SAT sii no hay variable x con x y $\neg x$ en la misma SCC.

Codigo clave. Conversion de clausulas a implicaciones:

```
// Clausula (a OR b) -> implica ~a -> b Y ~b -> a
addImplication(not(a), b);
addImplication(not(b), a);
// Forzar x: implica ~x -> x
if (force) addImplication(not(x), x);
```

Complejidad.

- $O(V + E) = O(N + M)$ con Kosaraju o Tarjan.
- Memoria $O(V + E)$.

Donde tocar para modificar.

- **Forzar variables:** anadir `setTrue` / `setFalse`.
- **3-SAT:** NP-completo; no usar este algoritmo.
- **Contar soluciones:** las SCCs condensation dan una formula para contar; multiplicar las formas de asignar variables en cada componente no-trivial.
- **Output de la asignacion:** anadir el vector `assignment[]`.

Trampas y errores frecuentes.

- La asignacion usa el **orden topologico inverso** del condensation: `val[i] = comp[2i] < comp[2i+1]`.
- El ID de $\neg x$ es $2*x + 1$ (o similar); verificar consistencia.
- Si hay clausulas con literales repetidos (e.g., $(x \vee x)$), tratar como (x) simple.

Codigo. Ver `cpp.tex`, seccion *2-SAT (implication graph + SCC)*.

5.2 Euler tour / Hierholzer (camino y circuito)

Idea intuitiva. Encuentra un **camino o circuito euleriano** (que usa cada arista exactamente una vez). **Condiciones:**

- **No dirigido:** todos los grados pares (circuito) o exactamente 2 impares (camino, empieza en uno, termina en el otro). Ademas, conexo sobre el subgrafo con aristas.
- **Dirigido:** $\text{indeg}(v) = \text{outdeg}(v)$ para todos (circuito) o exactamente un nodo con $\text{outdeg} = \text{indeg} + 1$ y otro con $\text{indeg} = \text{outdeg} + 1$ (camino).

Hierholzer : elegir un ciclo desde el inicio, mientras haya ciclos “laterales” desde un nodo del camino, mergearlos. Implementacion con pila: avanzar por aristas, al no poder mas anadir a la respuesta y backtrack. La demostracion de $O(V + E)$: cada arista se procesa una vez (entra y sale del stack una vez).

Propiedades clave (invariantes).

- Multigrafos permitidos; las aristas se marcan como usadas.
- No dirigido: cada arista aparece **dos veces** en `adj` (ida y vuelta).
- La construccion del path con ids consistentes es importante.
- El resultado es unico salvo orden de las aristas en multigrafos.

Codigo clave. Hierholzer con pila:

```
vector<int> path;
stack<int> st;
st.push(start);
while (!st.empty()) {
    int v = st.top();
    while (!adj[v].empty() && used[adj[v].back()]) adj[v].pop_back();
    if (adj[v].empty()) {
        path.push_back(v);
        st.pop();
    } else {
        int e = adj[v].back();
        used[e] = true;
        int u = edges[e].to(v);
        adj[v].pop_back();
        st.push(u);
    }
}
reverse(path.begin(), path.end());
```

Complejidad.

- $O(V + E)$.
- Memoria $O(V + E)$.

Donde tocar para modificar.

- **Reconstruir con multigrafo:** aniadir ids consistentes antes del algoritmo.
- **Aplicar a “de Bruijn” o problemas de strings:** modelar como multigrafo y buscar Euler.
- **Dirigido vs no dirigido:** el snippet tiene ambas versiones.
- **Letter addition:** modelar palabras como aristas y buscar el superstring.

Trampas y errores frecuentes.

- Si el grafo no es conexo (sobre las aristas), no existe euleriano. Verificar primero.
- Para multigrafos, los IDs de aristas son cruciales para distinguirlas.
- Si el problema es “encuentra todos los euler tours”, el algoritmo es exponencial.

Codigo. Ver `cpp.tex`, seccion *Euler tour / Hierholzer (camino y circuito)*.

5.3 Hamiltonian path / cycle (bitmask DP + backtracking)

Idea intuitiva. Un **camino hamiltoniano** visita cada vertice **exactamente una vez**. **Determinar** si existe es NP-completo en general, pero con $V \leq 20$ se puede hacer DP con bitmask en $O(V^2 \cdot 2^V)$. **Teoremas suficientes:** **Dirac** (grado $\geq n/2$) y **Ore** ($\deg(u) + \deg(v) \geq n$ para todo par no adyacente). La idea de la DP: cada estado es (conjunto de visitados, ultimo vertice); la transicion es agregar un vecino no visitado.

Propiedades clave (invariantes).

- DP: $dp[mask][v] = \text{true}$ si hay un camino que visita exactamente `mask` y termina en v .
- TSP: variante con pesos minimos (Hamilton cycle minimo).
- Reconstruccion: guardar `par[mask][v]` (predecesor).
- 2^V estados con V bits cada uno; factible para $V \leq 20$.

Codigo clave. DP con bitmask:

```
dp[1][0] = 0; // empezar en 0
for (int mask = 1; mask < (1<<n); ++mask)
    for (int v = 0; v < n; ++v) if (dp[mask][v] < INF)
        for (int u = 0; u < n; ++u)
            if (!(mask & (1<<u)))
                dp[mask | (1<<u)][u] = min(dp[mask | (1<<u)][u],
                                             dp[mask][v] + w[v][u]);
// TSP: respuesta = min(dp[(1<<n)-1][v] + w[v][0])
```

Complejidad.

- $O(V^2 \cdot 2^V)$ tiempo, $O(V \cdot 2^V)$ memoria.
- Para $V = 20$: $\sim 4 \cdot 10^8$ operaciones (lento pero factible).

Donde tocar para modificar.

- **TSP asimetrico**: anadir pesos $w[v][u]$ (no necesariamente simetricos).
- **Hamilton con inicio obligatorio**: fijar `src` en el DP inicial.
- **Bitmask DP mas compacto**: usar `long long` como mascara si $V \leq 64$.
- **TSP optimo aproximado**: MST, 2-opt, simulated annealing.
- **Reconstruir el path**: usar `par[mask][v]` y backtrack.

Trampas y errores frecuentes.

- TSP asume grafo **completo** (con `INF` para aristas faltantes); si no, anadir chequeo.
- La condicion "`mask == (1<<n) - 1`" cierra el ciclo retornando al origen.
- Para $V > 20$, la memoria 2^V se vuelve prohibitiva; usar meet-in-the-middle o heurísticas.

Codigo. Ver `cpp.tex`, seccion *Hamiltonian path / cycle (bitmask DP + backtracking)*.

6 DP

6.1 Knapsack 0/1

Idea intuitiva. Dado un conjunto de N objetos con peso w_i y valor v_i , maximizar $\sum v_i$ con la restriccion $\sum w_i \leq W$. La recurrencia clasica:

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w_i] + v_i) \quad \text{si } j \geq w_i.$$

La optimizacion clave: la DP es 1-dimensional si iteramos j **de mayor a menor** (asi no usamos el objeto i dos veces). La razon de la iteracion inversa: en cada iteracion queremos el estado del "item anterior"; iterar de menor a mayor contaminaria `dp[j - w_i]` con el item actual.

Propiedades clave (invariantes).

- $dp[j]$ al final = maximo valor con peso $\leq j$.
- El bucle inverso `for (j = W; j >= w[i]; j--)` es lo que distingue 0/1 de unbounded.
- Solucion: maximo valor con cualquier peso $\leq W$.
- La DP es exacta; no hay aproximacion.

Codigo clave. La version 1D optimizada:

```
vector<long long> dp(W + 1, 0);
for (int i = 0; i < N; ++i)
    for (int j = W; j >= w[i]; --j) // invertido
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
return dp[W];
```

Complejidad.

- $O(N \cdot W)$ tiempo, $O(W)$ memoria.
- Constante muy pequena (solo sumas y max).

Donde tocar para modificar.

- **Reconstruir la seleccion:** anadir `take[i][j]` (booleano o bool) en una version 2D.
- **Items con peso 0:** edge case; anadir guard.
- **Knapsack con restricciones de grupos:** “elegir a lo sumo 1 de cada grupo” se modela como 0/1 con un peso extra.
- **Tamano de W:** si $W \leq 10^9$ pero N es chico, usar meet-in-the-middle.
- **Cambiar max por min:** anadir INF en la inicializacion.

Trampas y errores frecuentes.

- Overflow en `dp[j - w[i]] + v[i]`; usar `long long` si los valores son grandes.
- El bucle **debe** ir de mayor a menor.
- Si los pesos son grandes y N pequeno, conviene meet-in-the-middle.
- Para Knapsack exacto ($\sum w_i = W$), anadir check al final.

Codigo. Ver `cpp.tex`, seccion *Knapsack 0/1*.

6.2 Knapsack unbounded

Idea intuitiva. Igual que Knapsack 0/1 pero **cada item se puede tomar multiples veces**. La recurrencia es similar:

$$dp[j] = \max(dp[j], dp[j - w_i] + v_i),$$

pero ahora el bucle va de **menor a mayor** (porque tomar el item i no afecta corridas futuras del mismo item). La idea: tras tomar el item i , queremos volver a tener la opcion de tomarlo; iterar de menor a mayor permite que el item “se acumule”.

Propiedades clave (invariantes).

- El bucle `for (j = w[i]; j <= W; j++)` es lo que distingue unbounded.
- A diferencia de 0/1, no hay riesgo de “usar dos veces el mismo item”.
- La DP es exacta; cada “subconjunto” se cuenta correctamente.

Codigo clave. La version 1D con bucle directo:

```
vector<long long> dp(W + 1, 0);
for (int i = 0; i < N; ++i)
    for (int j = w[i]; j <= W; ++j) // directo (no invertido)
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
return dp[W];
```

Complejidad.

- $O(N \cdot W)$ tiempo, $O(W)$ memoria.
- Constante muy pequena.

Donde tocar para modificar.

- **Reconstruir cuantos de cada item:** anadir `cnt[i]` y `backtrack`.
- **Cambiar moneda minima por maxima:** el snippet maximiza valor; para minimizar monedas, cambiar a `min` con inicializacion en `INF`.
- **Coin change:** si todos los pesos son positivos y queremos “minimo numero de monedas”, es unbounded con `min`.

Trampas y errores frecuentes.

- Misma que Knapsack 0/1 con overflow.
- El orden de iteracion de items no afecta el resultado (convexo).
- Si se quiere “maximo valor con exactamente k items”, anadir segunda dimension.

Codigo. Ver `cpp.tex`, seccion *Knapsack unbounded*.

6.3 Knapsack meet-in-the-middle

Idea intuitiva. Para $N \leq 40$ y W moderado, no se puede hacer $O(NW)$ directamente. Partir en dos mitades, enumerar todos los $2^{N/2}$ subconjuntos de cada mitad, y combinar. Para cada subconjunto de la segunda mitad con peso sw_2 , encontrar el maximo valor de la primera mitad con peso $\leq W - sw_2$. La idea: “divide y venceras” sobre el tamano de la entrada; cada mitad tiene mitad del espacio, asi que la combinacion es geometrica en lugar de multiplicativa.

Propiedades clave (invariantes).

- Hay $2^{N/2}$ subconjuntos por mitad; total $O(2^{N/2})$.
- `sums` guarda los valores de subconjuntos de la primera mitad ordenados (o preprocesados).
- Para cada subconjunto de la segunda mitad, busqueda lineal o binaria en `sums`.
- La mitad mas grande (con $N/2 + 1$ items para N impar) tiene $2^{N/2+1}$ subconjuntos.

Codigo clave. Enumerar subconjuntos de la primera mitad:

```
int n1 = N / 2, n2 = N - n1;
vector<pair<long long, long long>> sums1;
for (int mask = 0; mask < (1 << n1); ++mask) {
    long long peso = 0, valor = 0;
    for (int i = 0; i < n1; ++i) if (mask & (1 << i)) {
        peso += w[i]; valor += v[i];
    }
    sums1.push_back({peso, valor});
}
sort(sums1.begin(), sums1.end()); // por peso
// eliminar duplicados dominados
```

Complejidad.

- $O(2^{N/2})$ tiempo y memoria (mejor que $O(NW)$ cuando $W > 2^{N/2}$).
- Para $N = 40$: $\sim 2 \cdot 10^6$ subconjuntos, factible.

Donde tocar para modificar.

- **Tamano mayor**: si N es mayor pero W es chico, conviene iterar por peso, no por subconjuntos.
- **Varias mochilas**: adaptar para multiples W .
- **Optimizar el “combinado”**: usar `upper_bound` para encontrar el maximo valor con peso $\leq$ rem.
- **Eliminar pares dominados**: si un par (peso, valor) tiene otro con menos peso y mas valor, eliminarlo.

Trampas y errores frecuentes.

- N impar: una mitad tiene $N/2 + 1$ items.
- Overflow al sumar pesos/valores en `long long`.
- Sin deduplicacion, el algoritmo es correcto pero mas lento.

Codigo. Ver `cpp.tex`, seccion *Knapsack meet-in-the-middle*.

6.4 LIS $O(n \log n)$

Idea intuitiva. La **Longest Increasing Subsequence** se calcula en $O(n \log n)$ con el truco del **patience sorting**: mantener un arreglo `dp` donde `dp[len]` es el minimo ultimo valor de una IS de largo `len`. Para cada x , encontrar el primer `dp[len] >= x` y reemplazarlo. El largo final es la LIS. La demostracion: si hay dos IS del mismo largo, la que termina en un valor menor es “mejor” (puede extenderse mas facilmente).

Propiedades clave (invariantes).

- `dp` es estrictamente creciente.
- `lower_bound` da LIS estricta; `upper_bound` da no-estricta ($\leq$).
- El algoritmo es **online**: procesa elementos en orden.
- La longitud de `dp` es la LIS al final.

Codigo clave. La version online:

```
vector<int> dp;
for (int x : a) {
    auto it = lower_bound(dp.begin(), dp.end(), x);
    if (it == dp.end()) dp.push_back(x);
    else *it = x;
}
// dp.size() = longitud de LIS estricta
```

Complejidad.

- $O(n \log n)$.
- Memoria $O(n)$.

Donde tocar para modificar.

- **Reconstruir la LIS**: anadir `parent[]` y `posInDp[]` para backtrack.
- **Contar LIS**: DP clasica $O(n^2)$ o segment tree sobre `dp`.
- **LDS / Longest non-increasing**: invertir la condicion en `lower_bound` o multiplicar por -1.
- **Longest bitonic subsequence**: combinar LIS desde izq y desde der.
- **LIS en 2D**: ordenar por una dimension y aplicar LIS en la otra.

Trampas y errores frecuentes.

- `lower_bound` vs `upper_bound`: si tu comparacion es $\leq$ vs $<$, cambia.
- Si los elementos son `int` pero podrian ser negativos, aniadir offset para usar como indices.
- Para LIS reconstruida, aniadir el array `parent` que apunta al anterior en la IS.

Codigo. Ver `cpp.tex`, seccion *LIS* $O(n \log n)$.

6.5 LCS / Edit distance

Idea intuitiva. **LCS** (Longest Common Subsequence): para dos strings a, b , $dp[i][j]$ = LCS de $a[0..i]$ y $b[0..j]$.

Recurrencia:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{si } a[i-1] = b[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{si no} \end{cases}$$

Edit distance (Levenshtein): numero minimo de operaciones (insert, delete, replace) para convertir a en b . Recurrencia similar con un +1 adicional. La idea: “alinear” los strings eligiendo matches/mismatches/insert/delete.

Propiedades clave (invariantes).

- LCS y Edit Distance son DP $O(nm)$.
- Memoria se puede reducir a $O(\min(n, m))$ con rolling array.
- Edit distance con peso por operacion distinto: ajustar el +1 por el peso.
- La subestructura optima: la respuesta para prefijos se construye de prefijos menores.

Codigo clave. La recurrencia clasica de LCS:

```
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j)
        if (a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
        else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
return dp[n][m];
```

Complejidad.

- $O(NM)$ tiempo y memoria.
- Para $N, M \leq 5000$: $\sim 2.5 \cdot 10^7$ operaciones.

Donde tocar para modificar.

- **Reconstruir la LCS / las operaciones**: aniadir `prev[i][j]` para backtrack.
- **LCS de varios strings**: NP-hard para ≥ 3 strings; en la practica se hace con bitmask.
- **Edit distance con costos distintos**: cambiar el +1 por el costo apropiado.
- **Rolling array**: si memoria es problema, mantener solo 2 filas.
- **Hirschberg**: algoritmo $O(NM)$ tiempo, $O(\min(N, M))$ memoria, con reconstruccion.

Trampas y errores frecuentes.

- Indices en la recurrencia: `a[i-1]` y `b[j-1]` (no `a[i]`).
- Para reconstruccion, el caso `a[i-1] == b[j-1]` tiene prioridad.
- Si los strings son muy largos, considerar Hirschberg o bitset optimization.

Codigo. Ver `cpp.tex`, seccion *LCS / Edit distance*.

6.6 Bitmask DP over subsets

Idea intuitiva. Cuando el estado es un **subconjunto** de $\{0, \dots, N - 1\}$, se representa como una mascara de N bits. Para iterar sobre todos los subconjuntos de **mask**:

$$\text{for (sub = mask; sub; sub = (sub - 1) \& mask)\{\dots\}}$$

Para iterar sobre todos los superconjuntos de **mask** en $[0, U)$:

$$\text{for (sup = mask; sup \leq U; sup = (sup + 1) \& mask)\{\dots\}}$$

La idea: las mascaras forman un lattice de subconjuntos; los trucos aritmeticos $((\text{sub}-1) \& \text{mask})$ y $(\text{sup}+1) \& \text{mask}$ enumeran todo el lattice eficientemente.

Propiedades clave (invariantes).

- Iterar subsets: $O(2^k)$ para una mascara con k bits.
- $\text{sub} \& \text{mask} == \text{sub}$ siempre; $\text{mask} - \text{sub}$ no siempre da un subconjunto.
- `__builtin_popcount(mask)` cuenta los bits.
- $(\text{sub} - 1) \& \text{mask}$ itera subsets de **mask** en orden decreciente de inclusion.

Codigo clave. Iteracion sobre subsets de una mascara:

```
// Todos los subsets de mask
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // procesar sub
}
// Incluir sub = 0 si es necesario
sub = 0; // procesar primero
for (int sup = mask; sup < (1 << N); sup = (sup + 1) & mask) {
    // procesar sup (todos los superconjuntos)
}
```

Complejidad.

- $O(N \cdot 2^N)$ para una DP estandar sobre todos los subsets.
- Iterar subsets de **mask**: $O(2^{|\text{mask}|})$.

Donde tocar para modificar.

- **TSP / Hamiltonian path**: bitmask DP (ver tambien *Hamiltonian path* en `cpp.tex` seccion Grafos).
- **Set cover**: iterar subsets.
- **DP con supermask**: si el estado incluye “todos los elementos que **no** hemos usado”, usar supermask iteration.
- **Bitmasks grandes**: con $N > 20$, anadir compresion o simulacion con segmentos.

Trampas y errores frecuentes.

- El orden de iteracion de subsets de **mask** es decreciente (de **mask** a 0).
- $\text{mask} = 0$ no tiene subsets no triviales; el bucle lo saltea correctamente.
- El truco $(\text{sub} - 1) \& \text{mask}$ requiere cuidado con $\text{sub} = 0$ (termina el bucle).
- Superconjuntos: $\text{sup} < (1 \ll N)$ debe estar bien definido.

Codigo. Ver `cpp.tex`, seccion *Bitmask DP over subsets*.

6.7 SOS DP

Idea intuitiva. Para calcular, para cada `mask`, la suma de $f(\text{submask})$ sobre todos los $\text{submask} \subseteq \text{mask}$:

$$dp[\text{mask}] = \sum_{\text{sub} \subseteq \text{mask}} f(\text{sub}).$$

Se computa en $O(N \cdot 2^N)$ mediante un loop por bit: para cada i , sumar a $dp[\text{mask}]$ la contribucion de $dp[\text{mask} \oplus (1 \ll i)]$ cuando mask tiene el bit i . La idea: cada subconjunto $s \subseteq m$ se obtiene quitando bits uno a uno; así, sumar incrementalmente captura todos los subconjuntos en $O(N \cdot 2^N)$.

Propiedades clave (invariantes).

- Inicializar $dp = f$.
- Para cada bit i : `if (mask & (1 << i)) dp[mask] += dp[mask ^ (1 << i)]`.
- Variantes: supersets DP (sumar sobre superconjuntos).
- El orden de iteracion (bit externo, mascara interno) es crucial.

Codigo clave. SOS DP (Sum over Subsets):

```
vector<long long> dp(1 << N);
for (int i = 0; i < (1 << N); ++i) dp[i] = f[i];
for (int bit = 0; bit < N; ++bit)
    for (int mask = 0; mask < (1 << N); ++mask)
        if (mask & (1 << bit))
            dp[mask] += dp[mask ^ (1 << bit)];
// dp[mask] = suma de f(sub) para sub ⊆ mask
```

Complejidad.

- $O(N \cdot 2^N)$.
- Memoria $O(2^N)$.

Donde tocar para modificar.

- **Max/min en lugar de suma:** cambiar `+=` por la operacion correspondiente.
- **Supersets DP:** iterar `mask` de 0 a 2^N y añadir `dp[mask] += dp[mask | (1<<i)]`.
- **XOR convolution** (subset convolution): version mas avanzada.
- **Aplicaciones:** contar subconjuntos con propiedades especificas.

Trampas y errores frecuentes.

- Si N es 30 o mas, 2^N no entra en memoria; usar trucos o reducir N .
- El sentido del XOR (sumar a `mask ^ bit`) es para “agregar el bit i al subconjunto”.
- En supersets DP, la condicion es `(mask & (1<<bit)) == 0`.

Codigo. Ver `cpp.tex`, seccion *SOS DP*.

6.8 Digit DP template

Idea intuitiva. Contar numeros en $[0, N]$ que cumplen una propiedad sobre sus digitos. La idea: DP por posicion, manteniendo:

- **pos:** posicion actual (de izquierda a derecha).
- **tight:** si los digitos ya colocados son exactamente los de N (limite superior estricto).
- **state:** estado adicional (digitos usados, suma, etc.).

La clave: la condicion **tight** permite acotar correctamente el siguiente digito; sin ella, la DP contaria incorrectamente.

Propiedades clave (invariantes).

- Si `tight` es true, el siguiente dígito está acotado por el de N ; si no, puede ser 0-9.
- Memoria: $O(D \cdot 2 \cdot \text{states})$.
- Caso base: si `pos == D`, devolver 1 si el estado cumple la propiedad, sino 0.
- El bit `started` distingue “numero con menos dígitos” (con leading zeros).

Código clave. La estructura básica:

```
long long dp(int pos, bool tight, State s) {
    if (pos == D) return check(s) ? 1 : 0;
    if (memo[pos][tight][s] != -1) return memo;
    int limit = tight ? digits[pos] : 9;
    long long ans = 0;
    for (int d = 0; d <= limit; ++d)
        ans += dp(pos + 1, tight && (d == limit), update(s, d, pos));
    return memo[pos][tight][s] = ans;
}
```

Complejidad.

- $O(D \cdot 10 \cdot \text{states})$ donde D es el número de dígitos.
- Para $D \leq 19$ (hasta 10^{18}): $\sim 200 \cdot \text{states}$.

Donde tocar para modificar.

- **Cambiar la propiedad:** modificar `updateState` y la condición base.
- **Múltiples estados:** añadir más dimensiones al `memo`.
- **Leading zeros:** añadir un flag `started` para distinguir números con menos dígitos.
- **Rango $[L, R]$:** calcular $f(R) - f(L - 1)$.

Trampas y errores frecuentes.

- Olvidar resetear `memo` entre queries.
- Si la cantidad de estados es grande, el `memo[pos][tight][state]` puede ser muy grande; usar `map`.
- La condición `started` es crucial para números con menos dígitos que D .
- Si N es muy grande ($> 10^{18}$), añadir soporte para `long long`.

Código. Ver `cpp.tex`, sección *Digit DP template*.

6.9 Tree DP (reroot)

Idea intuitiva. Calcular alguna propiedad asumiendo que **cada nodo es la raíz** del árbol. La técnica clásica de **rerooting** en dos pasadas:

- **DFS 1:** desde una raíz arbitraria, calcular `dp[v]` para el subárbol de v .
- **DFS 2:** para cada hijo u de v , calcular `up[u]` = respuesta si la raíz fuera u .

La demostración: tras la primera DFS, cada nodo conoce “lo que aporta su subárbol”; en la segunda, se transmite “lo que aporta el resto del árbol”.

Propiedades clave (invariantes).

- $dp[v]$ depende solo del subarbol de v (con la raiz original).
- $up[u]$ se calcula de manera que cuando la raiz es u , “todo lo que estaba en v se ajusta”.
- Formula para reroot depende del problema; el snippet usa el ejemplo “suma de distancias”.
- La combinacion $dp[u] + up[u]$ da la respuesta con raiz u .

Codigo clave. Reroot para suma de distancias:

```
// dfs1: dp[v] = suma de distancias desde v al subarbol
void dfs1(int v, int p) {
    dp[v] = 0;
    for (int u : adj[v]) if (u != p) {
        dfs1(u, v);
        dp[v] += dp[u] + sz[u]; // cada hijo aporta su subarbol
    }
}
// dfs2: up[u] = distancia desde el resto del arbol a u
void dfs2(int v, int p) {
    for (int u : adj[v]) if (u != p) {
        up[u] = up[v] + dp[v] - dp[u] - sz[u] + (n - sz[u]);
        dfs2(u, v);
    }
}
```

Complejidad.

- $O(N)$ para ambas pasadas.
- Cada arista se procesa un numero constante de veces.

Donde tocar para modificar.

- **Cambiar la cantidad a calcular:** modificar la operacion en `dfs1` y la formula en `dfs2`.
- **Arbol con pesos:** anadir `weights[v]` y ajustar.
- **Re-root en arboles con ciclos:** requiere otra tecnica (DSU on tree).
- **Rooted trees diferentes:** si la propiedad no es invariante por raiz, usar otra tecnica.

Trampas y errores frecuentes.

- La formula de $up[u]$ debe sumar y restar correctamente las contribuciones de u y del resto.
- Para arboles con pesos, anadir el peso en las formulas.
- Verificar con ejemplos pequenos antes de generalizar.

Codigo. Ver `cpp.tex`, seccion *Tree DP (reroot)*.

7 Geometria

7.1 Puntos y segmentos

7.1 Point + cross + dot

Idea intuitiva. La estructura `Point` representa un punto en 2D con coordenadas enteras. Las dos operaciones vectoriales clave son:

- **Cross product** $P \times Q = P_x Q_y - P_y Q_x$. Da el “area con signo” del paralelogramo formado por P y Q . Usado para orientacion.

- **Dot product** $P \cdot Q = P_x Q_x + P_y Q_y$. Da el producto de las longitudes por el coseno del angulo. Usado para proyeccion.

La interpretacion geometrica del cross product: es el area con signo del paralelogramo; el signo indica la orientacion relativa.

Propiedades clave (invariantes).

- `cross(a, b) > 0` si b esta a la izquierda de a (sistema de coordenadas usual).
- `cross(a, b) = 0` si son colineales.
- `dot(a, b) > 0` si el angulo es agudo.
- El snippet define `cross` respecto a `this`, util para orientacion de 3 puntos.
- $|cross|^2 + |dot|^2 = |P|^2 \cdot |Q|^2$ (identidad de Lagrange).

Codigo clave. Las dos operaciones vectoriales:

```
long long cross(const Point& a, const Point& b) {
    return (long long)a.x * b.y - (long long)a.y * b.x;
}
long long dot(const Point& a, const Point& b) {
    return (long long)a.x * b.x + (long long)a.y * b.y;
}
```

Complejidad.

- $O(1)$ por operacion.
- Constante muy pequena (4 multiplicaciones + 2 sumas).

Donde tocar para modificar.

- **Coordenadas doubles:** cambiar `int` por `double` o `long double`. Cuidado con la precision.
- **3D:** aniadir `z` y adaptar `cross/dot`.
- **Operadores adicionales:** aniadir `operator+`, `operator-`, `operator*` (escalar).
- **Distancia al cuadrado:** aniadir `len2()` que retorna $x^2 + y^2$.

Trampas y errores frecuentes.

- Cross product **no es** conmutativo; $P \times Q = -Q \times P$.
- Overflow con coordenadas grandes ($|x|, |y| > 10^4$) si se hace `x * y`.
- El signo de cross depende del sistema de coordenadas (en computacion, usualmente CCW positivo).
- Para “igualdad” de doubles, aniadir tolerancia `eps`.

Codigo. Ver `cpp.tex`, seccion *Point + cross + dot*.

7.2 Interseccion de segmentos

Idea intuitiva. Dados dos segmentos $[a, b]$ y $[c, d]$, determinar si se intersectan. La idea clasica:

- Verificar orientaciones: a, b, c y a, b, d deben estar en lados opuestos de la recta ab ; igual para cd y ab .
- Caso colineal: si las rectas son paralelas (`cross = 0`) y los segmentos son colineales, verificar que los bounding boxes se intersectan.

La demostracion: dos segmentos se intersectan sii cada segmento “cruza” la linea que contiene al otro (estricto) o ambos son colineales con solapamiento.

Propiedades clave (invariantes).

- Funciona con segmentos en cualquier orientacion.
- Caso colineal: requiere check adicional de bounding box.
- Para intersectar en un **punto** especifico: anadir division con cuidado de precision.
- Funciona con segmentos verticales/horizontales sin casos especiales.

Codigo clave. Test clasico de interseccion:

```
int sign(long long x) { return (x > 0) - (x < 0); }
bool intersect(Point a, Point b, Point c, Point d) {
    long long o1 = cross(b - a, c - a);
    long long o2 = cross(b - a, d - a);
    long long o3 = cross(d - c, a - c);
    long long o4 = cross(d - c, b - c);
    if (sign(o1) != sign(o2) && sign(o3) != sign(o4)) return true;
    // casos colineales
    if (o1 == 0 && onSegment(a, b, c)) return true;
    if (o2 == 0 && onSegment(a, b, d)) return true;
    if (o3 == 0 && onSegment(c, d, a)) return true;
    if (o4 == 0 && onSegment(c, d, b)) return true;
    return false;
}
```

Complejidad.

- $O(1)$.
- Constante pequena.

Donde tocar para modificar.

- **Interseccion con punto:** anadir la division despues de verificar orientacion.
- **Interseccion rayo-segmento:** ajustar la primera condicion.
- **Poligonos convexos:** usar separacion de ejes (SAT).
- **Multiples queries:** precomputar bounding boxes o usar segment tree.

Trampas y errores frecuentes.

- Si los segmentos tocan solo en un extremo, el snippet lo considera interseccion (algunos problemas quieren estricto). Ajustar segun el problema.
- El caso colineal es comun; olvidar el check adicional de bounding box da falsos negativos.
- En problemas con muchos segmentos, considerar sweep line para $O(n \log n)$.

Codigo. Ver `cpp.tex`, seccion *Interseccion de segmentos*.

7.3 Contains colineal

Idea intuitiva. Verifica si un punto c esta sobre el segmento $[a, b]$ (asumiendo que ya son colineales). Si $\text{cross}(a, b, c) = 0$ y c esta dentro del bounding box de $[a, b]$, entonces c esta en el segmento. La razon: si los tres puntos son colineales, c esta en el segmento sii sus coordenadas estan entre las de a y b .

Propiedades clave (invariantes).

- Asume colinealidad (no la verifica mas alla de `cross == 0`).
- Usa comparaciones con `<=` para extremos **inclusivos**.
- Funciona con segmentos verticales (la condicion se reduce a una sola coordenada).

Codigo clave. Test de contencion en segmento (asume colinealidad):

```
bool onSegment(Point a, Point b, Point c) {
    return min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x) &&
        min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y);
}
```

Complejidad.

- $O(1)$.
- Constante minima.

Donde tocar para modificar.

- **Contener estricto:** cambiar $\leq$ a $<$ para excluir extremos.
- **Coordenadas doubles:** usar tolerancia (*eps*).
- **Solo x o solo y:** aniadir check especifico para segmentos verticales/horizontales.

Trampas y errores frecuentes.

- No usar este snippet para puntos **no colineales**; siempre verificar primero la colinealidad.
- Si los puntos tienen doubles, el chequeo `cross == 0` puede fallar; aniadir tolerancia.
- El bounding box incluye los extremos; verificar si el problema quiere exclusivo o inclusivo.

Codigo. Ver `cpp.tex`, seccion *Contains colineal*.

7.4 Distance point-segment / point-line

Idea intuitiva. Distancia minima de un punto p a un segmento $[a, b]$ o a la recta que pasa por a y b .

- **Linea:** $d = \frac{|ab \times ap|}{|ab|}$.
- **Segmento:** si la proyeccion de p sobre ab cae fuera del segmento, la distancia es al extremo mas cercano.

La razon geometrica: la distancia a una linea es el area del paralelogramo dividido por la base; la proyeccion ortogonal indica si estamos “dentro” o “fuera” del segmento.

Propiedades clave (invariantes).

- `dist2PointLine` retorna distancia **al cuadrado**.
- `dist2PointSegment` distingue 3 casos: p antes de a , despues de b , o proyectado sobre el segmento.
- Si `len2 = 0` (degenerate), la distancia es $|ap|$ o $|bp|$.
- La proyeccion se calcula con dot products y el parametro $t \in [0, 1]$.

Codigo clave. Distancia al cuadrado a segmento:

```
long long dist2Segment(Point p, Point a, Point b) {
    long long len2 = (a-b).len2();
    if (len2 == 0) return (p-a).len2(); // degenerate
    long long t = max(0LL, min(1LL, dot(p-a, b-a) / len2));
    Point proj = {a.x + t*(b.x - a.x), a.y + t*(b.y - a.y)};
    return (p - proj).len2();
}
```

Complejidad.

- $O(1)$.
- Constante minima.

Donde tocar para modificar.

- **Distancia (no al cuadrado):** añadir `sqrt` al final.
- **Coordenadas doubles:** añadir `long double` y tolerancia.
- **3D:** añadir coordenada z y adaptar.
- **Multiples queries:** usar KD-tree o segment tree para acelerar.

Trampas y errores frecuentes.

- Overflow en `area2 * area2` si el area es grande; `long long` alcanza para $|x|, |y| \leq 10^9$ pero no mas.
- Si $a = b$ (segmento degenerado), añadir check explicito.
- En doubles, añadir tolerancia `eps` para comparar.

Codigo. Ver `cpp.tex`, seccion *Distance point-segment / point-line*.

7.5 Projection of point onto line

Idea intuitiva. Proyeccion ortogonal de p sobre la recta que pasa por a y b . El parametro t tal que la proyeccion es $a + t \cdot (b - a)$ es:

$$t = \frac{(p - a) \cdot (b - a)}{|b - a|^2}.$$

La idea: descomponemos $\vec{ap}$ en una componente paralela a $\vec{ab}$ y una perpendicular; t mide la posicion en la primera.

Propiedades clave (invariantes).

- Si $t \in [0, 1]$, la proyeccion cae sobre el **segmento**; si no, cae sobre la extension.
- Retorna `double` (puede tener precision issues).
- $t < 0$: p esta “antes” de a ; $t > 1$: p esta “despues” de b .

Codigo clave. Proyeccion ortogonal:

```
Point projection(Point p, Point a, Point b) {
    Point ab = b - a, ap = p - a;
    double t = (double)dot(ap, ab) / ab.len2();
    return {a.x + t * ab.x, a.y + t * ab.y};
}
```

Complejidad.

- $O(1)$.
- Constante minima (dos dot products y una division).

Donde tocar para modificar.

- **Proyeccion sobre segmento:** clipar t a $[0, 1]$ antes de calcular el punto.
- **Distancia al segmento via proyeccion:** combinar con el modulo anterior.
- **Solo el parametro t :** si solo necesitas saber si esta dentro del segmento, retorna t sin construir el punto.

Trampas y errores frecuentes.

- Si $a = b$ ($|ab| = 0$), la proyeccion no esta definida; añadir guard.
- En doubles, añadir tolerancia `eps` para comparar t con 0 o 1.
- La division por `len2` puede ser cara si las coordenadas son grandes; usar `double` o precomputar.

Codigo. Ver `cpp.tex`, seccion *Projection of point onto line*.

7.2 Poligonos

7.1 Convex Hull (monotono)

Idea intuitiva. Algoritmo de **Andrew's monotone chain**: ordenar puntos por (x, y) , luego construir la hull inferior y superior por separado. Para cada punto, aniadirlo al hull y, mientras el ultimo giro no sea "counter-clockwise" (`cross > 0`), poppear. La demostracion: si los puntos se procesan en orden, el hull inferior y superior cubren la frontera completa sin auto-intersecciones.

Propiedades clave (invariantes).

- Ordena los puntos en $O(n \log n)$.
- Construye la hull en $O(n)$ despues.
- Output: poligono convexo en counter-clockwise, sin punto final duplicado.
- El ≤ 0 incluye colineales; usar < 0 para excluirlos.
- Cada punto aparece en la hull inferior o superior (no ambos).

Codigo clave. Construcccion del hull inferior y superior:

```
sort(p.begin(), p.end());
vector<Point> lower, upper;
for (auto& p : points) {
    while (lower.size() >= 2 && cross(lower.back() - lower[lower.size()-2],
                                     p - lower.back()) <= 0)
        lower.pop_back();
    lower.push_back(p);
}
for (int i = points.size() - 1; i >= 0; --i) {
    auto& p = points[i];
    while (upper.size() >= 2 && cross(upper.back() - upper[upper.size()-2],
                                     p - upper.back()) <= 0)
        upper.pop_back();
    upper.push_back(p);
}
vector<Point> hull = lower;
hull.insert(hull.end(), upper.begin() + 1, upper.end() - 1);
```

Complejidad.

- $O(n \log n)$.
- Constante muy pequena (solo cross products).

Donde tocar para modificar.

- **Colineales**: cambiar ≤ 0 por < 0 para excluirlos del hull.
- **Puntos repetidos**: `unique` despues de sort.
- **Hull 3D**: gift wrapping es $O(n^2)$; convex hull 3D es mas complejo.
- **Output con los puntos**: aniadir el hull completo (no solo upper/lower).

Trampas y errores frecuentes.

- El `pop_back` y el `reverse` son esenciales para no duplicar el ultimo punto.
- Para excluir colineales, ajustar el ≤ 0 vs < 0 .
- Si los puntos estan duplicados, aniadir `unique` antes del sort.

Codigo. Ver `cpp.tex`, seccion *Convex Hull (monotono)*.

7.2 Polygon Area

Idea intuitiva. El area (con signo) de un poligono se calcula con la **Shoelace formula**: $A = \frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i)$, donde el indice es modulo n . El snippet retorna el **doble** del area (en valor absoluto). La formula sale del “sumar areas de trapezoides”: cada lado del poligono define un trapezoide con el eje x ; la suma con signo da el area.

Propiedades clave (invariantes).

- El signo indica la orientacion: positivo si counter-clockwise.
- Para el doble: no dividir por 2 si no es necesario (evita fracciones).
- Funciona para poligonos simples (no auto-intersectantes).
- Si el poligono es no convexo, la formula da el area “con signo”.

Codigo clave. La suma de trapezoides:

```
long long shoelace(const vector<Point>& p) {
    long long s = 0;
    int n = p.size();
    for (int i = 0; i < n; ++i)
        s += (long long)p[i].x * p[(i+1)%n].y - (long long)p[(i+1)%n].x * p[i].y;
    return abs(s); // doble del area
}
```

Complejidad.

- $O(n)$.
- Constante minima.

Donde tocar para modificar.

- **Area exacta:** dividir por 2 (con cuidado si da fraccion; usar `double`).
- **Area firmada:** quitar `abs` y dejar el signo.
- **Punto dentro de poligono convexo:** con la hull, chequear que esta a un mismo lado de todas las aristas.
- **Triangulacion:** descomponer en triangulos y sumar areas.

Trampas y errores frecuentes.

- Si los puntos no estan en orden (CCW o CW), el area firmada puede ser incorrecta. Para signed area consistente, usar siempre el mismo orden.
- Para poligonos con holes, sumar las areas con signo apropiado.
- Con doubles, aniadir tolerancia `eps`.

Codigo. Ver `cpp.tex`, seccion *Polygon Area*.

7.3 Point in polygon (ray casting)

Idea intuitiva. Algoritmo de **ray casting**: contar cuantas veces un rayo horizontal desde p cruza los bordes del poligono. Si es impar, p esta dentro; si par, fuera. El snippet implementa la version clasica recorriendo aristas. La razon: cada cruce cambia el lado del rayo respecto al poligono; un numero impar de cambios significa “terminamos dentro”.

Propiedades clave (invariantes).

- Funciona con poligonos simples (no necesariamente convexos).
- Las intersecciones en vertices cuentan especialmente.
- Para poligonos con **huecos**, sigue funcionando.
- La respuesta es estrictamente 0 (fuera) o 1 (dentro) para poligonos simples.

Codigo clave. El test clasico de ray casting:

```
bool pointInPolygon(Point p, const vector<Point>& poly) {
    int n = poly.size();
    bool inside = false;
    for (int i = 0, j = n - 1; i < n; j = i++) {
        if ((poly[i].y > p.y) != (poly[j].y > p.y) &&
            p.x < (poly[j].x - poly[i].x) * (p.y - poly[i].y) /
                (poly[j].y - poly[i].y) + poly[i].x)
            inside = !inside;
    }
    return inside;
}
```

Complejidad.

- $O(n)$ por query.
- Memoria $O(1)$ (no se almacena el rayo).

Donde tocar para modificar.

- **Winding number:** version alternativa que cuenta el numero de vueltas.
- **Poligono convexo:** check lineal contra las aristas (mas rapido).
- **Multiples queries:** precomputar o usar segment tree sobre las aristas.
- **Punto en frontera:** anadir check adicional si es colineal con alguna arista.

Trampas y errores frecuentes.

- El manejo del vertice (cuando el rayo pasa por un vertice) puede causar double-counting; el snippet lo evita con la condicion $(poly[i].y > p.y) \neq (poly[j].y > p.y)$.
- Si el poligono tiene holes, el algoritmo sigue funcionando.
- Para poligonos no simples (auto-intersectantes), el resultado puede ser ambiguo.

Codigo. Ver `cpp.tex`, seccion *Point in polygon (ray casting)*.

7.4 Minkowski sum (convex polygons)

Idea intuitiva. La **suma de Minkowski** $A \oplus B$ de dos poligonos convexos es otro poligono convexo que representa $\{a + b : a \in A, b \in B\}$. Algoritmo: ordenar las aristas por angulo polar (CCW) y “mergearlas” como en merge sort; el resultado es un poligono CCW sin repetir el ultimo punto. La idea: el resultado tiene una arista paralela a cada arista de los originales; la suma se obtiene barriendo los angulos.

Propiedades clave (invariantes).

- Si A, B son convexos con n, m vertices, $A \oplus B$ tiene $\leq n + m$ vertices.
- Util para **detectar colision**: A colisiona con B si $0 \in A \oplus (-B)$.
- Asume poligonos en orden **counter-clockwise**.
- El resultado es convexo (suma de convexos es convexo).

Codigo clave. La suma de Minkowski:

```
vector<Point> minkowski(const vector<Point>& A, const vector<Point>& B) {
    vector<Point> result;
    int i = 0, j = 0;
    int n = A.size(), m = B.size();
    do {
        result.push_back(A[i] + B[j]);
        long long cross = (A[(i+1)%n] - A[i]) ^ (B[(j+1)%m] - B[j]);
        if (cross >= 0) i = (i + 1) % n;
        if (cross <= 0) j = (j + 1) % m;
    } while (i || j);
    return result;
}
```

Complejidad.

- $O(n + m)$.
- Memoria $O(n + m)$.

Donde tocar para modificar.

- **Deteccion de colision:** anadir $(-B)$ (reflejar) y verificar si 0 esta en el resultado.
- **Poligonos no convexos:** la suma puede no ser poligono simple; algoritmo distinto.
- **Multiples queries:** precomputar las aristas una vez.
- **Robot motion planning:** usar para planear trayectorias.

Trampas y errores frecuentes.

- Si los poligonos no son CCW, la suma es incorrecta. Normalizar antes.
- El bucle termina cuando ambos indices vuelven a 0; anadir check.
- Para la deteccion de colision, anadir el test 0 in $A \hat{\cup} (-B)$.

Codigo. Ver `cpp.tex`, seccion *Minkowski sum (convex polygons)*.

7.3 Herramientas

7.1 Li Chao Tree (long long)

Idea intuitiva. Mantiene un **lower envelope** (o upper) de un conjunto de rectas $y = mx + b$ sobre un dominio $[X_{LO}, X_{HI}]$ y responde “minimo y en un x dado” en $O(\log X)$. La estructura es un segment tree donde cada nodo guarda la recta “mejor” para su segmento medio; al anadir una recta, se compara con la actual y se decide cual es mejor en cada mitad, descendiendo recursivamente. La demostracion: en cada nivel, solo se guarda la mejor recta para el “punto medio”; las otras se transmiten a los hijos.

Propiedades clave (invariantes).

- **Online:** cada `addLine` en $O(\log X)$.
- Solo insercion; no hay borrar (para borrar, usar Li Chao segmentado con reset).
- La recta guardada en un nodo es la “mejor” en su punto medio.
- Las rectas “perdedoras” se transmiten a los hijos para futuros inserts.

Codigo clave. La insercion en Li Chao:

```
void addLine(Line nw, int v = 1, int l = X_LO, int r = X_HI) {
    int m = (l + r) / 2;
    bool left = nw.get(l) < tree[v].get(l);
    bool mid = nw.get(m) < tree[v].get(m);
    if (mid) swap(tree[v], nw);
    if (r - l == 1) return;
    if (left != mid) addLine(nw, v*2, l, m);
    else              addLine(nw, v*2+1, m, r);
}
```

Complejidad.

- $O(\log X)$ por insercion y query, donde X es el tamano del dominio.
- Memoria $O(\log X)$ por insercion, $O(X)$ total.

Donde tocar para modificar.

- **Upper envelope en vez de lower:** invertir el signo ($>$ en vez de $<$).
- **Queries de segmento** (no full line): si la recta solo esta activa en un rango, anadir bandera **active**.
- **Eliminar rectas:** usar Li Chao segmentado (mas complejo).
- **Pendientes grandes:** si las pendientes tienen magnitud $> 10^9$, anadir `__int128` para evitar overflow.

Trampas y errores frecuentes.

- El parametro `lo == hi` debe retornar el valor del nodo (caso base).
- La comparacion `newLine.get(mid) < node->line.get(mid)` decide quien es mejor en el medio.
- Si las rectas tienen m iguales pero b distintos, se queda con la primera (tie-break).

Codigo. Ver `cpp.tex`, seccion *Li Chao Tree (long long)*.

7.2 Sweep line template

Idea intuitiva. Patron de diseno para problemas geometricos donde se barre una linea sobre el plano y se mantiene una estructura de datos sobre el otro eje. Eventos (add, query, remove) se ordenan por la coordenada de barrido y se procesan secuencialmente.

Propiedades clave (invariantes).

- Eventos: tuplas $(x, \text{type}, \text{id})$.
- La estructura auxiliar (set, multiset, segtree) se mantiene sobre el otro eje.
- Procesar eventos en orden de x ascendente.

Complejidad.

- Depende del problema; tipico $O((N + E) \log N)$.

Donde tocar para modificar.

- **Anadir/quitar segmentos:** anadir eventos add/remove al barrido.
- **Anadir queries:** anadir eventos query.
- **Cambiar el eje de barrido:** si barres en y en vez de x , anadir comparador.

Trampas y errores frecuentes.

- El snippet es solo el **esqueleto**; los handlers `add/query/remove` se llenan por problema.
- Cuidado con eventos duplicados.

Codigo. Ver `cpp.tex`, seccion *Sweep line template*.

7.3 Closest pair of points

Idea intuitiva. Encuentra el par de puntos mas cercano en $O(n \log n)$ con **divide y venceras**. Algoritmo:

- Sort por x .
- Dividir en mitades; recursivo.
- Combinar: dado el minimo d de las dos mitades, strip = puntos en $[midX - d, midX + d]$; ordenar por y ; cada punto solo se compara con los siguientes hasta y difiera en $< d$ (maximo 7).

Propiedades clave (invariantes).

- Requiere ordenar por y despues (en el snippet se hace durante el merge).
- Strip tiene a lo sumo $O(n)$ puntos en total, pero la comparacion es $O(1)$ amortizada por punto.
- Resultado en `double`.

Complejidad.

- $O(n \log n)$.

Donde tocar para modificar.

- **Reconstruir el par:** aniadir indices en la recursion.
- **Distancia maxima:** cambiar `min` por `max` (mas facil; no requiere D&C).
- **Puntos duplicados:** aniadir check especial (distancia 0).

Trampas y errores frecuentes.

- El merge requiere ordenar por y ; el snippet lo hace in-place con un `tmp`.
- Si se hace en cada recursion, es $O(n \log n)$ extra.

Codigo. Ver `cpp.tex`, seccion *Closest pair of points*.

7.4 Rotating calipers (polygon diameter)

Idea intuitiva. Encuentra el **diametro** de un poligono convexo (par de vertices mas distantes) en $O(n)$. Asume poligono en orden counter-clockwise. Algoritmo: dos indices i (sobre las aristas) y k (sobre los vertices opuestos); en cada paso, rotar k mientras el area del paralelogramo aumenta. La distancia $i-k$ puede ser maxima en cualquier momento del barrido.

Propiedades clave (invariantes).

- Cada indice avanza como maximo n veces total.
- El “diametro” es la maxima distancia entre dos vertices.
- Retorna **distancia al cuadrado**.

Complejidad.

- $O(n)$.

Donde tocar para modificar.

- **Reconstruir el par de vertices:** aniadir indices.
- **Anchura minima / maxima:** variantes de calipers.
- **Distancia minima entre poligonos convexos:** otra variante.

Trampas y errores frecuentes.

- Asume poligono **estrictamente** convexo (sin vertices colineales). Si hay colineales, aniadir perturbacion o filtrar.

Codigo. Ver `cpp.tex`, seccion *Rotating calipers (polygon diameter)*.